

멀티모달 인지 기반 피지컬 AI 로봇 시스템 구현 과정

1일차 강의자료

AI 로봇 개론 · ESP32 하드웨어 및 MicroPython 기초

외부 모듈 연결 없이 ESP32 · 포니봇 보드의 내장 LED · 스위치 · 센서를 활용한 실습

5일 과정

1일 7시간

이론 3H + 실습 4H

5일 과정 한눈에 보기 · 오늘은 1일차

1일차

AI 로봇 개론 + ESP32·MicroPython 기초

2일차

포니봇 모션 제어 (메카넘 주행·서보)

3일차

센서 기반 자율주행 (초음파·라인)

4일차

AI 웹앱 연동 (객체·음성·포즈 인식)

5일차

미니 프로젝트 & 발표

※ 매일 7시간(이론+실습) · 하드웨어: ESP32(Mbites) + AI 포니봇

[01] 3H

AI 로봇 개론

AI 로봇의 정의·발전 역사·산업 동향·분류·핵심 기술을 알아봅니다. (이론)

AI 로봇이란?

어원 1920년 체코 희곡 「R.U.R.」의 'robota(고된 노동)'에서 유래했습니다.

로봇 (Robot)

명령·프로그램에 따라 작업을 수행하는 기계 장치

AI 로봇

센서로 환경을 인식하고, 데이터를 학습·판단해 스스로 행동하는 로봇

로봇을 이루는 3요소

감지

센서로 환경·상태 정보 수집

>

판단

수집한 데이터를 해석·결정

>

동작

모터·LED 등으로 실행

로봇의 동작 원리 — 감지·판단·동작의 순환



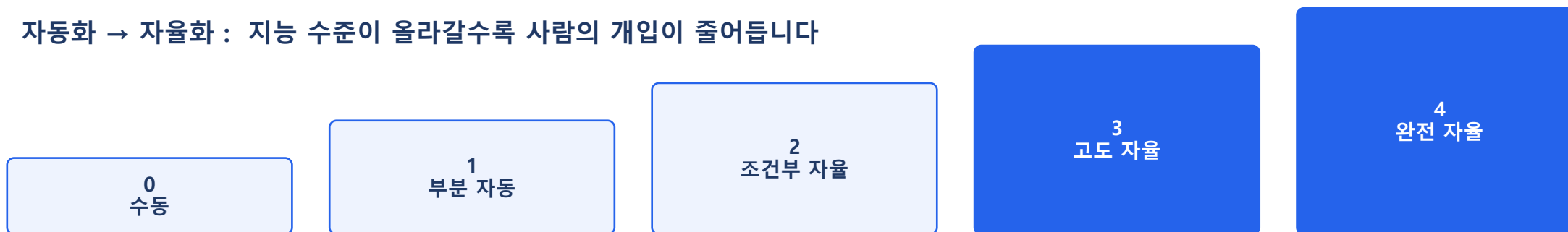
이 과정을 1초에도 수십 번 '매우 빠르게 반복'하며 환경에 실시간으로 반응합니다.

예) 자율주행 — 카메라로 장애물 '감지' → 피할 경로 '판단' → 바퀴 조향·감속 '동작'

전통 로봇 vs AI 로봇

구분	전통(산업) 로봇	AI 로봇
제어 방식	정해진 순서를 반복	상황을 인식해 실시간 판단
환경 변화	취약 (재프로그래밍 필요)	스스로 적응
학습	거의 없음	센서 데이터로 학습·개선
대표 기술	시퀀스 제어	머신러닝 · 비전 · SLAM
예시	용접·도장 로봇	자율주행·서비스·협동 로봇

자동화 → 자율화 : 지능 수준이 올라갈수록 사람의 개입이 줄어듭니다



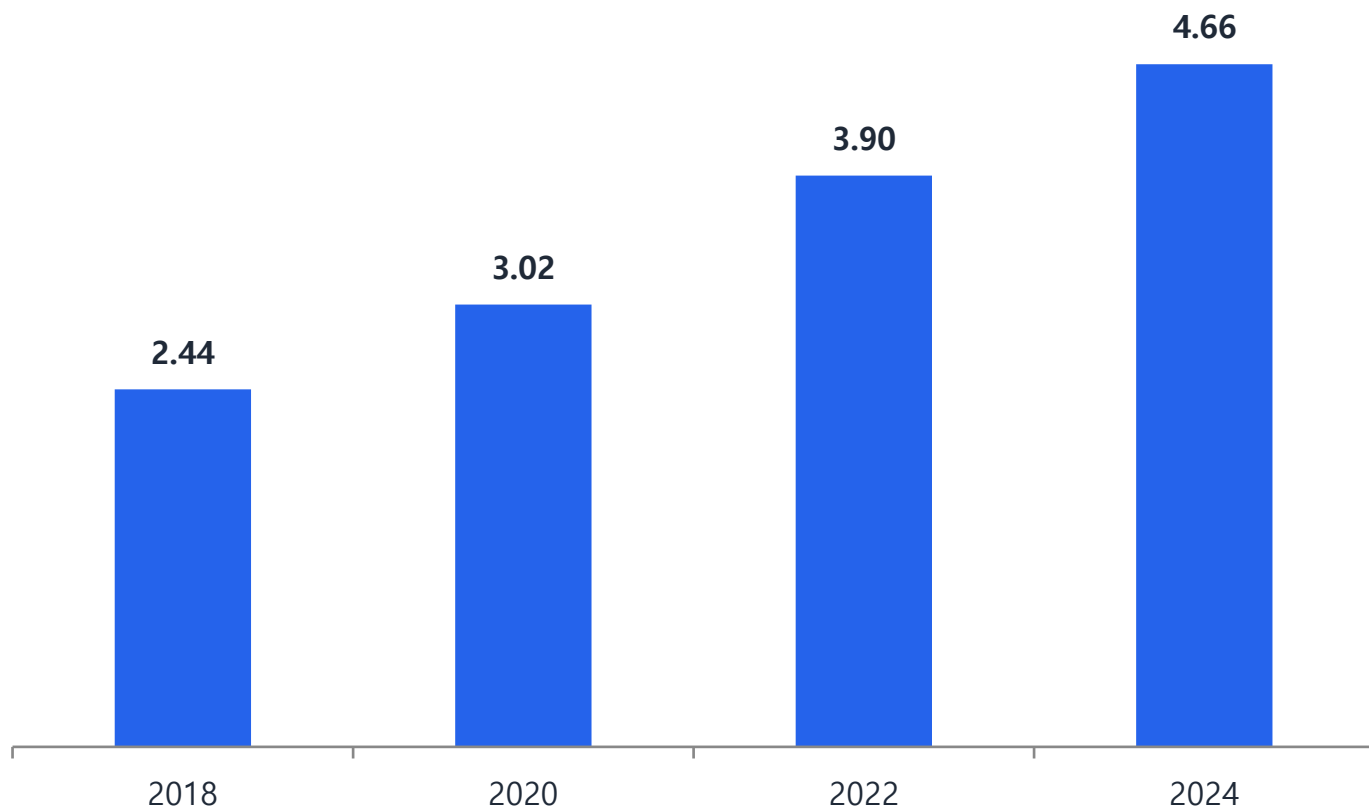
로봇의 발전 역사



전통적 '자동화'에서 데이터로 학습하는 '지능형 로봇(피지컬 AI)'으로 발전해 왔습니다.

로봇 산업 동향 ① — 빠른 성장

전 세계 산업용 로봇 누적 운용 대수 (백만 대)



466만 대

2024년 누적 운용 (+9%)

54만 대

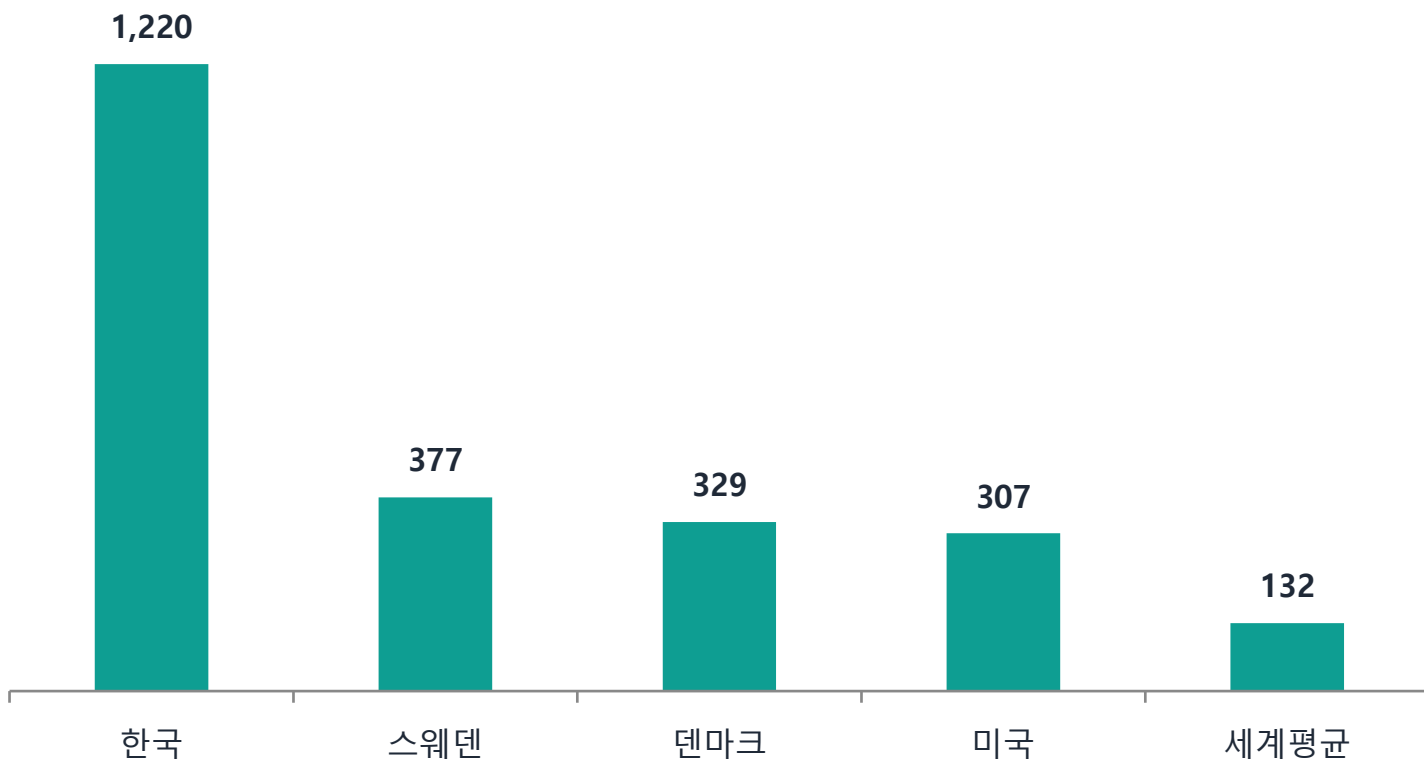
2024년 연간 설치 (역대 2위)

2배 ↑

최근 10년간 증가

로봇 산업 동향 ② — 한국이 세계 1위

제조업 로봇 밀도 (직원 1만 명당 로봇 수, 2024)



한국 1,220대

세계에서 로봇을 가장 많이 쓰는 나라! (세계 평균의 약 9배)

시사점

자동화 확산 → 로봇·AI 인재 수요 급증. 여러분이 배우는 기술이 곧 산업 현장의 기술!

로봇의 적용 분야

제조 · 산업

용접·조립·검사 자동화
스마트 팩토리

물류 · 유통

창고 자동화(AMR)
배달·분류 로봇

서비스

서빙·안내·응대
청소 로봇

의료 · 헬스케어

수술 보조·재활
돌봄 로봇

가정

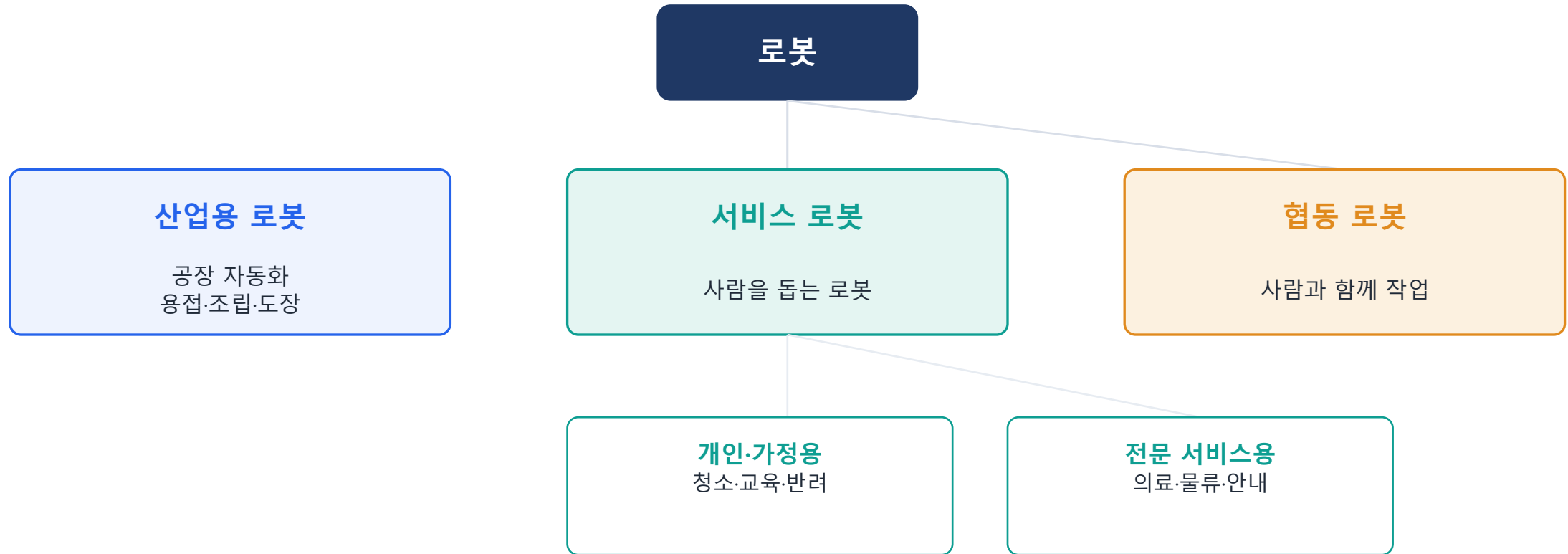
로봇청소기
교육·반려 로봇

모빌리티

자율주행차
순찰·배송 로봇

전문 서비스 로봇 설치의 52%가 운송·물류 분야 (IFR 2024) — 가장 빠르게 성장 중

로봇의 분류 체계



이동 방식으로는 AGV(지정 경로)와 AMR(스스로 경로 판단)로도 나뉩니다 → 다음 슬라이드

산업용 · 서비스 · 협동 로봇

산업용 로봇

- 정해진 작업을 빠르고 정밀하게 반복
- 안전을 위해 보통 사람과 공간 분리
- 예) 자동차 용접·도장 라인

서비스 로봇

- 사람을 돕는 일상/전문 작업
- 이동·인식 능력이 중요
- 예) 서빙·청소·안내·배달

협동 로봇(Cobot)

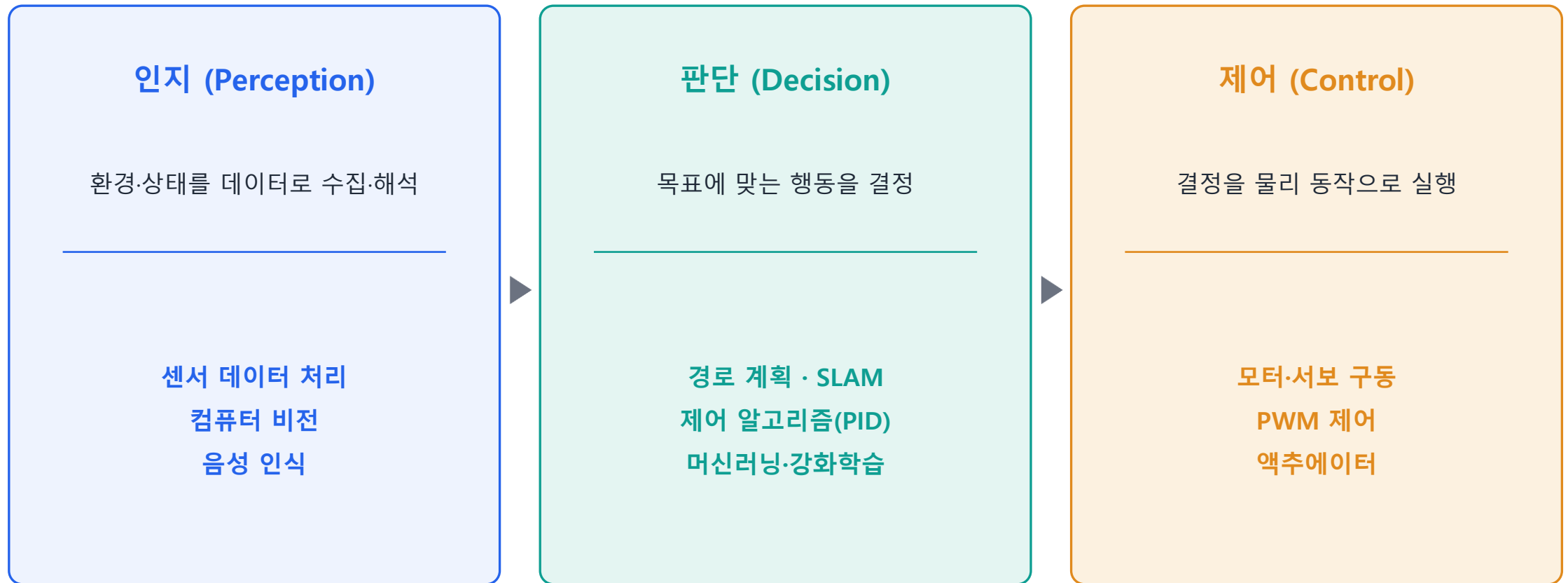
- 사람과 같은 공간에서 협업
- 충돌 감지로 안전 확보
- 예) 조립 보조·검사 코봇

이동 로봇: AGV vs AMR

AGV 바닥의 선·마커 등 '지정된 경로'만 따라 이동 (단순·저비용)

AMR 센서로 주변을 인식해 '스스로 경로 판단' (AI 적용) — 우리 포니봇이 여기에 해당!

AI 로봇 핵심 기술



세 가지가 끊임없이 순환하며 로봇이 '스스로 일하게' 만듭니다.

핵심 기술이 우리 수업으로

오늘 배우는 ESP32·포니봇 실습이 바로 이 기술들의 '기초 체험'입니다.

① 인지 · Perception

내장 마이크·가속도·온도 / 초음파·라인 센서로 '정보 수집'

② 판단 · Decision

조건문·임계값 처리 / AI 웹앱(객체·음성·포즈)으로 '판단'

③ 제어 · Control

NeoPixel·PWM / 메카닉 모터·서보로 '동작 실행'

→ 1일차에는 '인지'와 '제어'의 기초(LED·센서)를, 2일차부터 '판단·구동'으로 확장합니다.

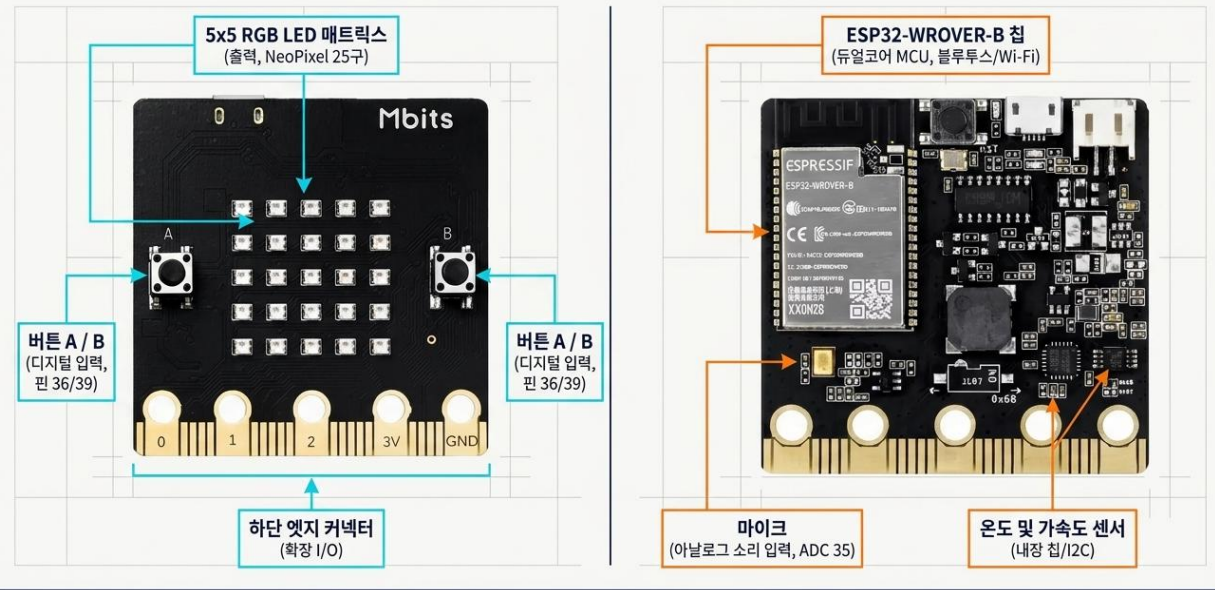
[02] 4H

ESP32 하드웨어 및 MicroPython

Thonny 설치 → 보드 연결 → 펌웨어 설치 → 내장 LED·스위치·센서 제어 & 복합 실습.

ESP32 보드(포니봇)와 내장 구성

핵심 신경망: ESP32 (Mbits) 메인보드



오늘 쓰는 '내장' 주변장치 (외부 연결 X)

5x5 RGB LED 매트릭스 출력 · NeoPixel 25구(GPIO13)

버튼 A·B 디지털 입력(36·39)

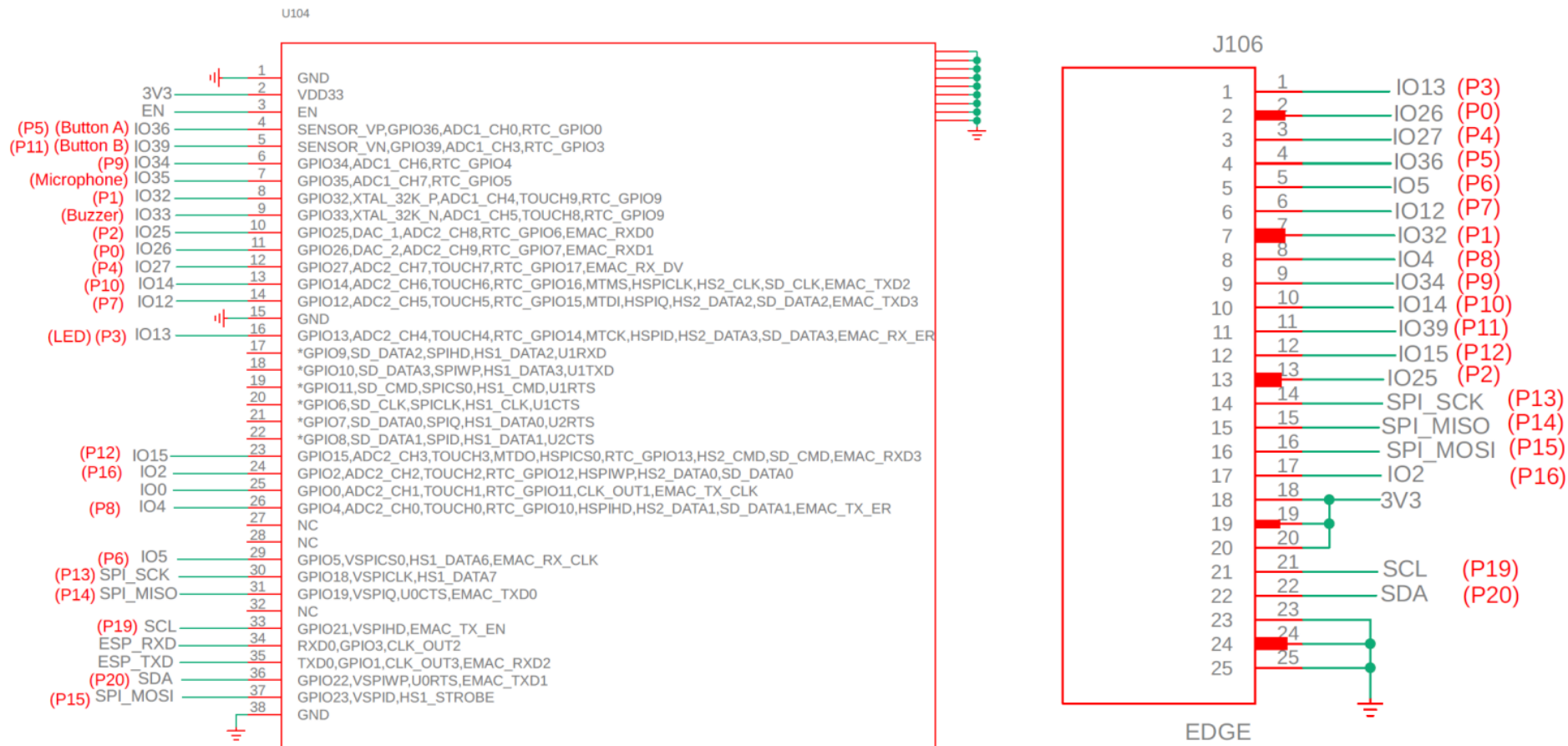
마이크 소리(아날로그) · ADC 35

가속도 센서 기울기·흔들림 · I2C

온도 센서 ESP32 칩 내장 온도

스피커 소리 출력

ESP32 하드웨어 인터페이스



개발 환경: Thonny · 보드 연결 · 펌웨어

1

Thonny 설치

thonny.org 에서 OS에 맞는 버전을 받아 설치합니다.

2

ESP32 보드 연결

USB 케이블로 PC와 연결. (장치 관리자에서 COM 포트 확인)

3

MicroPython 펌웨어 설치

Thonny → 도구 → 옵션 → 인터프리터 → MicroPython(ESP32) 펌웨어 설치.

4

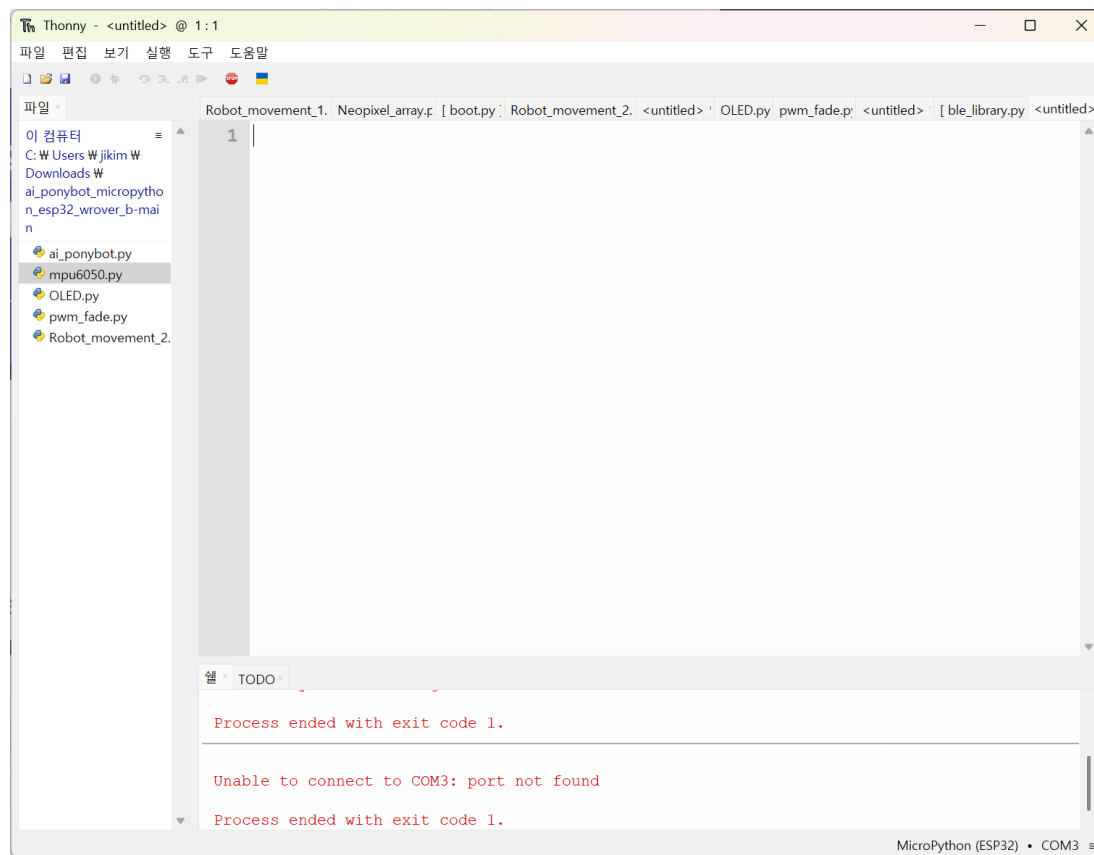
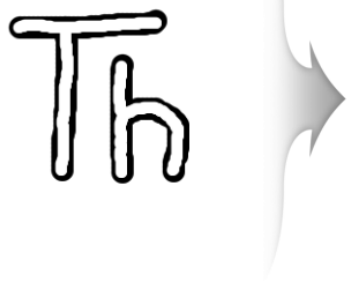
인터프리터·포트 지정

인터프리터: MicroPython(ESP32) / 포트: USB Serial @ COMx 선택.

TIP 셸에 >>> 프롬프트가 보이면 연결 성공! (재시작 시 'MPY: soft reboot' 표시)

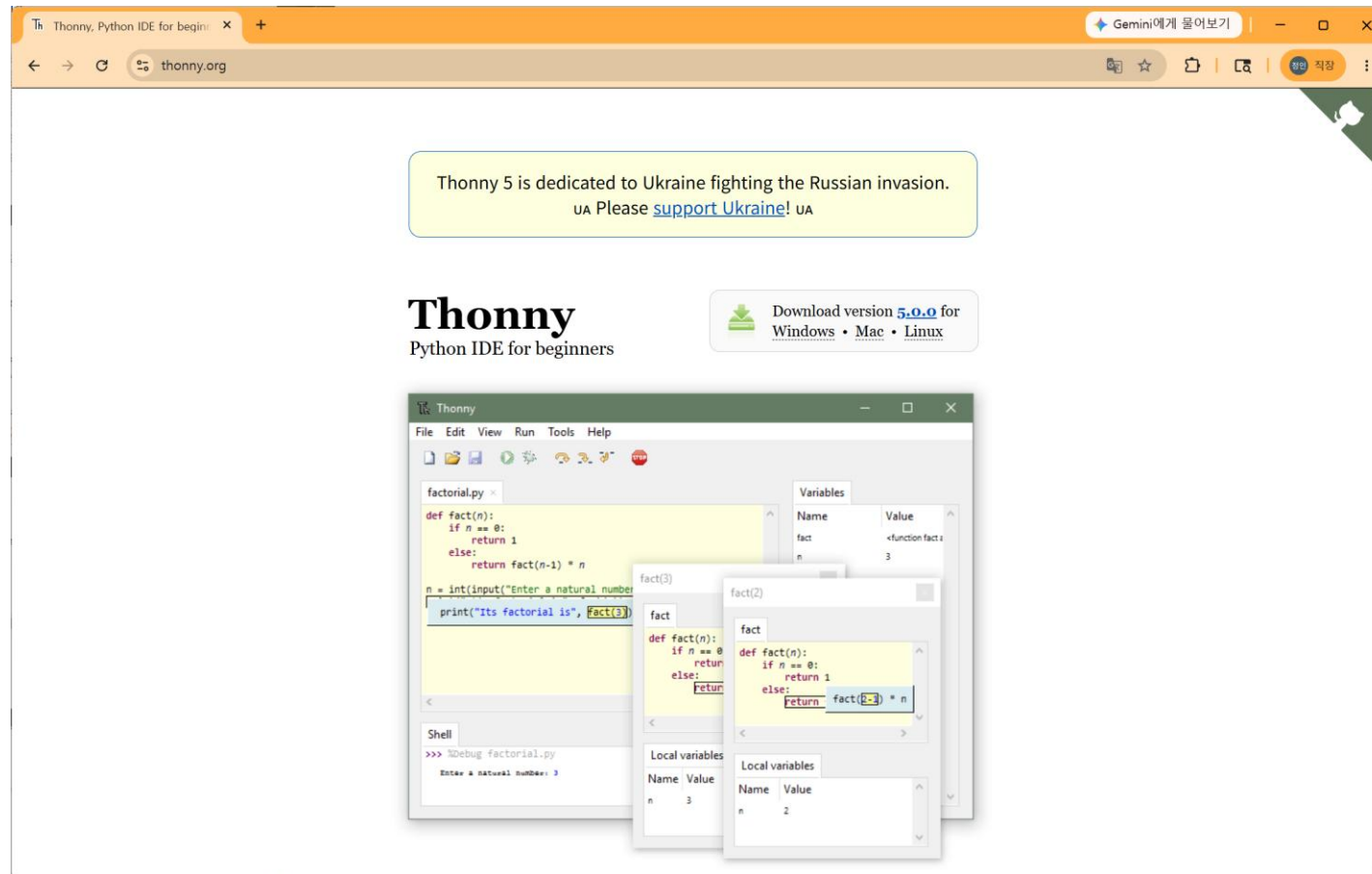
Thonny (통합 개발 환경, IDE)

- Thonny는 초급자와 교육적 환경을 대상으로 설계된 파이썬용 통합 개발 환경입니다.
- Thonny를 활용해 파이썬으로 다양한 마이크로 컨트롤러 보드를 제어할 수 있습니다.



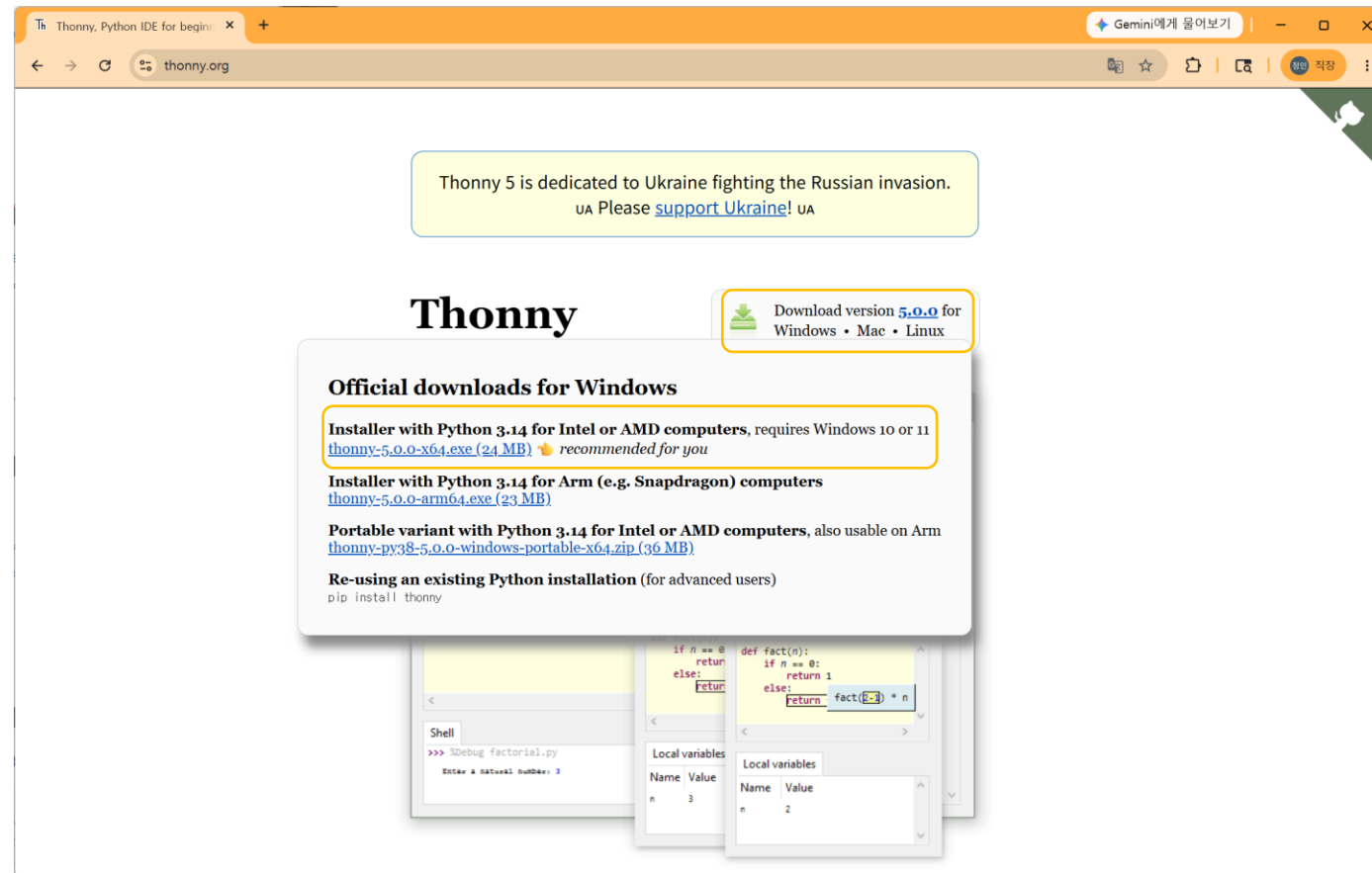
Thonny 설치(1/4)

1. Thonny를 설치하기 위해 홈페이지(<https://thonny.org/>)에 접속합니다.



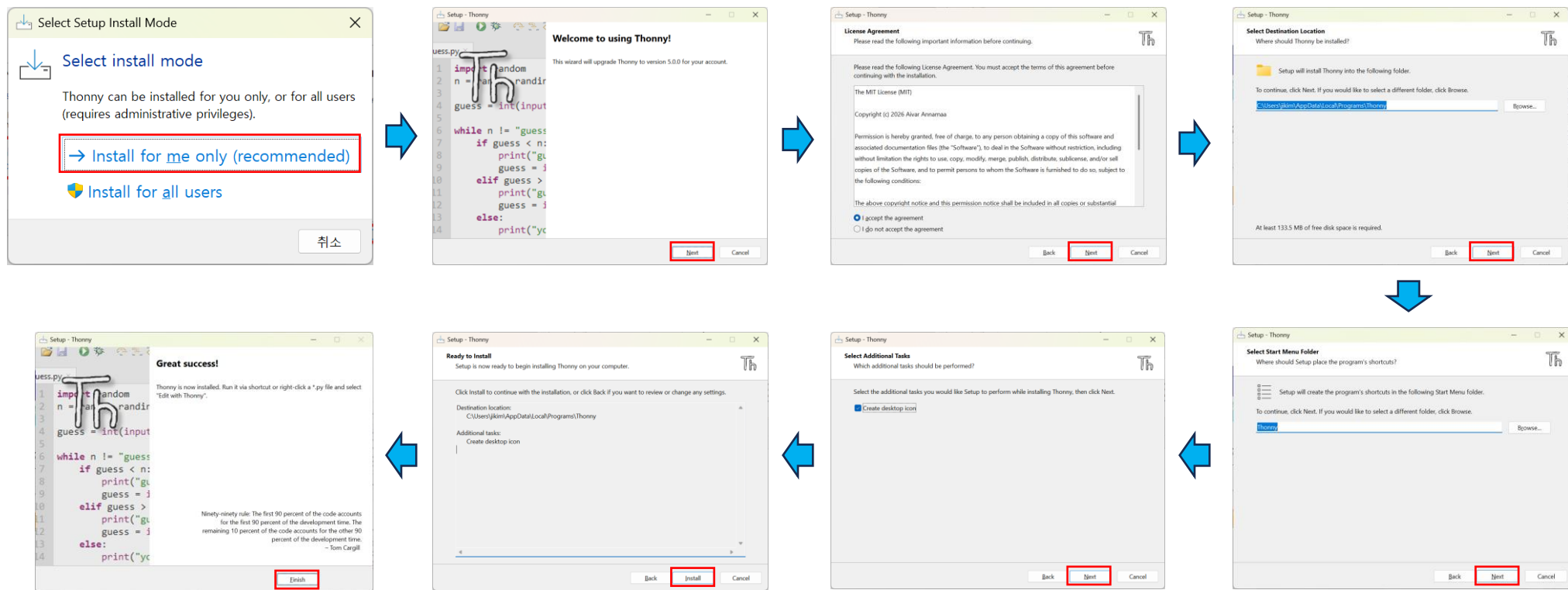
Thonny 설치(2/4)

2. 실습 PC에 해당하는 운영체제를 선택합니다.



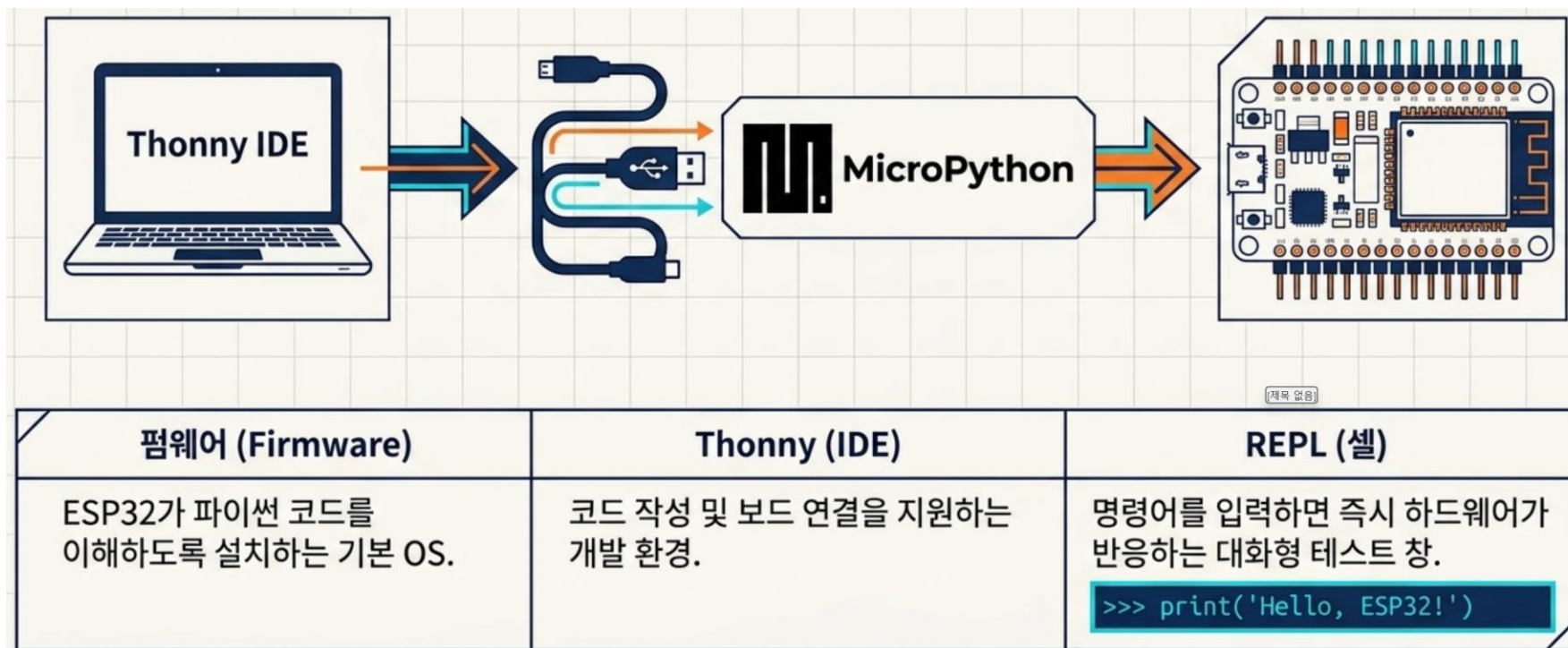
Thonny 설치(3/4)

3. 다운로드 받은 설치파일을 실행하여 설치를 진행합니다.(현재 최신 버전 - 5.0.0 버전)



ESP32 보드 연결

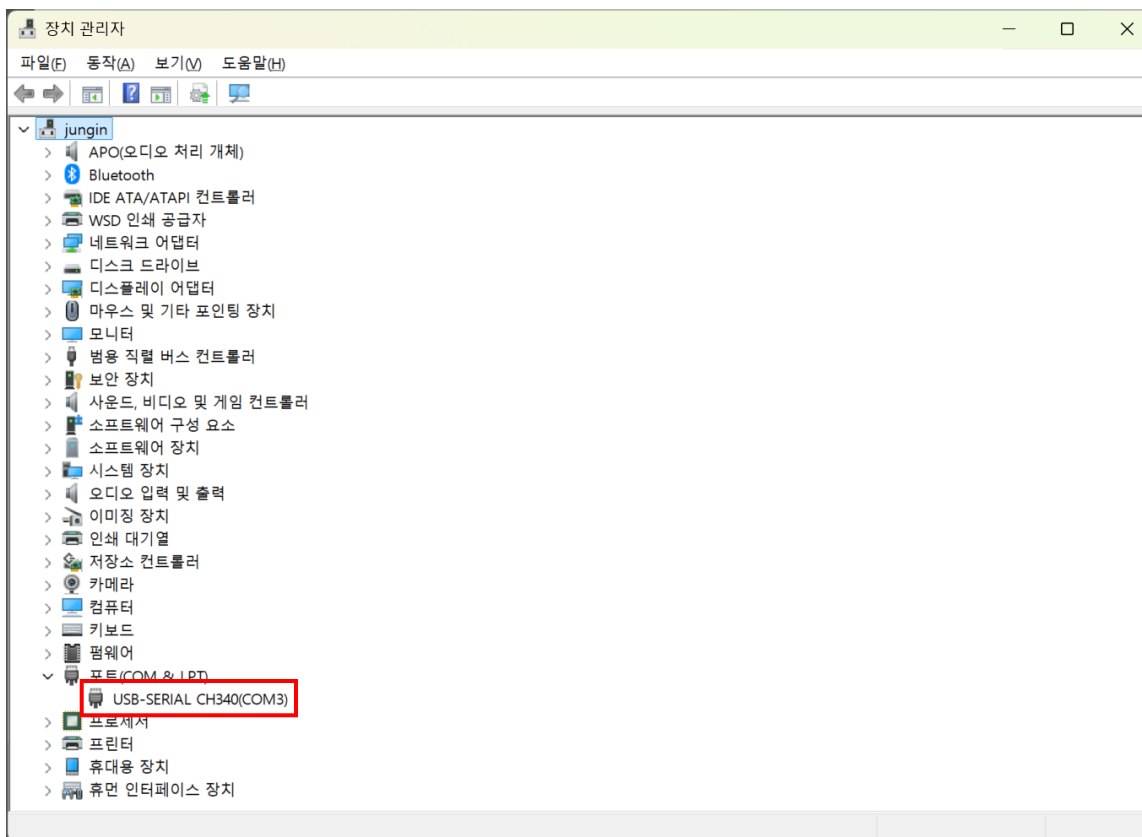
- PC와 ESP32 보드를 USB 케이블로 연결해 줍니다.
- ESP32 보드 전원 인디케이터가 정상적으로 작동하는지 확인합니다.



ESP32 보드 연결

- 장치가 올바르게 인식되었다면 ESP32 보드 연결 시 장치관리자에서 COM포트 번호를 확인할 수 있습니다.

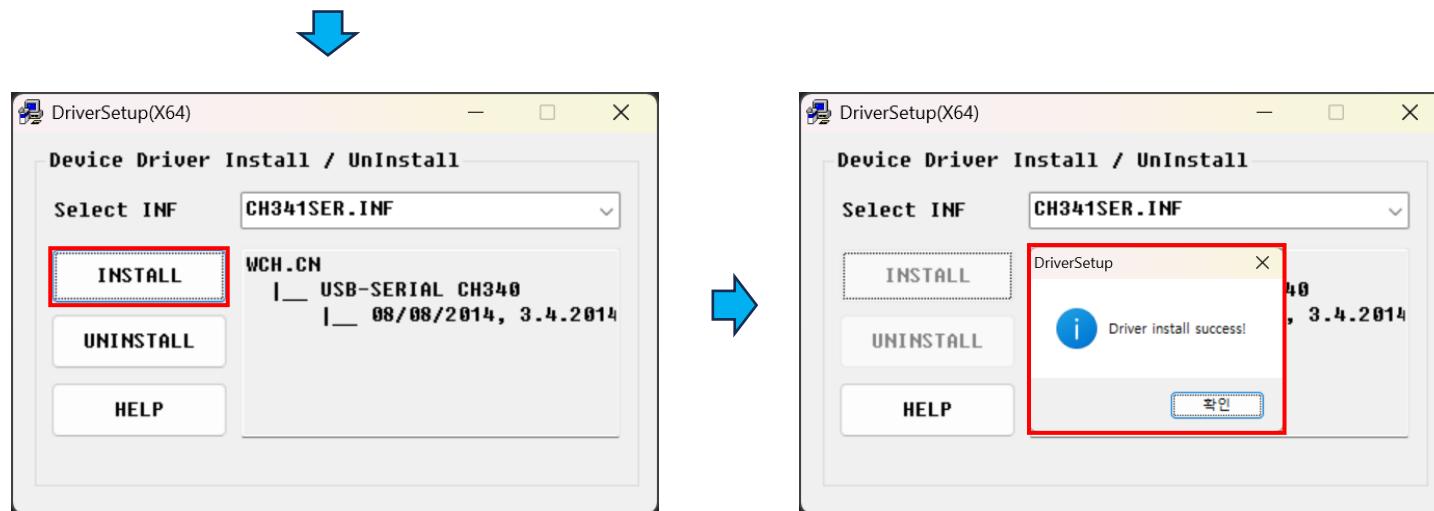
※ 장치가 올바르게 인식되지 않을 경우, [다운로드링크](#)를 통해 드라이버(CH340)를 설치



ESP32 드라이버 설치

압축 해제

 CH34x_Install_Windows_v3_4.zip →  CH34x_Install_Windows_v3_4.EXE

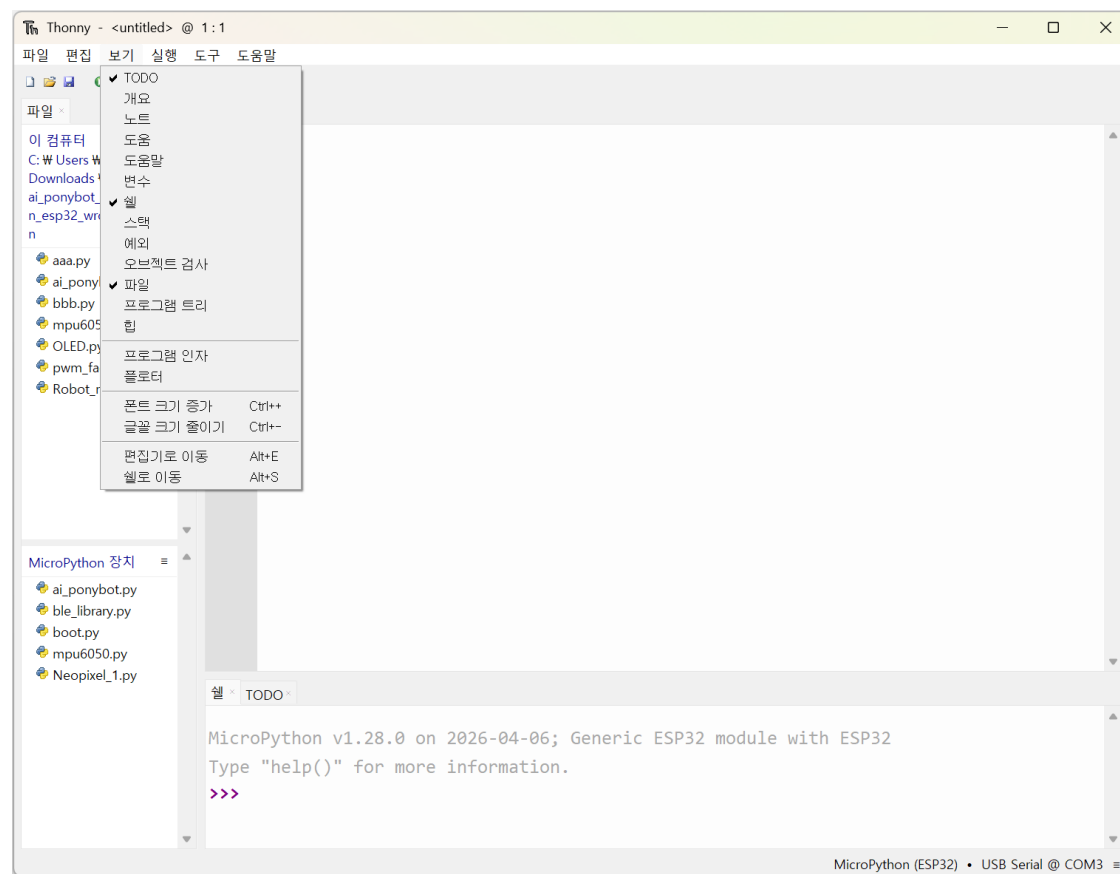


압축을 해제한 후, CH34x_Install_Windows_v3_4.exe 파일을 실행하여 장치 드라이버 설치를 진행합니다.

ESP32 드라이버 설치

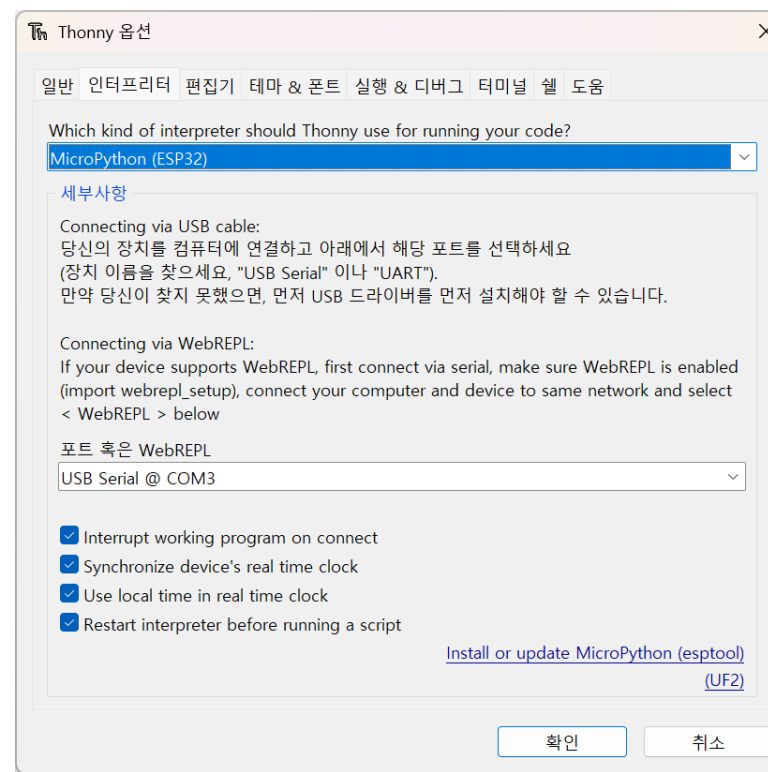
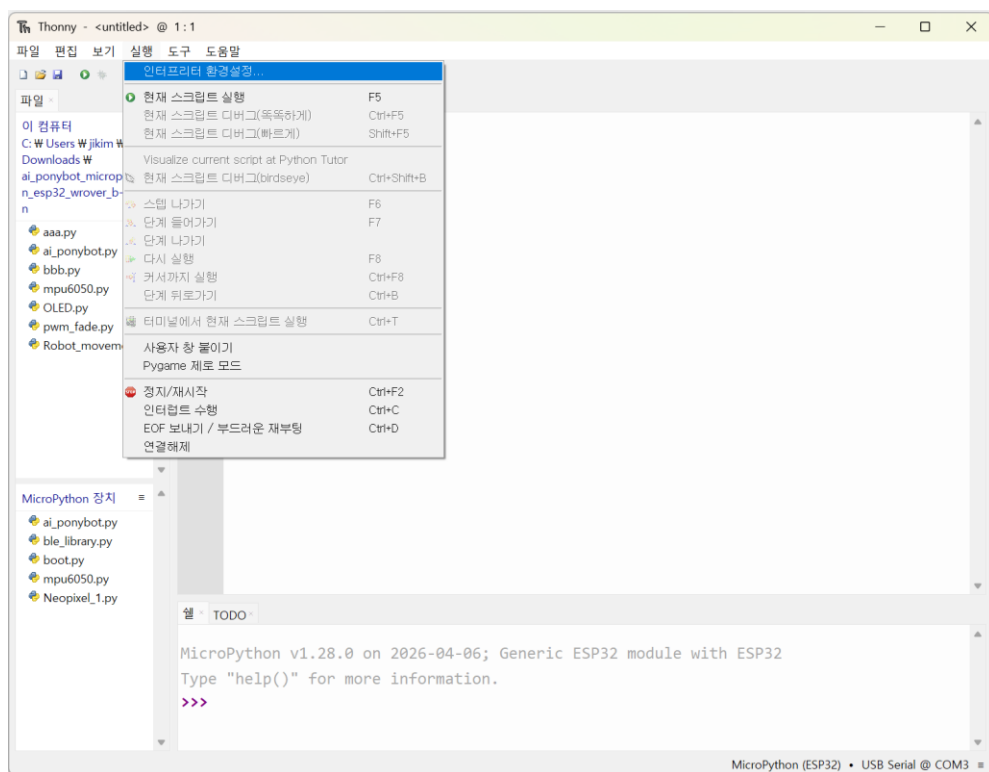
1. [보기] – [TODO], [셸], [파일]을 체크합니다.

프로그래밍 효율을 높이기 위해 인터페이스(기능창)를 설정하는 과정입니다



마이크로파이썬 펌웨어 설치

2. [실행] - [인터프리터 환경설정]을 클릭합니다.



마이크로파이썬 펌웨어 설치

3. https://micropython.org/download/ESP32_GENERIC/

홈페이지에 접속하여 펌웨어 최신 버전(Support for SPIRAM / WROVER)을 다운 받습니다.

Firmware (Support for SPIRAM / WROVER)

Releases

v1.28.0 (2026-04-06) .bin / [.app-bin] / [.elf] / [.map] / [Release notes] (latest)
 v1.27.0 (2025-12-09) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.26.1 (2025-09-11) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.26.0 (2025-08-09) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.25.0 (2025-04-15) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.24.1 (2024-11-29) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.24.0 (2024-10-25) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.23.0 (2024-06-02) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.22.2 (2024-02-22) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.22.1 (2024-01-05) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.22.0 (2023-12-27) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.21.0 (2023-10-05) .bin / [.app-bin] / [.elf and .map] / [Release notes]
 v1.20.0 (2023-04-26) .bin / [.elf and .map] / [Release notes]
 v1.19.1 (2022-06-18) .bin / [.elf and .map] / [Release notes]
 v1.18 (2022-01-17) .bin / [.elf and .map] / [Release notes]
 v1.17 (2021-09-02) .bin / [.elf and .map] / [Release notes]
 v1.16 (2021-06-23) .bin / [.elf and .map] / [Release notes]
 v1.15 (2021-04-18) .bin / [.elf and .map] / [Release notes]
 v1.14 (2021-02-02) .bin / [.elf and .map] / [Release notes]
 v1.13 (2020-09-02) .bin / [.elf and .map] / [Release notes]
 v1.12 (2019-12-20) .bin / [.elf and .map] / [Release notes]

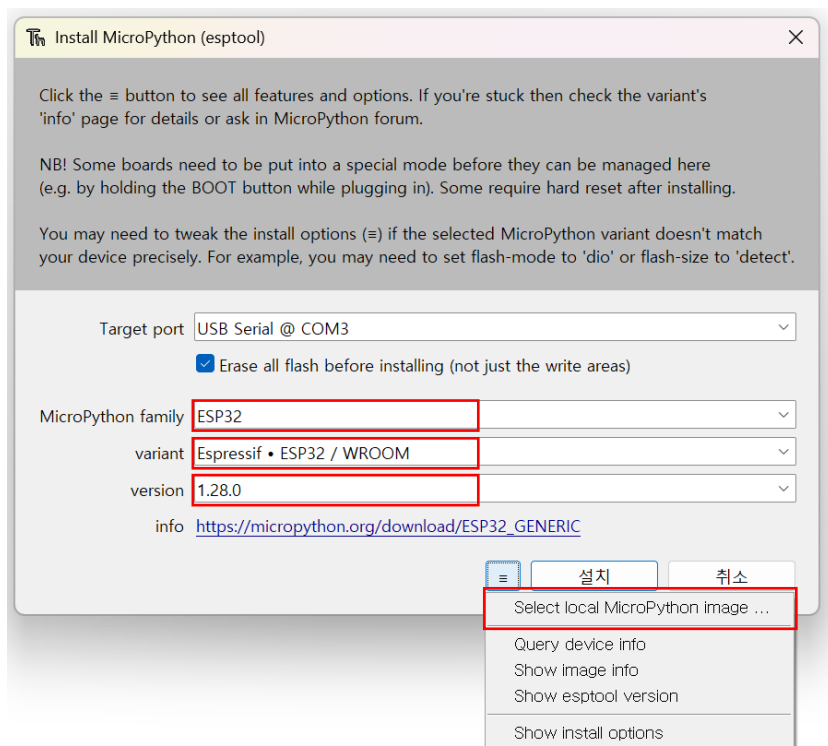
Preview builds

These are automatic builds of the development branch for the next release.

v1.29.0-preview.337.g44a569b637 (2026-05-29) .bin / [.app-bin] / [.elf] / [.map]
 v1.29.0-preview.317.g4fd7295703 (2026-05-28) .bin / [.app-bin] / [.elf] / [.map]
 v1.29.0-preview.278.gdc33f04c1b (2026-05-20) .bin / [.app-bin] / [.elf] / [.map]
 v1.29.0-preview.245.g1c63211817 (2026-05-15) .bin / [.app-bin] / [.elf] / [.map]

마이크로파이썬 펌웨어 설치

4. 다시 펌웨어 설치 화면으로 넘어와서 설치 버튼 옆에 ≡ 버튼을 눌러 'Select local MicroPython Image' 메뉴를 선택합니다.



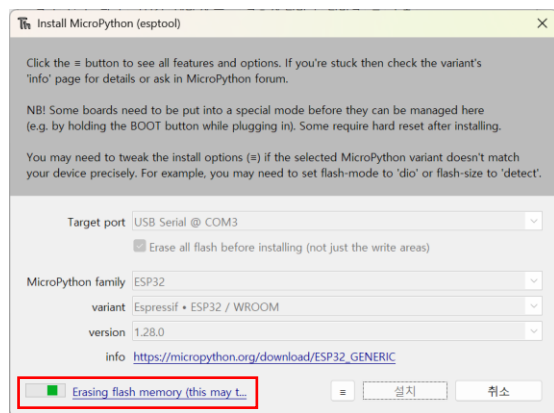
ESP32_GENERIC-SPIRAM-20260406-v1.28.0.bin

다운 받은 펌웨어 선택

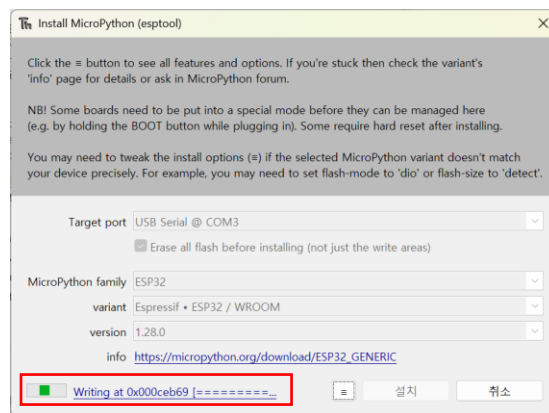
마이크로파이썬 펌웨어 설치

5. 펌웨어 설치가 완료되었다면 닫기 버튼을 클릭해 펌웨어 설치 과정을 종료합니다.

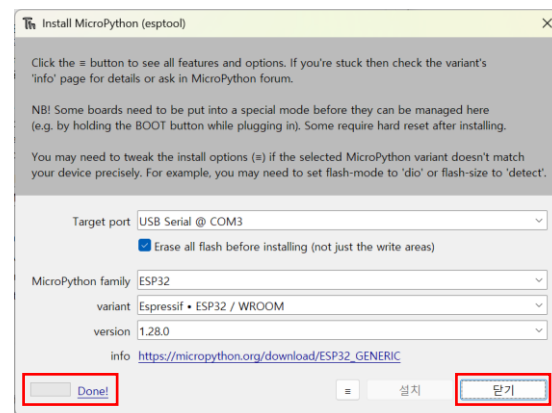
※ 펌웨어 설치 는 최초 1회만 진행하면 됩니다.



메모리 삭제



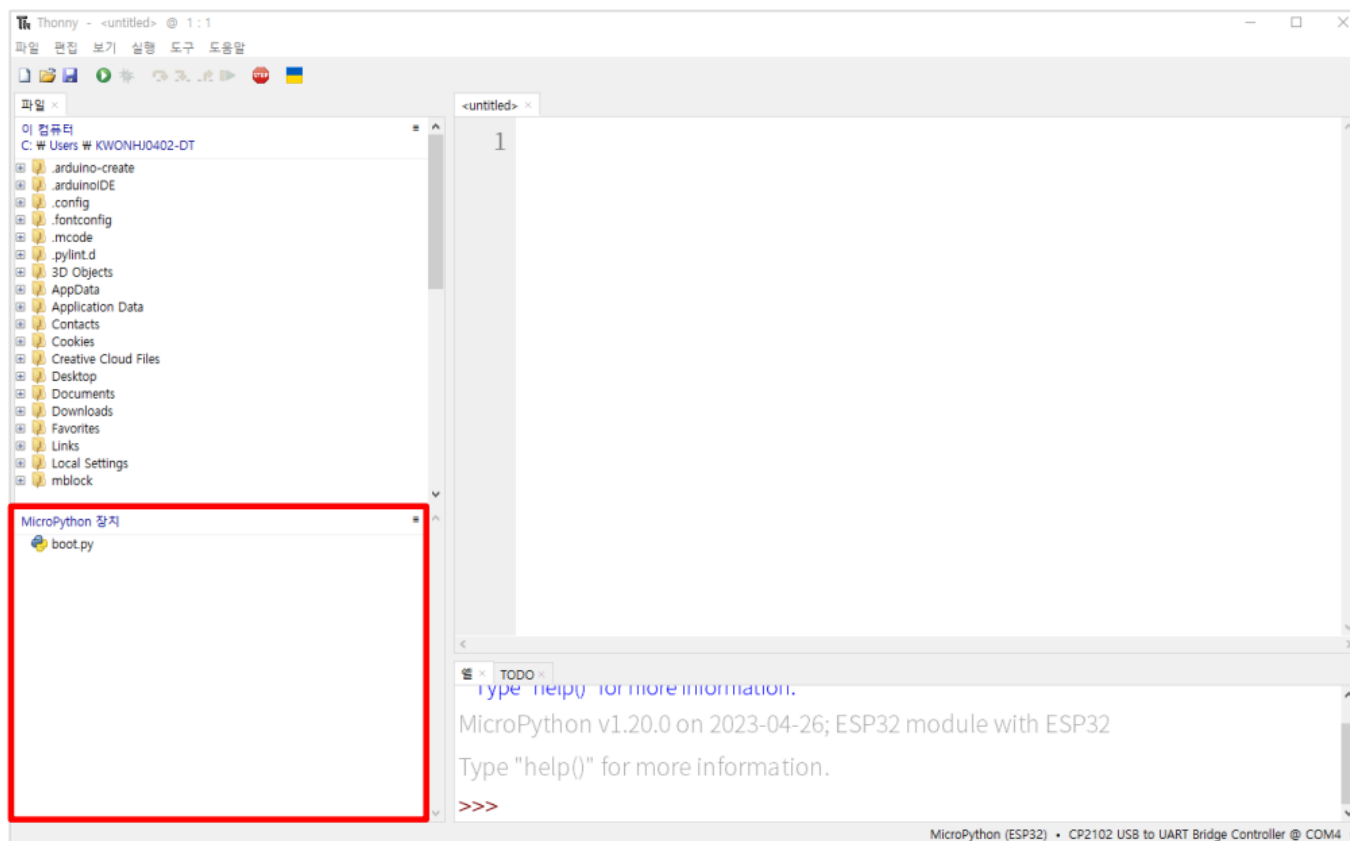
펌웨어 설치 중



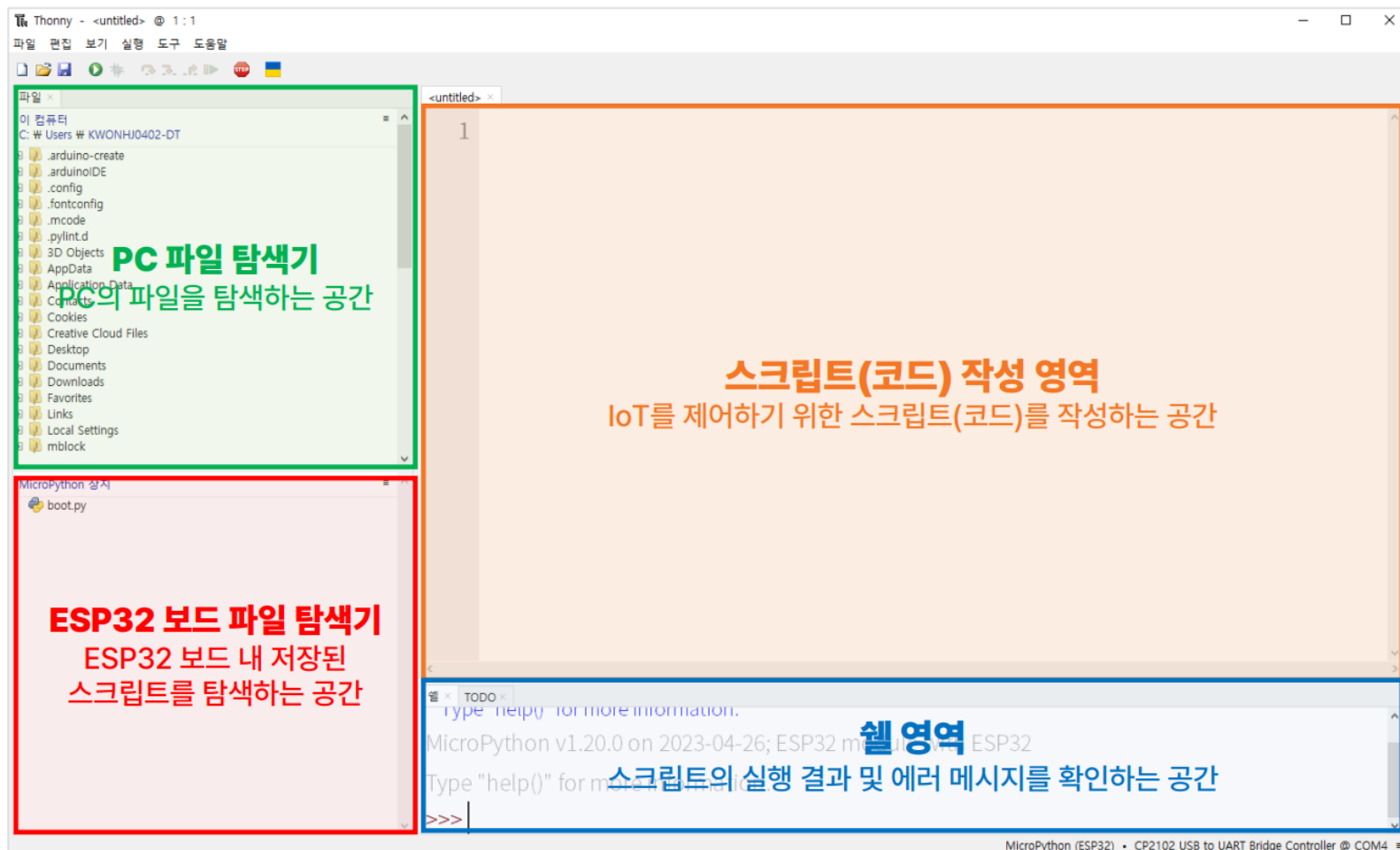
완료

마이크로파이썬 펌웨어 설치

6. 사진과 같이 MicroPython 장치가 인식되었다면 펌웨어가 정상적으로 설치된 것입니다.



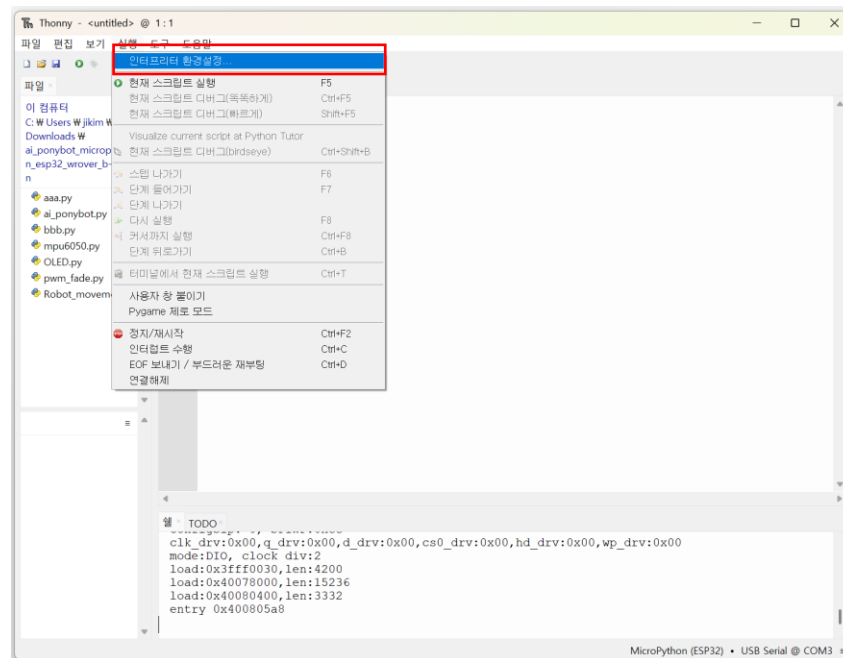
마이크로파이썬 펌웨어 설치



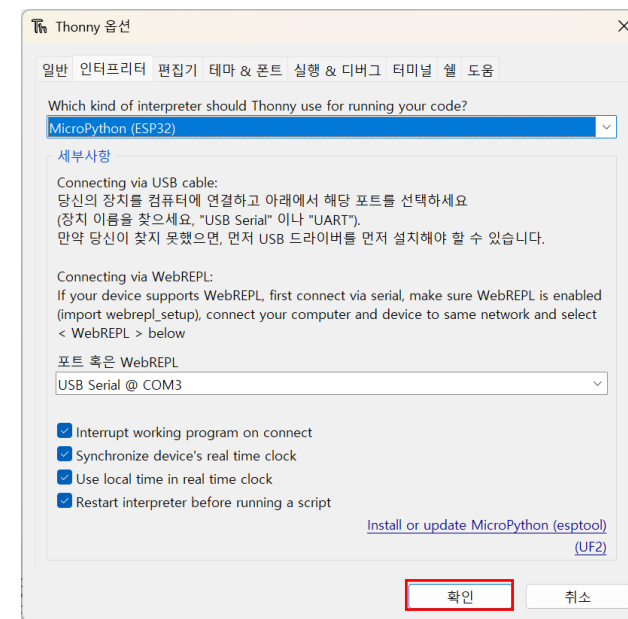
Thonny <-> ESP32 연결



1. PC에 ESP32 보드를 연결

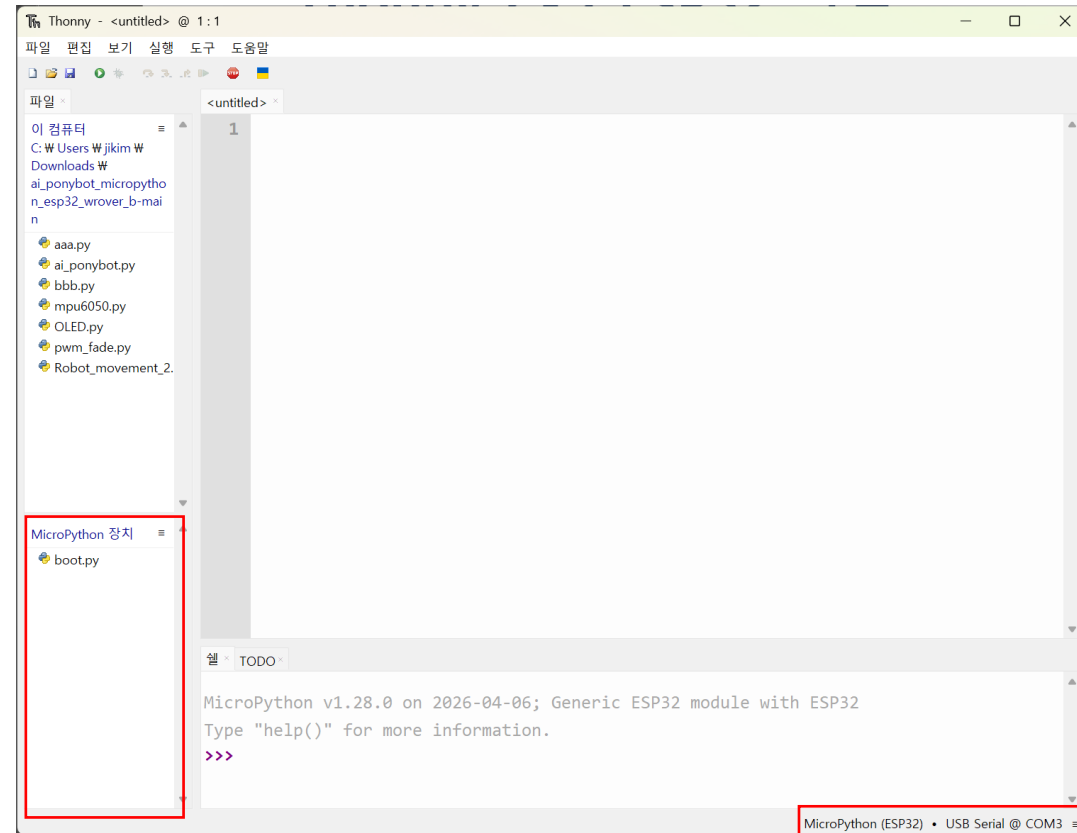


2. [실행] - 인터프리터 환경 설정 실행



3. 포트 선택 후 확인

Thonny <-> ESP32 연결



4. Thonny 재실행 후 장치 연결 완료

첫 코드 — Thonny 셸(REPL)에서 확인

셸에 직접 입력하면 즉시 실행됩니다(인터프리터). 파일로 저장해 [실행 ►] 하면 보드에서 동작합니다.

Shell (REPL)

```
>>> print('Hello, ESP32!')
Hello, ESP32!
>>> 1 + 2
3
>>> for i in range(3):
...     print(i)
0
1
2
```

blink.py

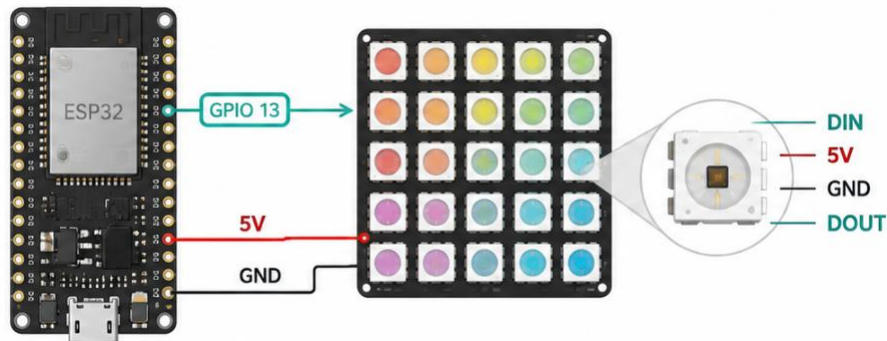
```
from time import sleep

while True:
    print('동작 중...')
    sleep(1)
```

모듈은 `from ... import ...` 로 먼저 불러오고, 들여쓰기는 공백 4칸으로 통일하세요.

LED 동작 원리 — 디지털 출력 & 내장 매트릭스

- 디지털 출력 = 0(끔)/1(켜) 신호를 핀으로 내보내는 것.
- ESP32(Mbits)에는 $5 \times 5 = 25$ 개 RGB LED(NeoPixel)가 내장(GPIO 13)
- 각 LED는 (R,G,B) 값 0~255로 색·밝기 표현 → `np.write()`로 반영.



NeoPixel 좌표

index 0(좌상) → 24(우하).
 $\text{index} = \text{행} \times 5 + \text{열}$.

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

행 ↓
 열 →

밝기 주의

값이 크면 매우 밝고 전류 ↑. 실습은 10~30 정도 낮은 값 권장.

[실습 1] 내장 LED 매트릭스 제어

matrix.py

```
from neopixel import NeoPixel
from machine import Pin
from time import sleep

np = NeoPixel(Pin(13, Pin.OUT), 25) # 내장 5x5

while True:
    for i in range(25):          # 하나씩 켜기
        np[i] = (10, 0, 0)      # 빨강(약하게)
        np.write()
        sleep(0.05)
    for i in range(25):          # 하나씩 끄기
        np[i] = (0, 0, 0)
        np.write()
        sleep(0.05)
```

예상 결과

0번부터 25개 LED가 빨강계 차례로 켜졌다 꺼집니다.

응용 과제

(R,G,B) 색 바꾸기. 2차원 배열로 하트·웃는 얼굴 만들기. $\text{index} = \text{row} * 5 + \text{col}$.

스위치(버튼) 동작 원리 — 디지털 입력

- 디지털 입력 = 핀 상태를 0/1로 읽기.
- 버튼 A = GPIO 36, 버튼 B = GPIO 39 (내장).
- 하드웨어 풀업 → 안 누르면 1, 누르면 0.

button.py

```
from machine import Pin
from time import sleep

btn_a = Pin(36, Pin.IN)
btn_b = Pin(39, Pin.IN)
while True:
    print(btn_a.value(),
          btn_b.value())
    sleep(0.2)
```

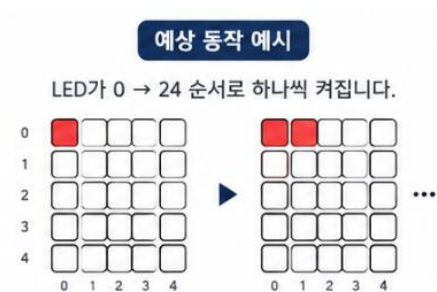
[실습 2] 버튼으로 LED 매트릭스 제어

button_led.py

```
from neopixel import NeoPixel
from machine import Pin
from time import sleep

np = NeoPixel(Pin(13, Pin.OUT), 25)
btn_a = Pin(36, Pin.IN) # 켜기
btn_b = Pin(39, Pin.IN) # 끄기

while True:
    if btn_a.value() == 0:
        for i in range(25): np[i] = (0, 12, 0)
    elif btn_b.value() == 0:
        for i in range(25): np[i] = (0, 0, 0)
    np.write()
    sleep(0.05)
```



예상 결과

A → 전체 초록 ON, B → 전체 OFF.

응용 과제

A 누를 때마다 색 토글. 이전 상태(prev) 변수로 한 번만 반응하게.

PWM 개념 — 밝기·세기를 조절하는 법

- 디지털 핀은 0/1뿐이지만, 빠르게 켜고 끄는 비율(듀티)을 바꾸면 '세기'를 표현 → PWM.
- LED 밝기·모터 속도 제어의 기본 원리.
- 이 보드의 내장 LED는 NeoPixel이라, 밝기를 'RGB 값(0~255)'으로 조절해 같은 효과를 안정적으로 구현합니다.

단일 RGB LED가 안 켜질 때

온보드 RGB LED는 공통애노드일 수 있어 PWM duty를 반대로(1023=꺼짐) 해야 하거나 freq 설정이 필요합니다. 그래서 본 실습은 확실한 NeoPixel로 진행합니다.

하드웨어 PWM은 어디에?

서보·모터·부저 제어에서 사용 → 2일차부터 다룹니다.

[실습 3] 내장 LED 밝기 조절 (PWM 효과)

led_fade.py

```
from neopixel import NeoPixel
from machine import Pin
from time import sleep

np = NeoPixel(Pin(13, Pin.OUT), 25)

while True:
    for v in range(0, 60, 2):      # 점점 밝게
        for i in range(25): np[i] = (v, 0, 0)
        np.write(); sleep(0.03)
    for v in range(60, -1, -2):    # 점점 어둡게
        for i in range(25): np[i] = (v, 0, 0)
        np.write(); sleep(0.03)
```

예상 결과

매트릭스 전체가 서서히 밝아졌다 어두워집니다 (숨쉬기 효과). 확실히 커집니다!

응용 과제

(v,0,0)을 (0,v,0)·(0,0,v)로 바꿔 색별 밝기. 최댓값 60→120으로 키워 비교.

내장 센서와 아날로그 입력(ADC)

- 아날로그 입력 = 연속 값을 숫자로 읽기. ADC가 0~4095로 변환.
- 마이크(소리) → ADC(Pin(35)).
- 가속도(움직임)·온도 센서도 내장 → 다음 실습에서 다룹니다.

마이크 (소리)

$\text{ADC(Pin(35))} \cdot \text{소리 크기} = \text{최댓값} - \text{최솟값}$

가속도 (움직임)

I2C 연결 · 기울기·흔들림 X/Y/Z

온도

`esp32.mcu_temperature()` (°C)

[실습 4] 센서 데이터 수집 / 표시 (마이크)

mic.py

```
from machine import Pin, ADC
from time import sleep

mic = ADC(Pin(35))          # 내장 마이크
mic.atten(ADC.ATTN_11DB)    # 0~3.3V

while True:
    s = [mic.read() for _ in range(100)]
    level = max(s) - min(s)  # 소리 크기
    print('소리 크기:', level)
    sleep(0.1)
```

예상 결과

조용하면 작고, 박수·말소리에 값이 커집니다.
(Thonny → 보기 → Plotter로 그래프)

확장

값이 일정 이상이면 LED를 켜는 '소리 감지등'으로 → 복합 실습에서!

가속도 센서 원리 & I2C 주소 확인

- 가속도 센서 = 기울기·움직임·흔들림을 X/Y/Z 3축 값으로 측정.
- 보드 내장이며 I2C로 연결됩니다 (SCL=21, SDA=22).
- 센서 칩마다 주소·라이브러리가 달라, 먼저 i2c.scan()으로 '주소'를 확인해야 합니다.

i2c_scan.py

```
from machine import I2C, Pin

i2c = I2C(0, scl=Pin(21),
          sda=Pin(22))
print(i2c.scan())
# 이 보드: 가속도=105(0x69)
```

주소 → 라이브러리

이 보드는 가속도 센서가 105=0x69(MPU6050 계열).
mpu6050.py를 올리고 addr=0x69로 사용합니다.

[실습 5] 가속도 센서 읽기

accel.py

```
from machine import I2C, Pin
import mpu6050          # mpu6050.py 업로드 필요
from time import sleep

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)  # 이 보드는 0x69
while True:
    v = imu.get_values()
    print('X', v['AcX'], 'Y', v['AcY'], 'Z', v['AcZ'])
    sleep(0.2)
```

예상 결과

보드를 기울이면 해당 축 값이 $\pm$ 로 변합니다. 평평하면 Z축이 큰 값.

준비

① mpu6050.py를 보드에 업로드 ② 이 보드의 가속도 센서는 0x69 (i2c.scan() 결과 105). v['Tmp']로 온도도 측정 가능.

[실습 6] 온도 센서 읽기

temp.py

```
import esp32
from time import sleep

while True:
    t = esp32.mcu_temperature()    # ESP32 칩 온도(°C)
    print('온도:', t, 'C')
    sleep(1)
```

예상 결과

현재 온도(°C)가 1초마다 출력. 보드를 손으로 감싸면 값이 오릅니다.

참고

esp32.mcu_temperature()는 칩 내부 온도라 주변보다 약간 높게 나옵니다. 가속도 센서가 MPU6050 계열이면 v['Tmp']로도 온도 측정 가능.

복합 실습 — 내장 요소를 조합하기

지금까지 배운 LED · 버튼 · 마이크 · 가속도 · 온도를 조합하면 '작은 제품'이 됩니다. (모두 외부 부품 없이 내장만 사용)

복합 ① 온도 경보등

온도 + LED 색

온도가 높으면 빨강, 낮으면 초록

복합 ② 반응속도 게임

버튼 + LED + 시간

초록 켜지면 빨리 누르기, ms 측정

복합 ③ 사운드 미터

마이크 + LED 매트릭스

소리 클수록 LED 많이 켜짐

복합 ④ 기울기 LED

가속도 + LED 매트릭스

기울인 쪽으로 LED 점이 굴러감

[복합 ①] 온도 경보등 (온도 + LED)

temp_alarm.py

```
import esp32
from neopixel import NeoPixel
from machine import Pin
from time import sleep

np = NeoPixel(Pin(13, Pin.OUT), 25)
while True:
    t = esp32.mcu_temperature()
    color = (12,0,0) if t >= 40 else (0,10,0) # 40↑ 빨강
    for i in range(25): np[i] = color
    np.write(); print(t); sleep(0.5)
```

동작

온도가 기준(40°C) 이상이면 전체 빨강, 아니면 초록.

도전

기준값 조절. 단계별 색(초록→노랑→빨강). 숫자(LED 패턴)로 온도 표시.

[복합 ②] 반응속도 게임 (버튼 + LED + 시간)

reaction.py

```
from machine import Pin
from neopixel import NeoPixel
from time import sleep, ticks_ms, ticks_diff
import random

np = NeoPixel(Pin(13, Pin.OUT), 25)
btn = Pin(36, Pin.IN)
while True:
    for i in range(25): np[i]=(0,0,0)
    np.write()
    sleep(random.uniform(1, 3))      # 무작위 대기
    for i in range(25): np[i]=(0,12,0) # 초록 = 지금!
    np.write()
    t0 = ticks_ms()
    while btn.value()==1: pass        # 누를 때까지
    print('반응:', ticks_diff(ticks_ms(), t0), 'ms')
    sleep(1)
```

동작

초록불이 켜지는 순간 A버튼을 누르면 반응시간(ms)이 출력됩니다.

도전

최고 기록 저장. 너무 빨리 누르면 '반칙!' 표시.
2인 대결로 확장.

[복합 ③] 사운드 미터 (마이크 + LED)

sound_meter.py

```
from machine import Pin, ADC
from neopixel import NeoPixel
from time import sleep

mic = ADC(Pin(35)); mic atten(ADC.ATTN_11DB)
np = NeoPixel(Pin(13, Pin.OUT), 25)
while True:
    s = [mic.read() for _ in range(100)]
    level = max(s) - min(s)
    n = min(level // 80, 25)          # 켜는 LED 개수
    for i in range(25):
        np[i] = (8,0,0) if i < n else (0,0,0)
    np.write(); sleep(0.05)
```

동작

소리가 커질수록 켜지는 LED 개수가 늘어납니다.

도전

// 80 값으로 민감도 조절. 단계별 색(초록→노랑→빨강)으로 레벨 표시.

[복합 ④] 기울기 LED 점 (가속도 + LED)

tilt_led.py

```
from machine import I2C, Pin
import mpu6050
from neopixel import NeoPixel
from time import sleep

i2c=I2C(0,scl=Pin(21),sda=Pin(22)); imu=mpu6050.accel(i2c,0x69)
np=NeoPixel(Pin(13,Pin.OUT),25)
clamp=lambda v: max(0, min(4, v))
while True:
    a=imu.get_values()
    col=clamp(2 + a['AcX']//6000) # 기울기→열
    row=clamp(2 + a['AcY']//6000) # 기울기→행
    for i in range(25): np[i]=(0,0,0)
    np[row*5+col]=(0,0,12)
    np.write(); sleep(0.05)
```

동작

보드를 기울이면 파란 LED 점이 기운 쪽으로 이동합니다.

준비

[실습 5]처럼 i2c.scan() 후 가속도 라이브러리를 업로드해야 동작합니다. //6000 값으로 민감도 조절.

APPLY

가속도 응용 실습

기울기·흔들림·움직임을 활용한 5가지 미니 프로젝트. (가속도 0x69 + 내장 LED, 외부 부품 없음)

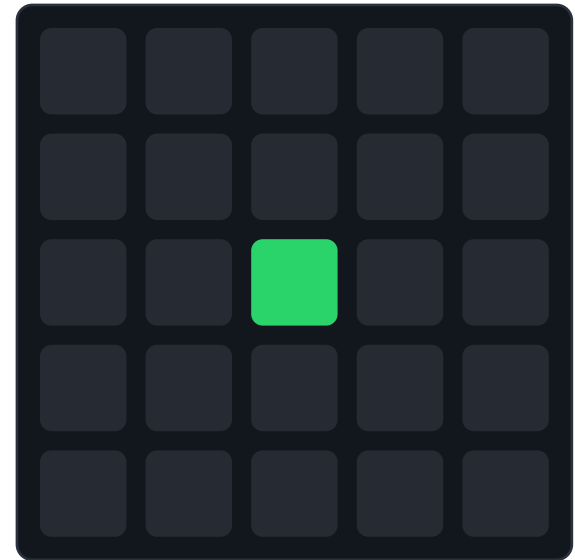
[응용 ①] 디지털 수평계

level.py

```
from machine import I2C, Pin
import mpu6050
from neopixel import NeoPixel
from time import sleep

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)
np = NeoPixel(Pin(13, Pin.OUT), 25)
clamp = lambda v: max(0, min(4, v))
while True:
    a = imu.get_values()
    col = clamp(2 + a['AcX'] // 4000)
    row = clamp(2 + a['AcY'] // 4000)
    for i in range(25): np[i] = (0,0,0)
    if col == 2 and row == 2:
        np[12] = (0, 12, 0)      # 수평!
    else:
        np[row*5+col] = (14, 6, 0) # 기포
    np.write(); sleep(0.05)
```

예상 LED 패턴



동작

기울이면 주황 '기포'가 움직이고, 수평이 되면 가운데가 초록불이 됩니다.

[응용 ②] 흔들기 주사위

dice.py

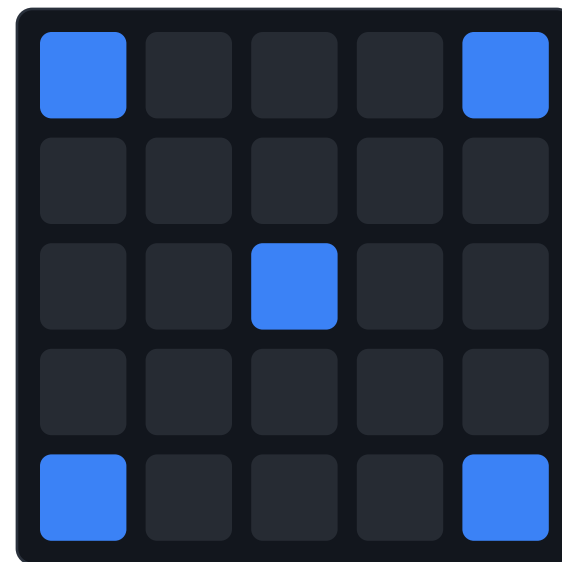
```

from machine import I2C, Pin
import mpu6050
from neopixel import NeoPixel
from time import sleep
import random, math

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)
np = NeoPixel(Pin(13, Pin.OUT), 25)
DICE = {1:[12], 2:[6,18], 3:[6,12,18], 4:[0,4,20,24],
        5:[0,4,12,20,24], 6:[0,4,10,14,20,24]}
def show(n, c=(0,0,14)):
    for i in range(25): np[i] = (0,0,0)
    for i in DICE[n]: np[i] = c
    np.write()
show(1)
while True:
    a = imu.get_values()
    m = math.sqrt(a['AcX']**2+a['AcY']**2+a['AcZ']**2)
    if m > 30000:
        for _ in range(8):
            show(random.randint(1,6),(10,10,0)); sleep(0.08)
            show(random.randint(1,6)); sleep(0.6)
        sleep(0.05)

```

예상 LED 패턴



동작

흔들면 주사위 눈이 빠르게 바뀌다 1~6 중 하나로 멈춥니다. (그림은 '5' 예시)

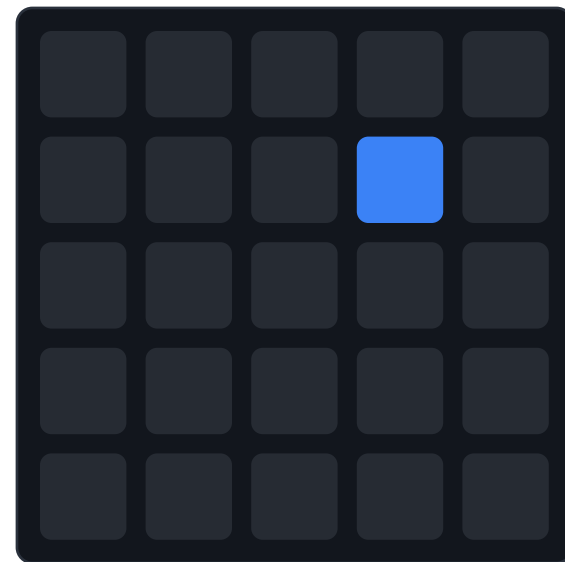
[응용 ③] 공 굴리기

ball.py

```
from machine import I2C, Pin
import mpu6050
from neopixel import NeoPixel
from time import sleep

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)
np = NeoPixel(Pin(13, Pin.OUT), 25)
x, y, vx, vy = 2.0, 2.0, 0.0, 0.0
while True:
    a = imu.get_values()
    vx += a['AcX'] / 80000      # 가속
    vy += a['AcY'] / 80000
    vx *= 0.9; vy *= 0.9      # 마찰
    x = min(4, max(0, x + vx))
    y = min(4, max(0, y + vy))
    for i in range(25): np[i] = (0,0,0)
    np[int(y)*5 + int(x)] = (0,0,15)
    np.write(); sleep(0.04)
```

예상 LED 패턴



동작

기울인 방향으로 파란 공이 관성·마찰을 받으며
부드럽게 굴러다닙니다.

[응용 ④] 방향 화살표

arrow.py

```

from machine import I2C, Pin
import mpu6050
from neopixel import NeoPixel
from time import sleep

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)
np = NeoPixel(Pin(13, Pin.OUT), 25)
UP=[2,6,7,8,12,17]; DOWN=[7,12,16,17,18,22]
LEFT=[6,10,11,12,13,16]; RIGHT=[8,11,12,13,14,18]
def show(idxs, c=(0,10,10)):
    for i in range(25): np[i]=(0,0,0)
    for i in idxs: np[i]=c
    np.write()
TH = 8000
while True:
    a = imu.get_values()
    if a['AcX'] > TH: show(RIGHT)
    elif a['AcX'] < -TH: show(LEFT)
    elif a['AcY'] > TH: show(DOWN)
    elif a['AcY'] < -TH: show(UP)
    else: show([12], (0,12,0))
    sleep(0.1)

```

예상 LED 패턴



동작

기울인 쪽을 화살표로 표시하고, 평평하면 가운데 점이 초록으로 켜집니다. (그림은 위쪽)

[응용 ⑤] 만보계 (흔들림 카운터)

step.py

```
from machine import I2C, Pin
import mpu6050, math
from time import sleep, ticks_ms, ticks_diff

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)
count, last = 0, 0
while True:
    a = imu.get_values()
    m = math.sqrt(a['AcX']**2+a['AcY']**2+a['AcZ']**2)
    now = ticks_ms()
    if m > 24000 and ticks_diff(now, last) > 300:
        count += 1; last = now
        print('걸음 수:', count)
    sleep(0.02)
```

셸(Shell) 출력 예

```
걸음 수: 1
걸음 수: 2
걸음 수: 3
걸음 수: 4
걸음 수: 5
```

동작

걸음(흔들림)을 감지해 셸에 누적 걸음 수를 출력합니다. 임계값 24000으로 민감도 조절.

종합 실습 — 멀티모드 스마트 장치

all_in_one.py

```

from machine import I2C, Pin, ADC
from neopixel import NeoPixel
from time import sleep
import esp32, mpu6050, math

i2c = I2C(0, scl=Pin(21), sda=Pin(22))
imu = mpu6050.accel(i2c, 0x69)
mic = ADC(Pin(35)); mic.atten(ADC.ATTN_11DB)
np = NeoPixel(Pin(13, Pin.OUT), 25)
btn = Pin(36, Pin.IN); mode = 0

while True:
    if btn.value() == 0:                # A = 모드 변경
        mode = (mode + 1) % 4; sleep(0.3)
    a = imu.get_values()
    for i in range(25): np[i] = (0,0,0)
    if mode == 0:                       # ① 수평계
        c = max(0,min(4, 2+a['AcX']//4000))
        r = max(0,min(4, 2+a['AcY']//4000))
        np[r*5+c] = (0,12,0) if c==2 and r==2 else (14,6,0)
    elif mode == 1:                    # ② 사운드 미터
        s = [mic.read() for _ in range(80)]
        for i in range(min((max(s)-min(s))//80,25)): np[i]=(10,0,0)
    elif mode == 2:                    # ③ 온도등
        np[12] = (12,0,0) if esp32.mcu_temperature()>=40 else (0,0,12)
    else:                              # ④ 흔들기
        if math.sqrt(a['AcX']**2+a['AcY']**2+a['AcZ']**2)>28000:
            for i in range(25): np[i]=(8,8,0)
    np.write(); sleep(0.05)

```

모드 (A 버튼으로 전환)

① 수평계

가속도 · 기울기 표시

② 사운드 미터

마이크 · 소리 크기

③ 온도등

온도 · 40°C ↑ 빨강

④ 흔들기

가속도 · 흔들면 노랑

도전: B 버튼으로 밝기 조절, 5번째 모드 추가!

종합 미션 — 나만의 내장형 장치 만들기

배운 내장 요소(LED · 버튼 · 마이크 · 가속도 · 온도)를 2가지 이상 조합해 '작은 제품'을 만들어 봅시다. (외부 부품 없이!)

기분 표시등

버튼으로 색을 바꿔 감정 표현

박수 스위치

박수(마이크) 2번 → LED ON/OFF

수평계

가속도로 기울기 0 맞추기 게임

흔들기 주사위

흔들면(가속도) 1~6 LED 패턴 랜덤

온도 알리미

온도 구간별 색 + 경고 깜빡임

반응속도 대결

2인용 반응속도 게임 확장

1일차 정리 & 다음 차시

오늘 한 것

- AI 로봇의 정의·역사·분류·핵심 기술(인지-판단-제어) 이해
- Thonny + MicroPython(ESP32) 개발 환경 구축
- 내장 LED 매트릭스·버튼·밝기(PWM)·마이크·가속도·온도 제어
- 내장 요소를 조합한 복합 실습(온도 경보등·반응 게임·사운드 미터·기울기 LED)

다음 차시 (2일차) 포니봇 본체에 ESP32를 결합해 **메카닉 휠 주행 제어**를 실습합니다. (하드웨어 PWM이 모터·서보 제어로 확장)

멀티모달 인지 기반 피지컬 AI 로봇 시스템 구현 과정

2일차 강의자료

AI 포니봇 플랫폼 이해 · 포니봇 모션 제어

메카넘 휠 기구학 · 전방향 이동 · DC 모터/서보/OLED 제어 실습

5일 과정

1일 7시간

이론 3H + 실습 4H

5일 과정 한눈에 보기 · 오늘은 2일차

1일차

AI 로봇 개론 + ESP32·MicroPython 기초

2일차

포니봇 플랫폼 이해 + 모션 제어 (메카넘·서보·OLED)

3일차

센서 기반 자율주행 (초음파·라인)

4일차

AI 웹앱 연동 (객체·음성·포즈 인식)

5일차

미니 프로젝트 & 발표

※ 1일차 '출력·센서 제어'가 오늘 '로봇의 움직임 제어'로 확장됩니다.

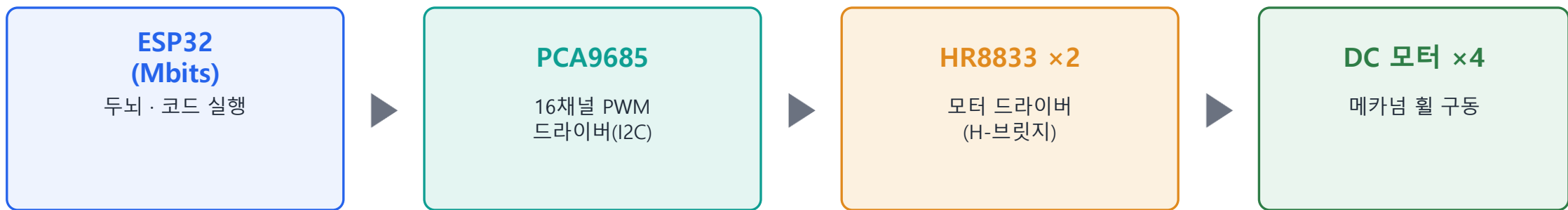
[03] 3H

AI 포니봇 플랫폼 이해

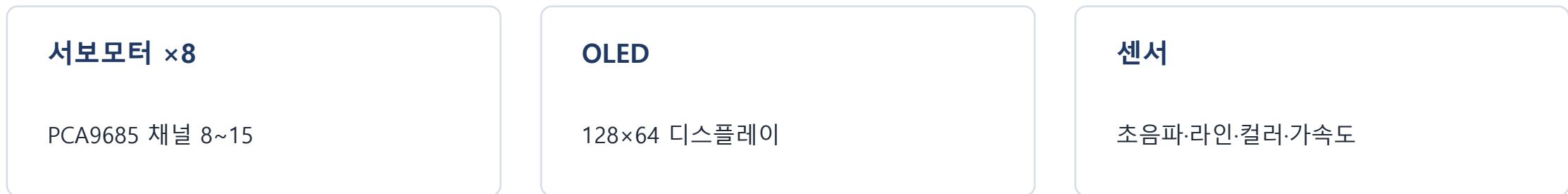
포니봇의 시스템 구조·구성요소, 4축 메카넘 휠 기구학, 전방향 이동, I2C 확장 인터페이스를 알아봅니다. (이론)

AI 포니봇 시스템 구조

두뇌(ESP32)의 명령이 I2C → PWM 드라이버 → 모터 드라이버를 거쳐 바퀴를 움직입니다.



I2C로 함께 연결되는 장치



포니봇 구성요소

메인보드

ESP32(Mbits)
코드 실행·제어

DC 모터 ×4

메카넘 휠 구동
전방향 이동

서보 포트 ×8

각도 제어
로봇팔·그리퍼

OLED

128×64 디스플레이
정보 표시

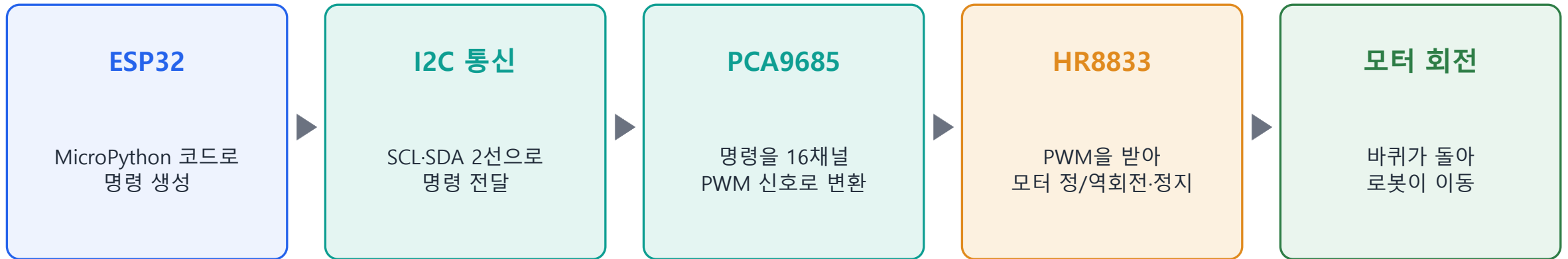
센서

초음파·라인·컬러
(자율주행용)

전원/배터리

구동 전원 공급

제어 신호의 흐름

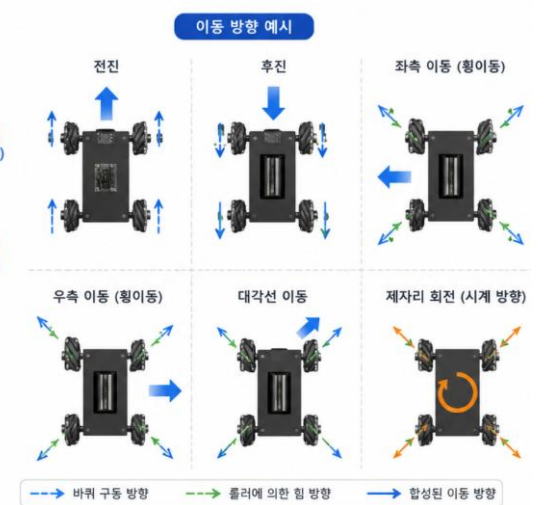


I2C 통신 (Inter-Integrated Circuit)

SCL(클럭)·SDA(데이터) 단 2개의 선으로 여러 장치(1:N)를 제어하는 근거리 직렬 통신. NXP(옛 필립스)가 개발.

메카넘 휠이란?

- 바퀴 둘레에 45° 기울어진 작은 롤러가 달린 특수 바퀴입니다.
- 각 바퀴가 만드는 힘이 대각선으로 작용 → 네 바퀴의 회전 조합으로 어느 방향이든 이동 가능.
- 제자리에서 옆으로(횡이동)·대각선·회전까지 — 좁은 공간에서 자유롭게 움직입니다.



핵심

일반 바퀴는 앞/뒤만 가능하지만, 메카넘 휠은 '전방향(Omnidirectional)' 이동이 가능합니다.

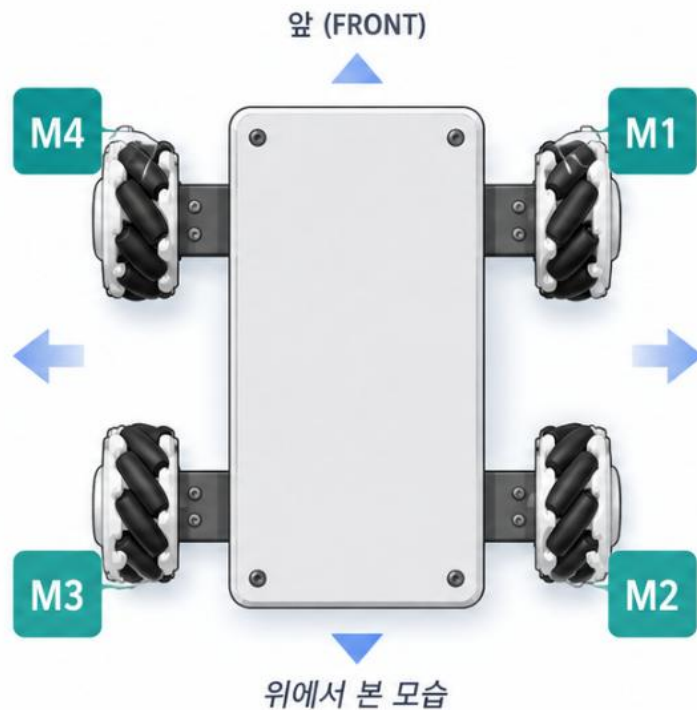
4축 메카넘 휠 배치 & 기구학

네 바퀴(M1~M4)의 회전 방향·속도 조합이 로봇의 이동 방향을 결정합니다 (기구학).

기구학(Kinematics)이란?

각 바퀴의 회전(방향·속도)을 어떻게 조합하면 로봇이 원하는 방향·속도로 움직이는지를 계산하는 원리입니다.

라이브러리가 이 계산을 대신 처리해 줍니다.



이동의 비밀

- 네 바퀴 같은 방향 → 전진/후진



- 좌우가 반대 → 횡이동



- 앞뒤 대각선 → 대각선



- 좌우 짝이 반대 → 제자리 회전



전방향(Omnidirectional) 이동 메커니즘

각 바퀴의 회전 방향 조합(↑정회전 / ↓역회전 / — 정지)으로 8방향 + 회전이 만들어집니다.

동작	M1	M2	M3	M4
전진	↑	↑	↑	↑
후진	↓	↓	↓	↓
좌 횡이동	↑	↓	↑	↓
우 횡이동	↓	↑	↓	↑
대각선 ↗	—	↑	—	↑
제자리 회전	↑	↑	↓	↓

전방향 이동이란?

앞·뒤뿐 아니라 옆·대각선·제자리 회전까지 — 몸체 방향을 바꾸지 않고 자유 이동.

장점

좁은 공간 주차·정렬에 유리. 물류 로봇·정밀 위치 제어에 활용.

범용 I/O 및 I2C 확장 인터페이스

포니봇의 거의 모든 부품은 'I2C 버스' 하나에 주소로 구분되어 연결됩니다. 선 2개로 여러 장치를 제어!



확장성

i2c.scan()으로 연결 장치의 주소를 확인할 수 있고, 같은 버스에 새 센서·모듈을 '주소만 다르면' 더 붙일 수 있습니다.

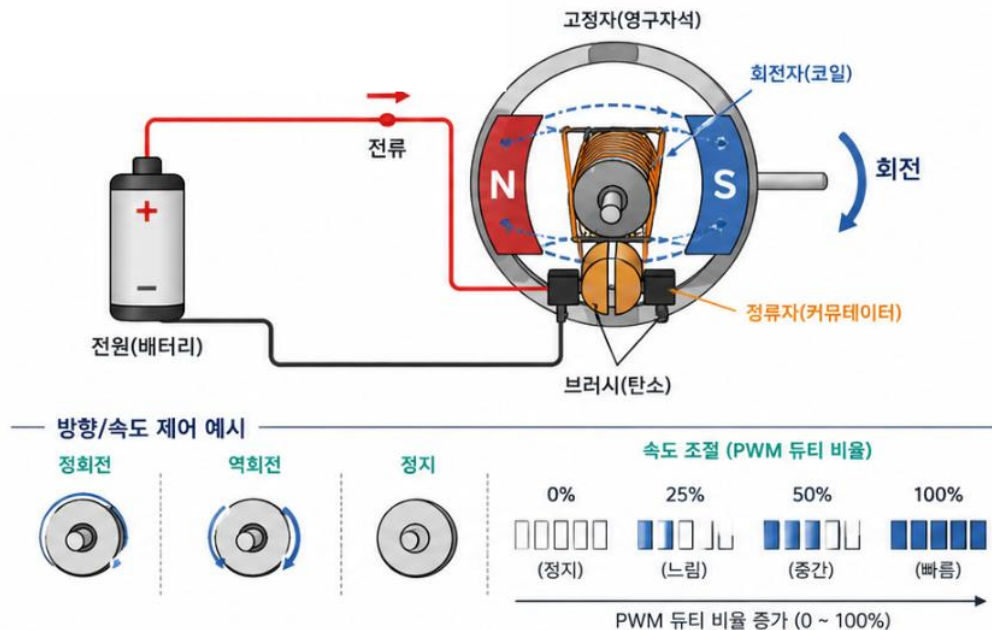
[04] 4H

포니봇 모션 제어

DC 모터·PWM 속도 제어, 메카넘 주행, 서보(8채널) 각도 제어, OLED 출력을 코드로 실습합니다. (이론 & 실습)

DC 모터의 원리

- 모터는 전기 에너지를 회전 운동 에너지로 바꾸는 '출력 장치(액추에이터)'입니다.
- 고정자(영구자석) 안에서 회전자(코일)에 흐르는 전류 방향을 바꾸며 자기력으로 회전합니다.
- 전류 방향 → 회전 방향, 전류 세기(PWM) → 회전 속도를 결정합니다.



어디에 쓰일까?

세탁기·에어컨부터 전기차·드론까지. 포니봇에서는 4개의 메카닉 휠을 돌립니다.

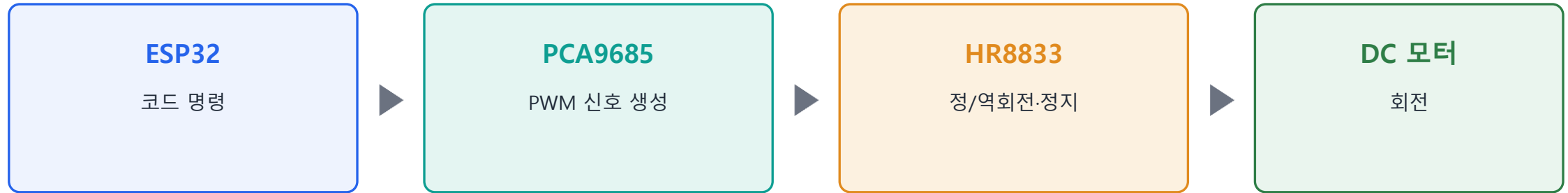


방향 & 속도

- 정회전 / 역회전 / 정지
- 속도 = PWM 듀티 비율로 조절 (0 ~ 100%)



모터 회로와 PWM 속도 제어



- PWM(펄스 폭 변조): 빠르게 ON/OFF하며 켜진 시간 비율(듀티)로 '평균 세기'를 조절.
- 듀티가 클수록 모터에 가해지는 평균 전압이 커져 → 더 빠르게 회전합니다.
- 포니봇은 speed 값 0~100(%)으로 속도를 지정합니다.

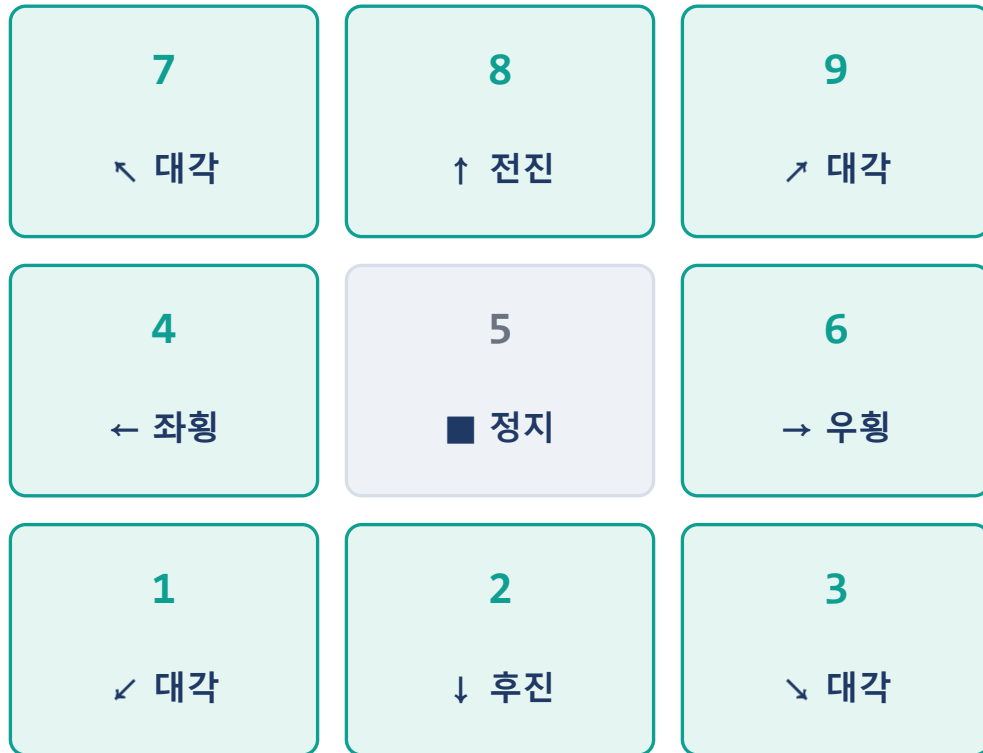
속도 = 듀티(%)

speed 30 → 천천히
speed 60 → 보통
speed 100 → 최고 속도

```
motor.drive("forward", 60)
```

메카넘 방향 제어 — 넘버패드 코드

mecanum(코드, 속도)의 방향 코드는 키패드 숫자 배치와 같아 직관적입니다. 회전은 drive("left"/"right").



mecanum 사용

```
from ai_ponybot import i2c, PonyMotor
from time import sleep

motor = PonyMotor(i2c)
motor.mecanum(8, 60) # 전진
sleep(1)
motor.mecanum(6, 60) # 우 횡이동
sleep(1)
motor.mecanum(5)    # 정지
```

제자리 회전

motor.drive("left", 60) / drive("right", 60) 로 좌·우 회전합니다.



drive("left", 60)
좌회전



drive("right", 60)
우회전

[실습 1] 기본 주행 (전·후·좌·우)

drive_basic.py

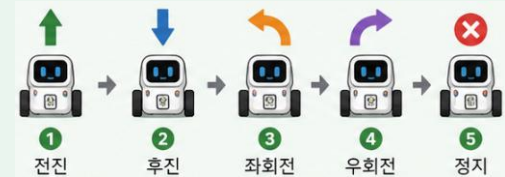
```
from ai_ponybot import i2c, PonyMotor
from machine import Pin
from time import sleep

motor = PonyMotor(i2c)
btn = Pin(36, Pin.IN)      # A 버튼으로 시작

while True:
    if btn.value() == 0:
        motor.drive('forward', 60); sleep(1)
        motor.drive('backward', 60); sleep(1)
        motor.drive('left', 60);      sleep(1)    # 좌회전
        motor.drive('right', 60);     sleep(1)    # 우회전
        motor.drive('stop')
        sleep(0.05)
```

동작

A 버튼을 누르면 전진→후진→좌회전→우회전 후 정지합니다.



주의 & 도전



- 바퀴가 뜨도록 받침 위에서 먼저 테스트



- 속도·시간(sleep) 값을 바꿔 보기

[실습 2] 대각선 · 횡이동 주행 (메카넘)

drive_mecanum.py

```
from ai_ponybot import i2c, PonyMotor
from time import sleep

motor = PonyMotor(i2c)

# 8전진 2후진 4좌횡 6우횡 7↖ 9↗ 1↘ 3↙ 5정지
seq = [8, 6, 2, 4, 9, 1]

for code in seq:
    motor.mecanum(code, 60)
    sleep(1)
motor.mecanum(5) # 정지
```

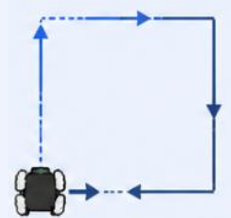
동작

전진 → 우횡이동 → 후진 → 좌횡이동 → 대각 ↗ → 대각 ↘ 순서로 움직입니다.



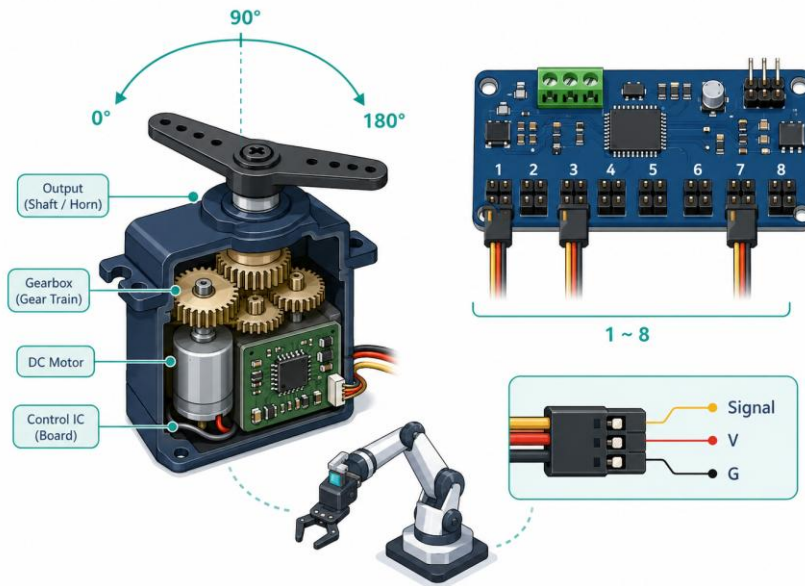
도전

정사각형 경로를 '옆으로만' 이동해 그려 보기. 회전(drive)도 섞어 보기.



서보모터 원리 & 8채널 포트

- 서보모터 = DC 모터 + 기어박스 + 제어 IC. '정확한 각도'로 움직이고 그 위치를 유지합니다.
- 포니봇은 PCA9685로 최대 8채널(서보 1~8)의 서보를 제어할 수 있습니다.
- 보통 0~180° 범위. 로봇팔·집게(그리퍼)·센서 방향 조절 등에 사용합니다.



각도 제어 (0° ~ 180°)

```
set_angle(1, 90)
```



연결

서보 3선: 신호 / V(빨강) / G(검정). 전원은 최소 4.8V 이상 공급(MG90 기준).

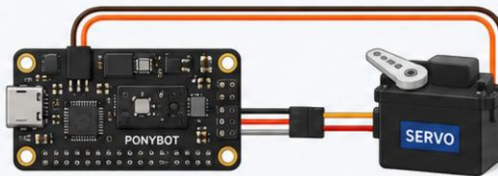
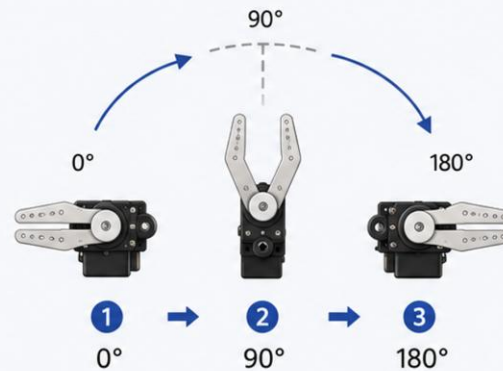
[실습 3] 서보 각도 제어

servo.py

```
from ai_ponybot import i2c, PonyMotor, PonyServo
from time import sleep
```

```
motor = PonyMotor(i2c)
servo = PonyServo(motor.pwm) # PWM 공유
```

```
while True:
    servo.set_angle(1, 0) # 0도
    sleep(1)
    servo.set_angle(1, 90) # 90도
    sleep(1)
    servo.set_angle(1, 180) # 180도
    sleep(1)
```



동작

1번 서보가 $0^\circ \rightarrow 90^\circ \rightarrow 180^\circ$ 를 반복합니다.
(집게 열고 닫기 등)

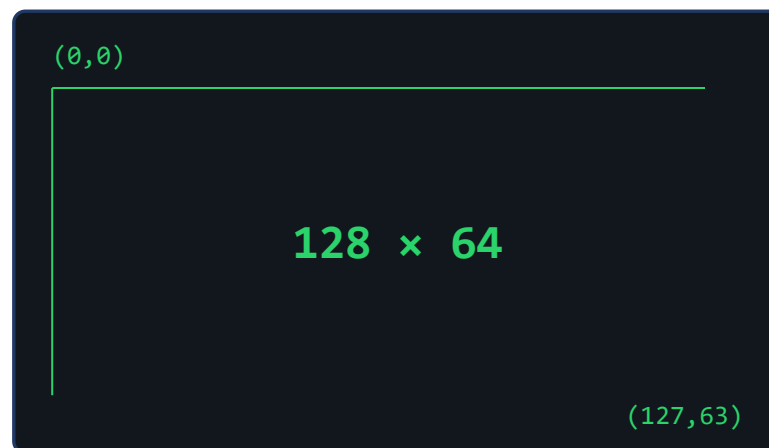


도전

servo.release(1)로 힘 풀기. 서보 번호(1~8)를 바꿔 여러 개 동시 제어.

OLED 구동 원리 & 좌표계

- OLED는 픽셀이 '스스로 빛을 내는' 디스플레이(백라이트 불필요) — 선명하고 시인성이 좋습니다.
- 포니봇 OLED는 가로 128 × 세로 64 픽셀. 각 픽셀을 좌표(x, y)로 ON/OFF 합니다.
- 텍스트/도형을 버퍼에 그린 뒤 show()로 한 번에 화면에 표시합니다.



출력 방법

write_line(줄, 글자) — 줄 단위
draw_text(x, y, 글자) — 좌표 단위
→ 마지막에 show()

[실습 4] OLED 출력 (다양한 정보 표시)

oled.py

```
from ai_ponybot import i2c, PonyOLED
from time import sleep

oled = PonyOLED(i2c)

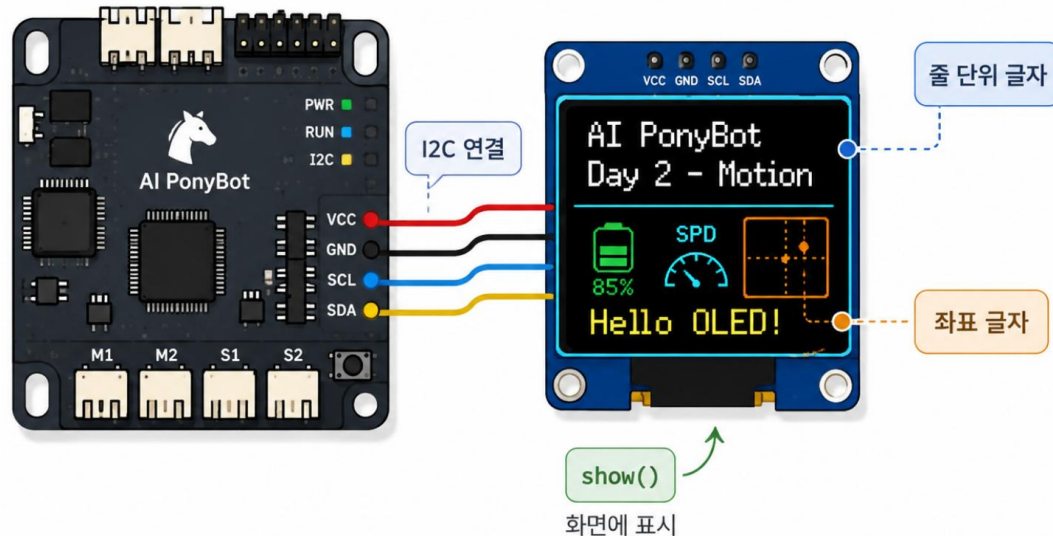
oled.clear()
oled.write_line(0, 'AI PonyBot')
oled.write_line(1, 'Day 2 - Motion')
oled.draw_text(0, 40, 'Hello OLED!')
oled.show()      # 화면에 표시
```

동작

줄 단위 글자(write_line)와 좌표 글자(draw_text)가 화면에 함께 표시됩니다.

도전

센서값·속도·모드를 실시간 표시. draw_rect로 테두리·게이지 그리기.



[실습 5] 주행 + OLED 상태 표시

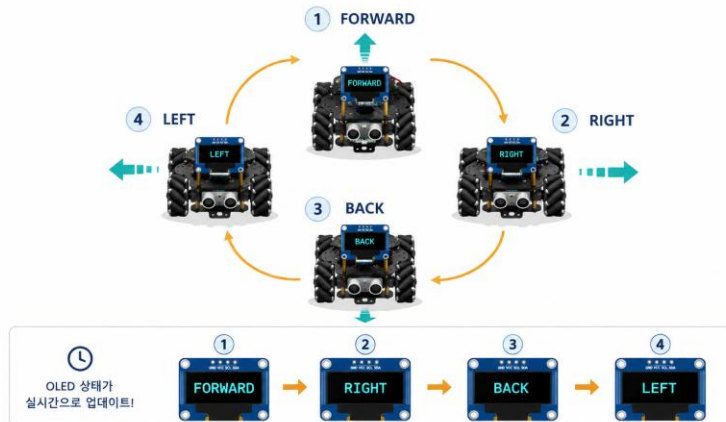
움직이는 방향을 OLED에 실시간으로 표시하는 '상태판 달린 로봇'을 만들어 봅시다.

drive_oled.py

```
from ai_ponybot import i2c, PonyMotor, PonyOLED
from time import sleep

motor = PonyMotor(i2c)
oled = PonyOLED(i2c)

moves = [('FORWARD', 8), ('RIGHT>>', 6),
         ('BACK', 2), ('<<LEFT', 4)]
for name, code in moves:
    oled.clear()
    oled.write_line(0, 'Moving:')
    oled.write_line(1, name)
    oled.show()
    motor.mecanum(code, 60); sleep(1.5)
motor.mecanum(5) # 정지
```



동작

전진·우회·후진·좌회으로 움직이며, 그때마다 OLED에 현재 동작을 표시합니다.

도전

서보(그리퍼)도 함께 제어. 버튼으로 시작/정지. 사각형 경로 주행 후 'Done!' 표시.

DRIVE+

주행 응용 실습

지금까지 배운 주행·버튼·마이크·OLED로 만드는 응용 주행 5종과 종합 실습. (외부 센서·기울기 제외)

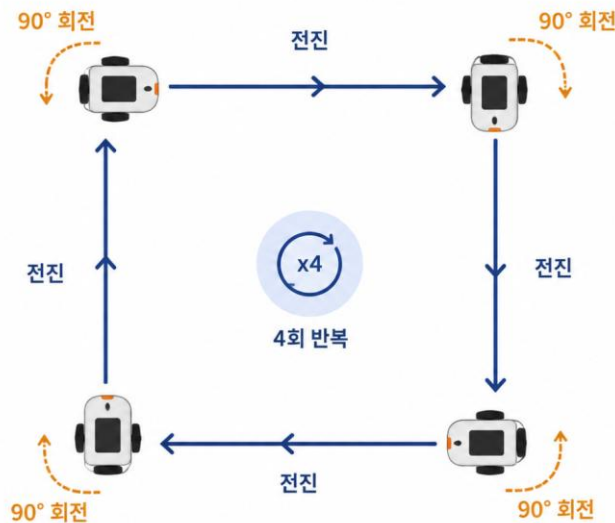
[응용 ①] 사각형 경로 주행

square.py

```
from ai_ponybot import i2c, PonyMotor
from time import sleep

motor = PonyMotor(i2c)
SPEED = 60
TURN = 0.6          # 90도 회전 시간(조정)

for _ in range(4):  # 변 4개
    motor.drive('forward', SPEED)
    sleep(1.0)
    motor.drive('left', SPEED)    # 제자리 90도 회전
    sleep(TURN)
    motor.drive('stop')
```



동작

전진→90° 회전을 4번 반복해 정사각형을 그립니다.

도전

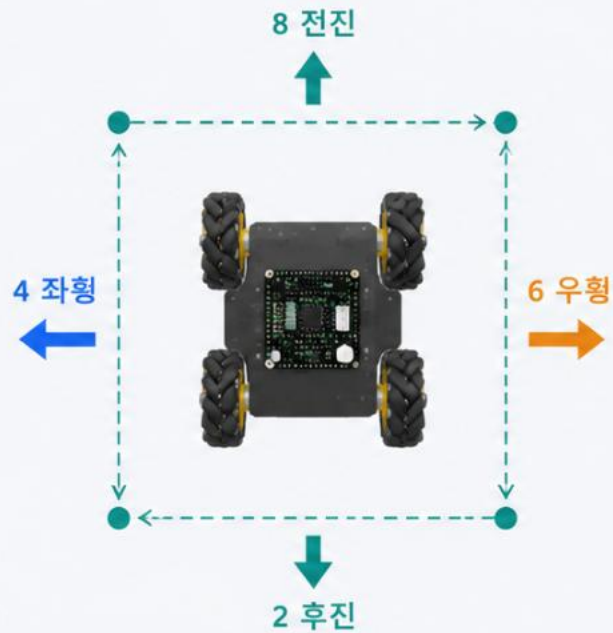
TURN 값으로 회전 각도를 맞추기. 삼각형(3회)·오각형으로 바꿔 보기.

[응용 ②] 게걸음 사각형 (횡이동)

crab.py

```
from ai_ponybot import i2c, PonyMotor
from time import sleep

motor = PonyMotor(i2c)
# 8전진 6우회 2후진 4좌회 (회전 없이!)
for code in [8, 6, 2, 4]:
    motor.mecanum(code, 60)
    sleep(1.2)
motor.mecanum(5)      # 정지
```



동작

회전하지 않고 옆으로만 이동해 사각형을 그립니다 (메카넘의 장점).

도전

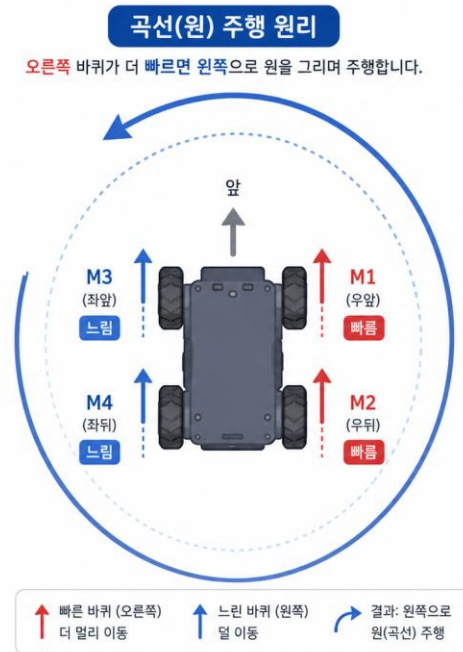
대각선 코드(7·9·1·3)를 섞어 마름모 경로 만들기.

[응용 ③] 곡선 주행 (원 그리기)

circle.py

```
from ai_ponybot import i2c, PonyMotor
from time import sleep

motor = PonyMotor(i2c)
# 오른쪽(M1,M2) 빠르게 + 왼쪽(M3,M4) 느리게
# → 왼쪽으로 도는 원
motor.move(1, 70); motor.move(2, 70) # 오른쪽
motor.move(3, 35); motor.move(4, 35) # 왼쪽
sleep(4) # 한 바퀴 도는 시간
motor.drive('stop')
```



동작

좌·우 바퀴 속도를 다르게 해서 곡선(원)을 그립니다.

도전

속도 차이를 줄이면 큰 원, 키우면 작은 원. 8자 그리기 도전.

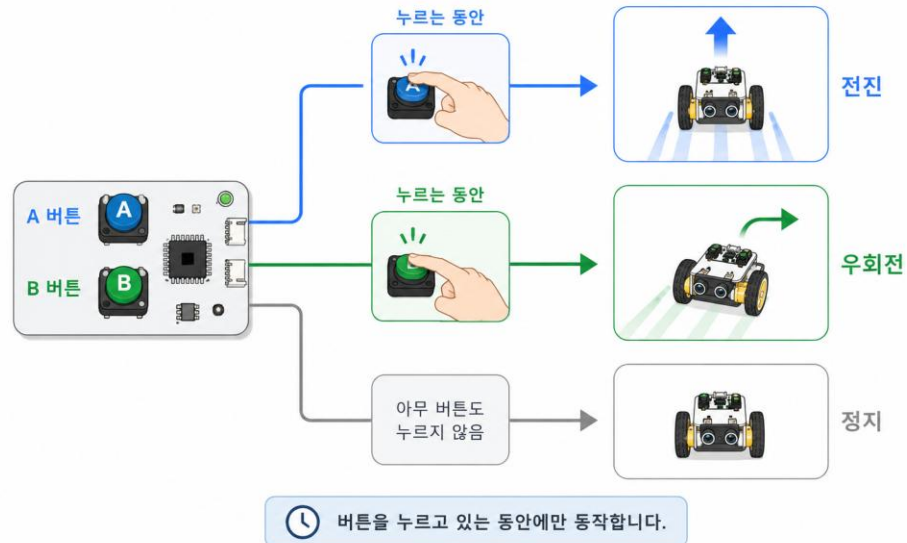
[응용 ④] 버튼 리모컨 주행

remote.py

```
from ai_ponybot import i2c, PonyMotor
from machine import Pin
from time import sleep
```

```
motor = PonyMotor(i2c)
a = Pin(36, Pin.IN) # A = 전진
b = Pin(39, Pin.IN) # B = 우회전
```

```
while True:
    if a.value() == 0:
        motor.drive('forward', 60)
    elif b.value() == 0:
        motor.drive('right', 50)
    else:
        motor.drive('stop')
    sleep(0.05)
```



동작

내장 버튼 A·B를 눌러 직접 조종합니다 (누르는 동안 이동).

도전

버튼 조합으로 더 많은 방향 제어. 두 버튼 동시 = 특수 동작.

[응용 ⑤] 가속·감속 주행

ramp.py

```
from ai_ponybot import i2c, PonyMotor
from time import sleep

motor = PonyMotor(i2c)
for spd in range(0, 101, 10):      # 가속
    motor.drive('forward', spd); sleep(0.2)
for spd in range(100, -1, -10):    # 감속
    motor.drive('forward', spd); sleep(0.2)
motor.drive('stop')
```

동작

속도(PWM)를 0→100→0으로 바꿔 부드럽게 출발·정지합니다.



도전

step 값을 작게 하면 더 부드럽게. 후진 가속·감속도 추가.

종합 응용 실습 — 소리·버튼 주행 컨트롤러

sound_drive.py

```
from ai_ponybot import i2c, PonyMotor, PonyOLED
from machine import Pin, ADC
from time import sleep

motor = PonyMotor(i2c); oled = PonyOLED(i2c)
btnA = Pin(36, Pin.IN); btnB = Pin(39, Pin.IN)
mic = ADC(Pin(35)); mic atten(ADC.ATTN_11DB)
MODES = ['Square', 'Crab', 'Dance']; mode = 0

def sound():
    s = [mic.read() for _ in range(50)]
    return max(s) - min(s)

def screen(state):
    oled.clear()
    oled.write_line(0, 'Sound Drive Bot')
    oled.write_line(1, 'Mode : ' + MODES[mode])
    oled.write_line(2, 'State: ' + state)
    oled.write_line(4, 'Sound: ' + str(sound()))
    oled.show()

def run():
    seq = [[('forward',0.8),('left',0.6)]*4,
           [(8,0.9),(6,0.9),(2,0.9),(4,0.9)],
           [(7,0.3),(9,0.3),(3,0.3),(1,0.3)]*2][mode]
    for a, t in seq:
        if sound() > 1500 or btnB.value()==0: break # 박수=정지
        motor.drive(a,60) if isinstance(a,str) else motor.mecanum(a,65)
        sleep(t)
    motor.drive('stop')

screen('READY')
while True:
    if btnA.value()==0: mode=(mode+1)%3; screen('READY'); sleep(0.3)
    if btnB.value()==0: screen('RUN'); run(); screen('STOP'); sleep(0.3)
    screen('READY'); sleep(0.1)
```

제어 방법

A 버튼: 패턴 변경
 B 버튼: 시작
 마이크: 박수 = 비상 정지
 OLED: 모드·상태·소리

동작 흐름

대기 중 소리값이 실시간 표시 → A로
 패턴(사각형·계걸음·댄스) 선택 → B로
 출발 → 주행 중 박수치면 즉시 정지.

2일차 정리 & 다음 차시

오늘 한 것

- 포니봇 시스템 구조(ESP32→PCA9685→HR8833→모터)와 구성요소 이해
- 4축 메카넘 휠 기구학과 전방향 이동 메커니즘
- DC 모터·PWM 속도 제어, 메카넘 주행(전후좌우·횡이동·대각선)
- 서보 8채널 각도 제어, OLED 정보 출력, 주행+OLED 종합 실습

다음 차시 (3일차) 초음파·라인 센서를 더해 **센서 기반 자율주행**을 구현합니다. 오늘의 '주행 제어'가 '스스로 판단하는 주행'으로 발전합니다.

멀티모달 인지 기반 피지컬 AI 로봇 시스템 구현 과정

3일차 강의자료

센서 통합 및 환경 인지 · 센서 기반 자율주행

초음파 · 라인 · 컬러 센서 + 다중 센서 퓨전 · PID · 자율주행 실습

5일 과정

1일 7시간

이론 3H + 실습 4H

5일 과정 한눈에 보기 · 오늘은 3일차

1일차

AI 로봇 개론 + ESP32·MicroPython 기초

2일차

포니봇 플랫폼 이해 + 모션 제어

3일차

센서 통합·환경 인지 + 센서 기반 자율주행

4일차

AI 웹앱 연동 (객체·음성·포즈 인식)

5일차

미니 프로젝트 & 발표

※ 2일차 '주행 제어'가 오늘 '스스로 판단하는 자율주행'으로 발전합니다.

[05] 3H

센서 통합 및 환경 인지

초음파·라인·컬러 센서의 원리와 활용, 다중 센서 퓨전, 환경 인지 알고리즘 설계를 배웁니다. (이론)

환경 인지란? — 로봇의 '감각'

로봇이 여러 센서로 주변을 측정하고, 그 데이터를 해석해 '상황을 이해'하는 것이 환경 인지입니다. (인지 → 판단 → 제어의 첫 단계)

초음파 센서

무엇을 측정?
앞 물체까지의 거리(cm)



→ 장애물 회피

라인 센서

무엇을 측정?
바닥의 검은 선 감지(흑/백)



→ 경로 추종

컬러 센서

무엇을 측정?
바닥/물체의 색(RGB)



→ 구역·객체 분류

오늘은 이 세 센서를 '함께' 써서 스스로 길을 찾고 임무를 수행하는 로봇을 만듭니다.

초음파 거리 센서 원리

- 사람이 못 듣는 높은 소리(초음파, 약 40kHz)를 '발사'합니다.
- 물체에 부딪혀 반사되어 '돌아온' 시간을 측정합니다.
- 거리 = (왕복 시간 × 소리 속도) ÷ 2. 소리 속도 ≈ 340 m/s.



포니봇 초음파 센서

측정 범위 약 2cm ~ 400cm. 연결 핀: Trig=P13(GPIO18), Echo=P14(GPIO19). 측정 범위 밖이면 -1 반환.

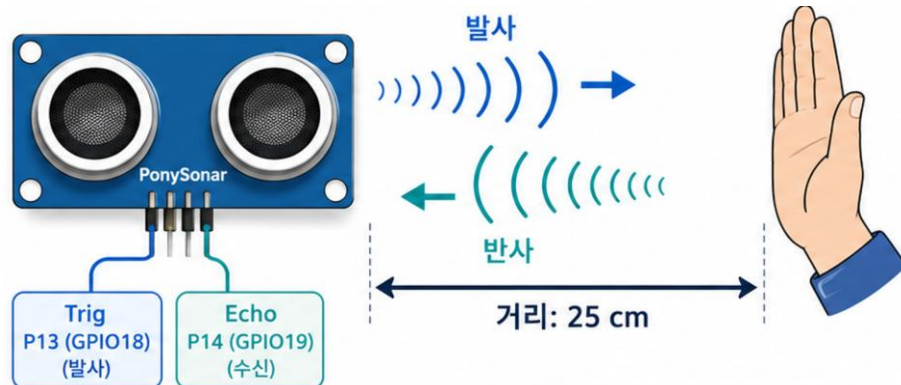
[기초] 초음파 거리 측정

sonar.py

```
from ai_ponybot import i2c, PonySonar
from machine import Pin
from time import sleep

sonar = PonySonar()
trig = Pin(18, Pin.OUT)   # P13 송신
echo = Pin(19, Pin.IN)    # P14 수신

while True:
    d = sonar.measure(trig, echo)
    print('거리:', d, 'cm') # 없으면 -1
    sleep(0.2)
```



동작

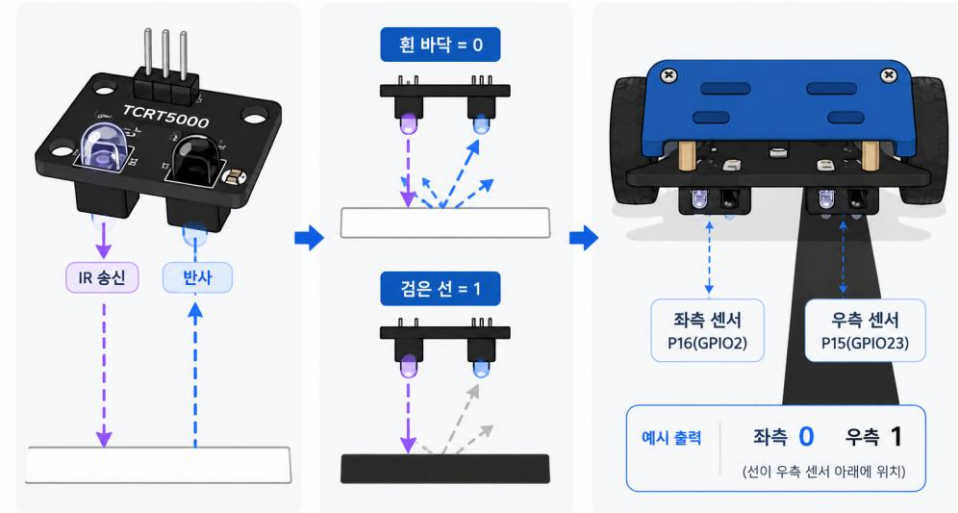
앞에 손을 대면 거리(cm)가 줄고, 치우면 -1이 나옵니다.

도전

거리에 따라 다른 메시지 출력(가까움/보통/멈). Plotter로 그래프 보기.

라인 트레이싱 센서 원리 (TCRT5000)

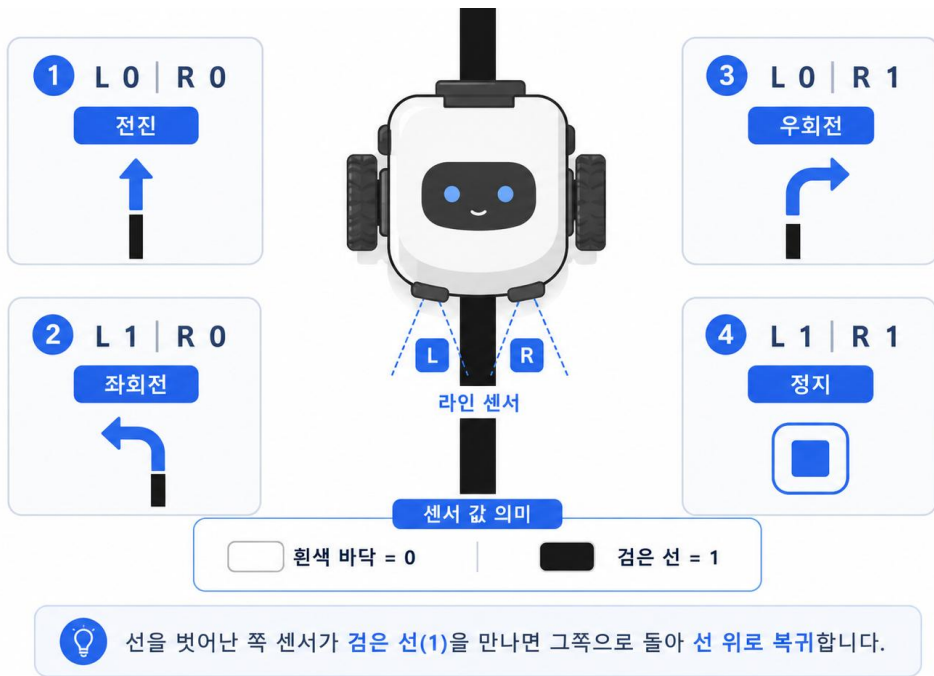
- 적외선(IR) 송신부에서 빛을 쏘고, 바닥에서 반사된 빛을 수신부가 받습니다.
- 흰색 바닥은 빛을 잘 반사(값 0), 검은 선은 빛을 흡수(값 1)해 '흑/백'을 구분.
- 포니봇은 좌·우 2개의 라인 센서로 검은 선의 위치를 판단합니다.



연결 핀 (디지털 입력)

좌측 라인센서 = P16(GPIO2), 우측 라인센서 = P15(GPIO23). 값은 0(흰색)/1(검은 선)으로 읽힙니다.

라인 인식 주행 원리 & 값 읽기



line_read.py

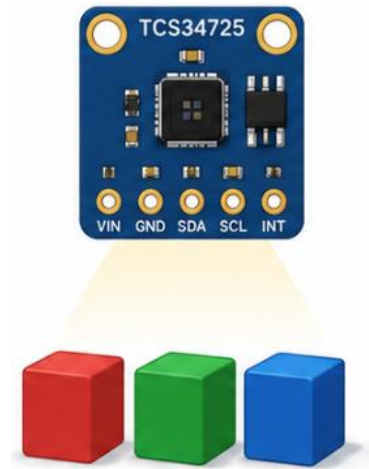
```
from machine import Pin
from time import sleep
```

```
L = Pin(2, Pin.IN) # 좌 P16
R = Pin(23, Pin.IN) # 우 P15
```

```
while True:
    l = L.value()
    r = R.value()
    print('좌:', l, '우:', r)
    sleep(0.1)
```

컬러 센서 기반 객체 분류 (TCS34725)

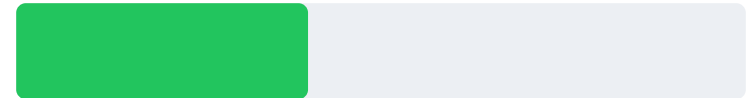
- 컬러 센서는 빛의 3원색 R(빨강)·G(초록)·B(파랑) 양을 각각 측정합니다.
- 세 값의 비율로 '무슨 색인지' 판단 → 바닥 구역·물체를 색으로 분류.
- I2C로 연결(주소 0x29). rgb()로 [R,G,B], is_color()로 색 판별.



R



G



B



색 = R·G·B 비율의 조합

객체 분류란?

'이 색이면 A 구역, 저 색이면 B 구역'처럼 색으로 물체·구역을 나누는 것. 물류 분류 로봇의 핵심 기능입니다.

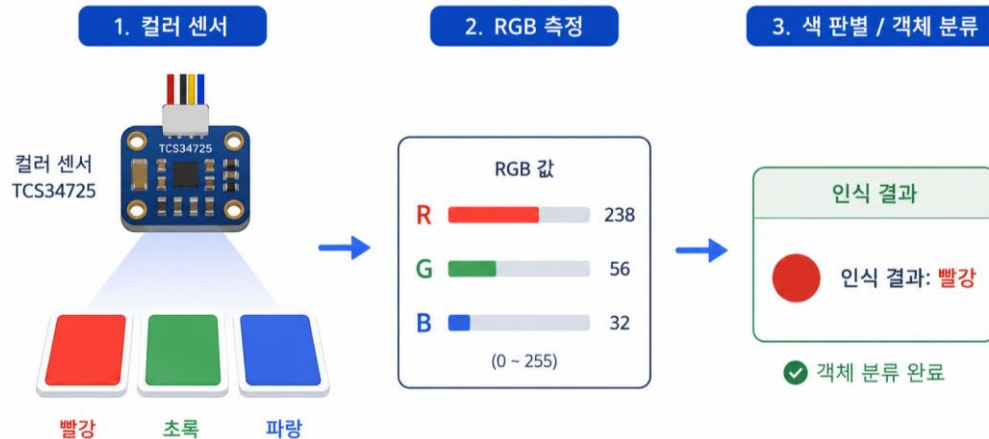
[기초] 컬러 인식 / 객체 분류

color.py

```
from ai_ponybot import i2c, PonyColor
from time import sleep

color = PonyColor(i2c)
color.set_integration_time(150) # 정확도

while True:
    r, g, b = color.rgb()
    print('RGB:', r, g, b)
    if color.is_color('red'): print('빨강!')
    elif color.is_color('green'): print('초록!')
    elif color.is_color('blue'): print('파랑!')
    sleep(0.3)
```



동작

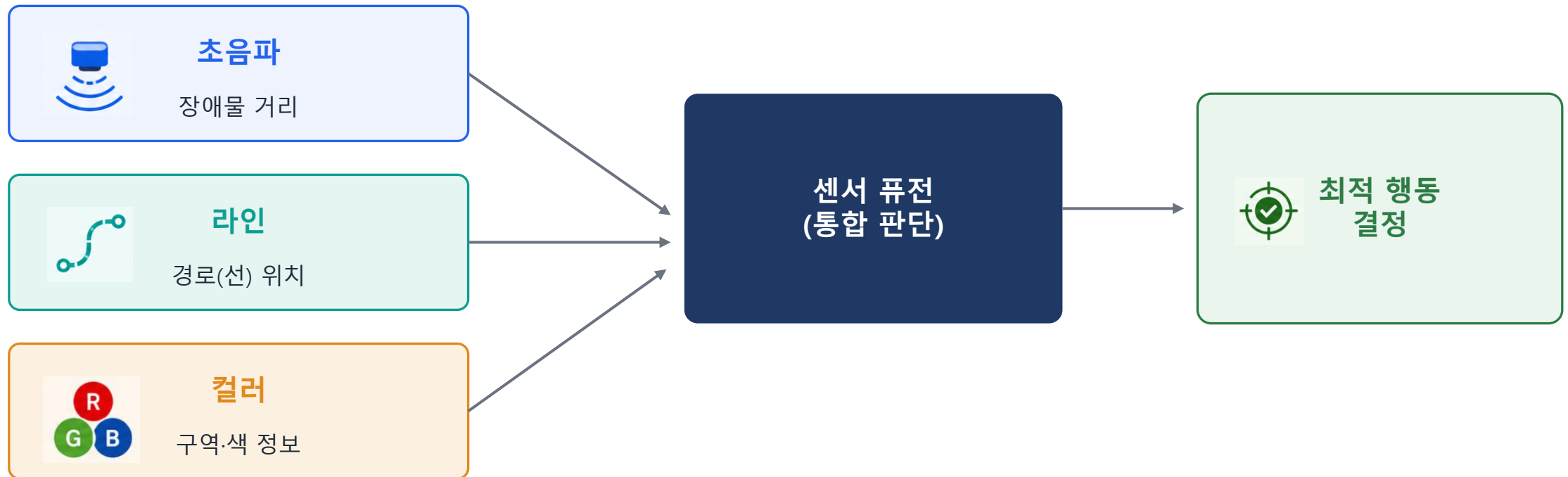
센서에 빨강/초록/파랑 물체를 대면 해당 색 이름이 출력됩니다.

도전

노랑(yellow)도 추가. RGB 값을 OLED·LED로 표시해 보기.

다중 센서 퓨전 (Multi-sensor Fusion)

여러 센서의 정보를 '합쳐서' 더 똑똑하게 판단하는 것. 한 센서의 약점을 다른 센서가 보완합니다.



예) 라인을 따라가다(라인) 앞에 장애물이 있으면(초음파) 멈추고, 빨강 구역(컬러)에서 임무를 수행.

환경 인지 알고리즘 설계 방법론

자율주행 프로그램은 보통 '센싱 → 해석 → 판단 → 행동'의 순환으로 설계합니다.



설계 팁

'상태(state)'를 두면 깔끔합니다. 예: 주행중 / 장애물정지 / 구역작업 — 상황에 따라 상태를 바꾸며 행동을 결정.

[06] 4H

센서 기반 자율주행 실습

장애물 회피·라인 트레이싱·PID 제어를 익히고, 컬러 구역 분류까지 다양한 자율주행을 직접 구현합니다. (이론 & 실습)

초음파 기반 장애물 회피 알고리즘

앞 거리를 계속 재면서 '안전 거리'보다 가까워지면 멈추고 방향을 바꿔 피합니다.



[실습 1] 초음파 장애물 회피 주행

avoid.py

```
from ai_ponybot import i2c, PonyMotor, PonySonar
from machine import Pin
from time import sleep

motor = PonyMotor(i2c)
sonar = PonySonar()
trig = Pin(18, Pin.OUT); echo = Pin(19, Pin.IN)
SAFE = 15

while True:
    d = sonar.measure(trig, echo)
    if d == -1 or d > SAFE:
        motor.drive('forward', 55)
    else:
        motor.drive('backward', 50); sleep(0.3)
        motor.drive('right', 55); sleep(0.5)
        sleep(0.05)
```

1 안전 거리 이상 → 전진



2 장애물 가까움



3 후진 후 우회전으로 회피



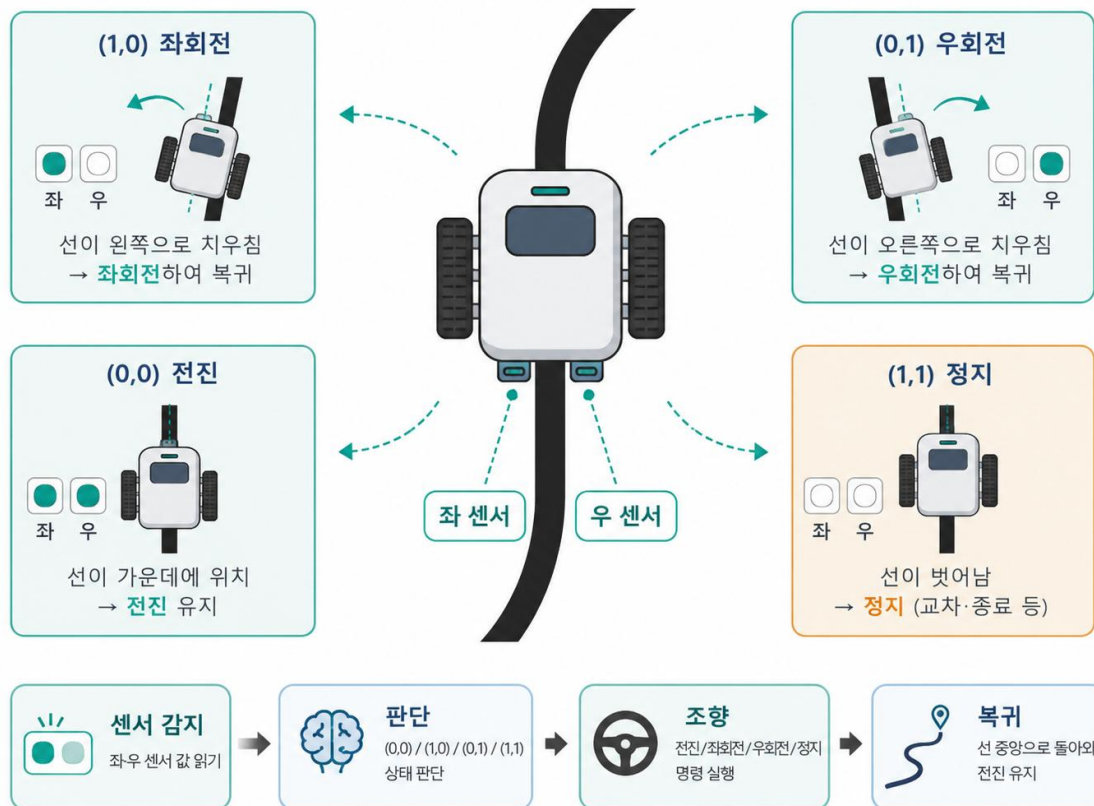
동작

앞이 비면 전진, 장애물이 가까우면 후진 후 우회전으로 피합니다.

도전

SAFE 값 조절. 좌/우 번갈아 회피. 막히면 제자리 회전으로 새 길 찾기.

라인 트레이싱 자율주행 원리



센서 → 동작

(0,0) → 전진
 (1,0) → 좌회전
 (0,1) → 우회전
 (1,1) → 정지(교차)

왜 '추종'이 될까?

선을 벗어나려는 순간 센서가 감지해 곧바로 반대로 돌려주기 때문에 선 위를 유지합니다.

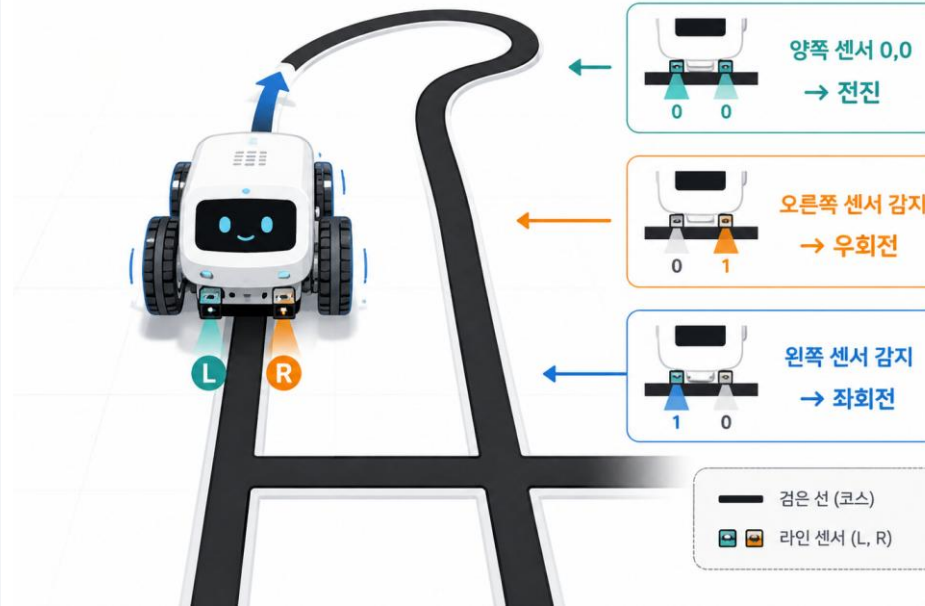
[실습 2] 라인 트레이싱 코스 주행

line_follow.py

```
from ai_ponybot import i2c, PonyMotor
from machine import Pin
from time import sleep
```

```
motor = PonyMotor(i2c)
L = Pin(2, Pin.IN)    # 좌 P16
R = Pin(23, Pin.IN)   # 우 P15
```

```
while True:
    l, r = L.value(), R.value()
    if l == 0 and r == 0:
        motor.drive('forward', 45)
    elif r == 1:
        motor.drive('right', 40)
    elif l == 1:
        motor.drive('left', 40)
    sleep(0.02)
```



동작

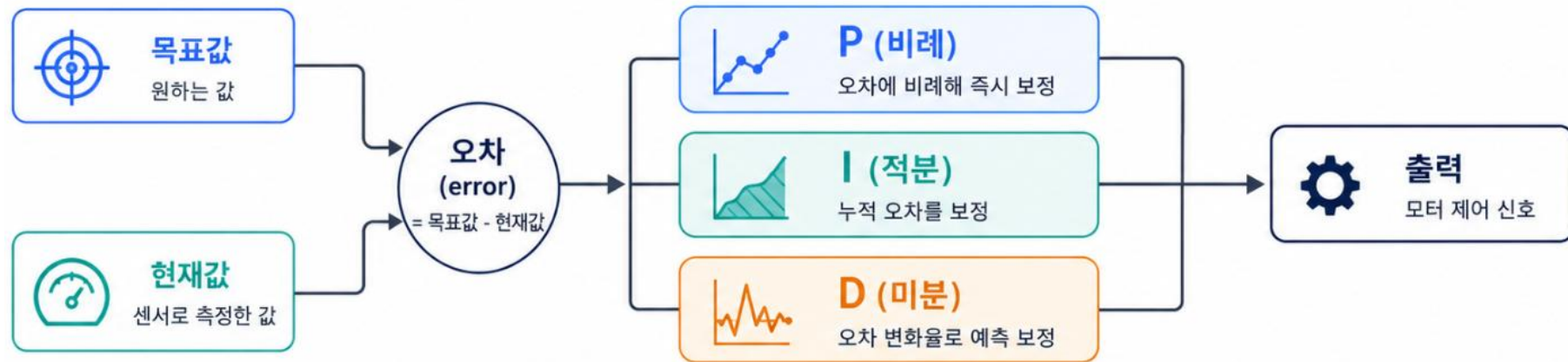
검은 선을 따라 코스를 자동으로 달립니다.

도전

속도를 올려도 선을 놓치지 않게 조정. 곡선·교차 코스 도전.

PID 제어 기초

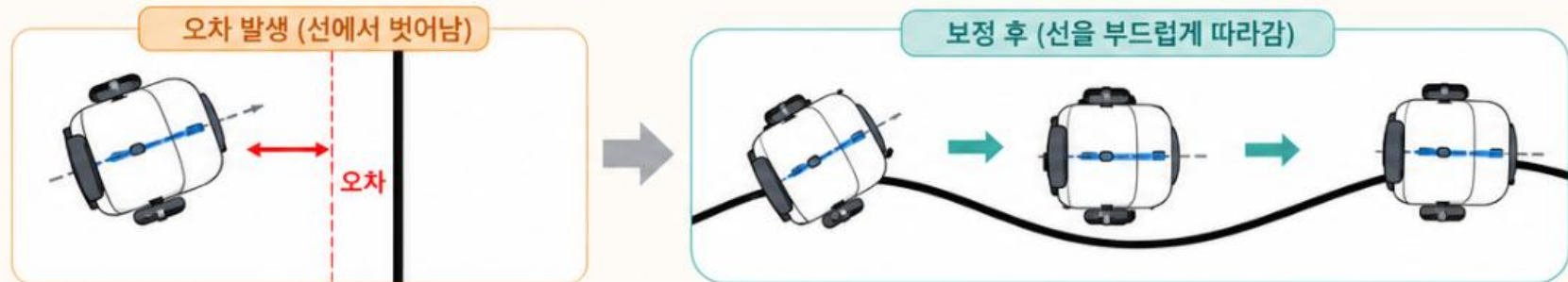
목표값과 현재값의 '오차(error)'를 이용해 출력을 부드럽고 정확하게 보정하는 대표적 제어 기법입니다.



$$\text{출력} = K_p \cdot \text{오차} + K_i \cdot (\text{오차의 합}) + K_d \cdot (\text{오차의 변화})$$

라인 트레이싱 적용

오차에 따라 좌우 바퀴
속도를 조절해
선을 부드럽게 따라갑니다.



[실습 3] P 제어 라인 트레이싱

line_pid.py

```
from ai_ponybot import i2c, PonyMotor
from machine import Pin
from time import sleep

motor = PonyMotor(i2c)
L = Pin(2, Pin.IN); R = Pin(23, Pin.IN)
Kp = 25      # 비례 상수
base = 45    # 기본 속도

while True:
    err = L.value() - R.value()  # +1 왼쪽/-1 오른쪽 치우침
    turn = Kp * err
    motor.move(1, base + turn); motor.move(2, base + turn)  # 우측
    motor.move(3, base - turn); motor.move(4, base - turn)  # 좌측
    sleep(0.02)
```

동작

오차에 비례해 좌·우 바퀴 속도를 조절 → 더 부드럽게 선을 추종합니다.

도전

Kp를 키우면 반응 ↑ (흔들림 ↑), 줄이면 부드럽지만 굽뚝. 최적값 찾기.



[실습 4] 컬러 센서 구역 분류 미션

color_zone.py

```
from ai_ponybot import i2c, PonyMotor, PonyColor, PonyOLED
from time import sleep

motor = PonyMotor(i2c); color = PonyColor(i2c); oled = PonyOLED(i2c)

while True:
    oled.clear()
    if color.is_color('red'):
        oled.write_line(0, 'RED: STOP'); motor.drive('stop')
    elif color.is_color('green'):
        oled.write_line(0, 'GREEN: GO'); motor.drive('forward', 50)
    elif color.is_color('blue'):
        oled.write_line(0, 'BLUE: TURN'); motor.drive('left', 50)
    else:
        oled.write_line(0, 'searching'); motor.drive('forward', 35)
    oled.show(); sleep(0.2)
```



동작

바닥 색에 따라 정지(빨강)·직진(초록)·회전(파랑)하며 OLED에 구역을 표시합니다.

도전

색마다 다른 임무(서보 동작 등) 추가. 노랑 구역도 넣기.

APPLY

센서 응용 실습

초음파·라인·컬러를 다양하게 조합한 응용 자율주행 6종과 종합 미션. (배운 센서만 사용)

[응용 ①] 거리 유지 (따라가기)

keep_dist.py

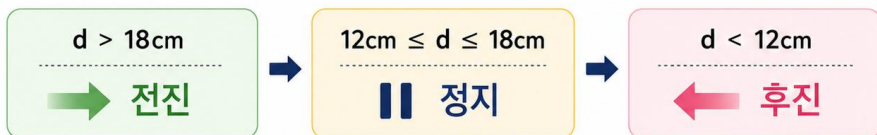
```
from ai_ponybot import i2c, PonyMotor, PonySonar
from machine import Pin
from time import sleep

motor = PonyMotor(i2c); sonar = PonySonar()
trig = Pin(18, Pin.OUT); echo = Pin(19, Pin.IN)
TARGET = 15

while True:
    d = sonar.measure(trig, echo)
    if d == -1:
        motor.drive('stop')
    elif d > TARGET + 3:
        motor.drive('forward', 45) # 멀면 다가감
    elif d < TARGET - 3:
        motor.drive('backward', 45) # 가까우면 물러남
    else:
        motor.drive('stop') # 적정 거리
    sleep(0.05)
```



동작 로직 요약



동작

앞 물체와 약 15cm 거리를 유지합니다. 물체가 다가오면 물러나고, 멀어지면 따라갑니다.

도전

TARGET 거리·여유(±3)를 조절. 사람 손을 따라다니게 만들기.

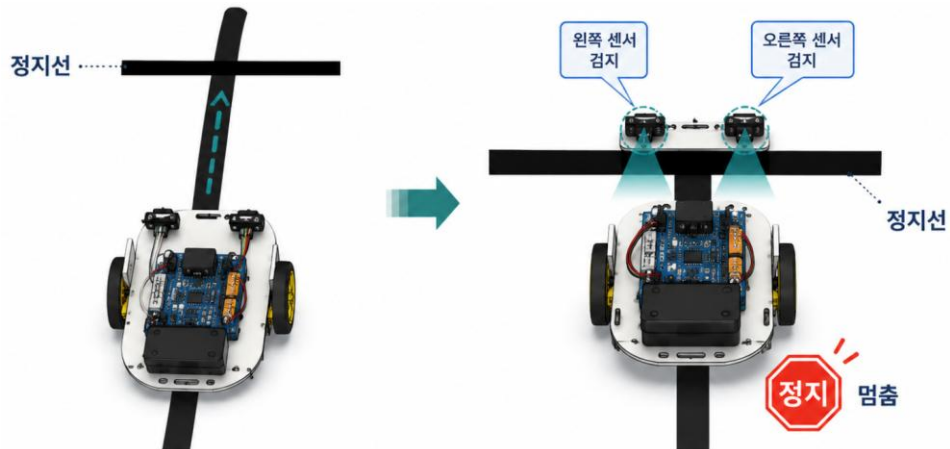
[응용 ②] 정지선에서 멈추기

stopline.py

```
from ai_ponybot import i2c, PonyMotor
from machine import Pin
from time import sleep

motor = PonyMotor(i2c)
L = Pin(2, Pin.IN); R = Pin(23, Pin.IN)

while True:
    l, r = L.value(), R.value()
    if l == 1 and r == 1:      # 양쪽 모두 검정 = 정지선
        motor.drive('stop'); break
    elif l == 0 and r == 0: motor.drive('forward', 45)
    elif r == 1:             motor.drive('right', 40)
    elif l == 1:             motor.drive('left', 40)
    sleep(0.02)
print('정지선 도착!')
```



동작

라인을 따라가다 양쪽 센서가 동시에 선을 만나면(정지선) 멈춥니다.

도전

정지 후 3초 뒤 다시 출발. 정지선 개수를 세어 '몇 번째 역' 표시.

[응용 ③] 색깔 신호등 주행

traffic.py

```
from ai_ponybot import i2c, PonyMotor, PonyColor
from time import sleep

motor = PonyMotor(i2c); color = PonyColor(i2c)

while True:
    if color.is_color('red'):
        motor.drive('stop')          # 빨강 = 정지
    elif color.is_color('green'):
        motor.drive('forward', 50)   # 초록 = 출발
    sleep(0.1)
```

동작

바닥 색이 빨강이면 멈추고, 초록이면 출발하는 신호등 규칙 주행.

도전

노랑 = 천천히(속도 ↓) 추가. OLED에 현재 신호 표시.

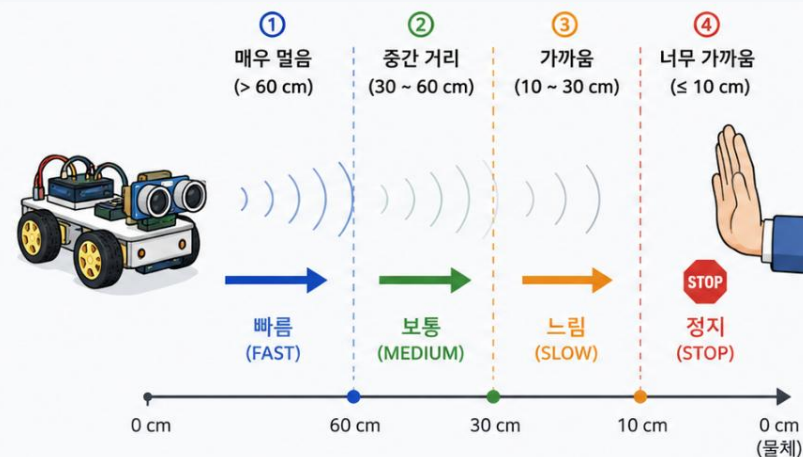
[응용 ④] 거리별 속도 조절

auto_speed.py

```
from ai_ponybot import i2c, PonyMotor, PonySonar
from machine import Pin
from time import sleep

motor = PonyMotor(i2c); sonar = PonySonar()
trig = Pin(18, Pin.OUT); echo = Pin(19, Pin.IN)

while True:
    d = sonar.measure(trig, echo)
    if d == -1: motor.drive('forward', 80) # 비면 빠르게
    elif d < 10: motor.drive('stop') # 너무 가까우면 정지
    else:
        spd = min(d * 2, 90) # 멀수록 빠르게
        motor.drive('forward', spd)
        sleep(0.05)
```



동작

앞이 멀수록 빠르게, 가까울수록 느리게 — 거리로 속도를 부드럽게 조절합니다.

도전

$d \times 2$ 대신 다른 식으로 속도 곡선 바꾸기. 최소·최고 속도 제한.

[응용 ⑤] 라인 추종 + 장애물 정지

line_safe.py

```

from ai_ponybot import i2c, PonyMotor, PonySonar
from machine import Pin
from time import sleep

motor = PonyMotor(i2c); sonar = PonySonar()
trig = Pin(18, Pin.OUT); echo = Pin(19, Pin.IN)
L = Pin(2, Pin.IN); R = Pin(23, Pin.IN)

while True:
    d = sonar.measure(trig, echo)
    if d != -1 and d < 12:          # 전방 장애물 → 정지
        motor.drive('stop'); sleep(0.1); continue
    l, r = L.value(), R.value()    # 아니면 라인 추종
    if l == 0 and r == 0: motor.drive('forward', 45)
    elif r == 1:              motor.drive('right', 40)
    elif l == 1:              motor.drive('left', 40)
    sleep(0.02)

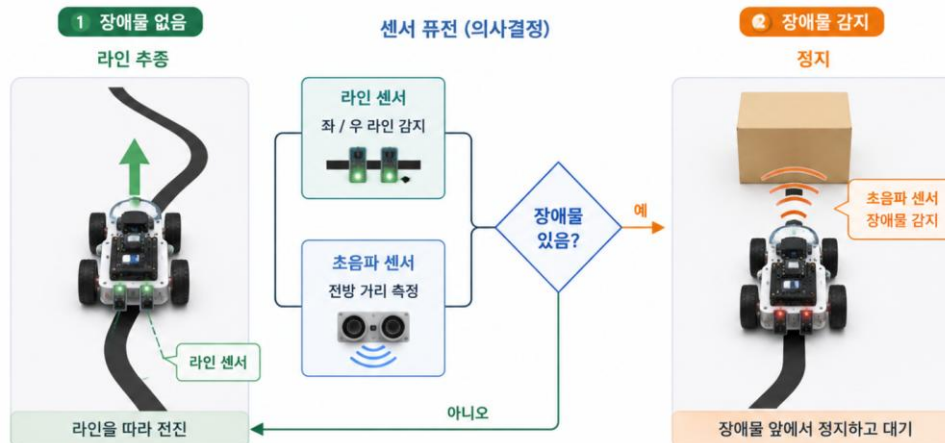
```

동작

라인을 따라가다 앞에 장애물이 나타나면 멈추고, 치워지면 다시 출발 (센서 퓨전).

도전

장애물이 있으면 정지 대신 옆으로 피해서 다시 라인 찾기 도전.



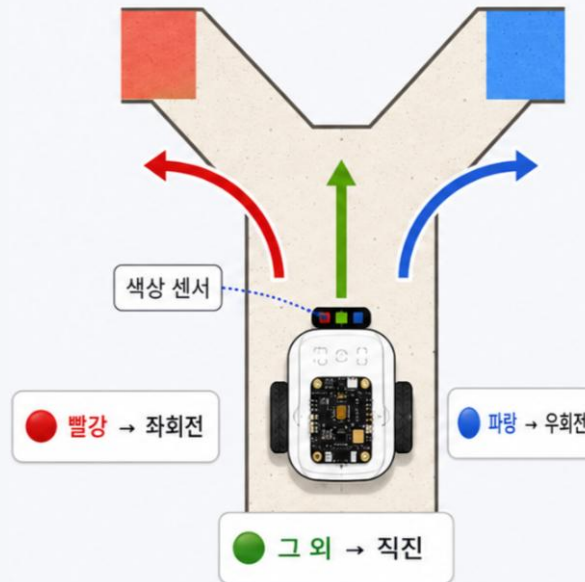
[응용 ⑥] 컬러 구역 분기 주행

color_branch.py

```
from ai_ponybot import i2c, PonyMotor, PonyColor
from time import sleep

motor = PonyMotor(i2c); color = PonyColor(i2c)

while True:
    if color.is_color('red'):
        motor.drive('left', 50); sleep(0.6)
    elif color.is_color('blue'):
        motor.drive('right', 50); sleep(0.6)
    else:
        motor.drive('forward', 50)
        sleep(0.05)
```



동작

바닥 색에 따라 갈림길에서 다른 방향으로 분기합니다 (색=경로 신호).

도전

색 3가지로 3방향 분기. 색 순서를 기억해 미로 탈출 도전.

종합 미션 — 스마트 물류 로봇

라인 추종 + 장애물 정지 + 컬러 구역 분류를 모두 결합한 자율주행 물류 로봇입니다.

logistics_bot.py

```
from ai_ponybot import i2c, PonyMotor, PonySonar, PonyColor, PonyOLED
from machine import Pin
from time import sleep

motor = PonyMotor(i2c); sonar = PonySonar()
color = PonyColor(i2c); oled = PonyOLED(i2c)
trig = Pin(18, Pin.OUT); echo = Pin(19, Pin.IN)
L = Pin(2, Pin.IN); R = Pin(23, Pin.IN)

def status(m):
    oled.clear(); oled.write_line(0, 'Logistics Bot')
    oled.write_line(1, m); oled.show()

while True:
    d = sonar.measure(trig, echo)
    if d != -1 and d < 12:          # 1) 장애물 정지
        motor.drive('stop'); status('Obstacle!'); sleep(0.1); continue
    if color.is_color('red'):       # 2) 빨강 구역 = 임무
        motor.drive('stop'); status('RED: drop'); sleep(1); continue
    if color.is_color('green'): status('GREEN zone')
    l, r = L.value(), R.value()     # 3) 평소엔 라인 추종
    if l == 0 and r == 0: motor.drive('forward', 45)
    elif r == 1: motor.drive('right', 40)
    elif l == 1: motor.drive('left', 40)
    sleep(0.02)
```

동작 시나리오

검은 선을 따라가다 → 앞에 장애물이
면 정지 → 빨강 구역에서 '하역(정지)'
→ 초록 구역 통과 표시.

확장 도전

서보(그리퍼)로 실제 집기·놓기. 정지
선으로 구역 카운트. 버튼으로 시작/
정지. OLED에 운행 로그.

3일차 정리 & 다음 차시

오늘 한 것

- 초음파·라인·컬러 센서의 원리와 값 읽기, 다중 센서 퓨전 개념
- 환경 인지 알고리즘 설계(센싱→해석→판단→행동) 방법론
- 장애물 회피·라인 트레이싱·P(PID) 제어·컬러 구역 분류 자율주행 구현
- 센서를 조합한 응용 6종 + 스마트 물류 로봇 종합 미션

다음 차시 (4일차) 스마트폰의 **AI 웹앱(객체·음성·포즈 인식)**을 포니봇과 연동해, 카메라로 보고 음성으로 듣는 더 똑똑한 로봇으로 확장합니다.

멀티모달 인지 기반 피지컬 AI 로봇 시스템 구현 과정

4일차 강의자료

로봇 연동 AI 웹앱 서비스 · AI 기반 로봇 제어

Web Bluetooth · 음성(Web Speech/Whisper) · 손 제스처(MediaPipe) · 객체 인식 · 다양한 AI 예제

이론 3H + 실습 4H

실습 가능한 웹앱 포함

AI 예제 다수

5일 과정 한눈에 보기 · 오늘은 4일차

1일차

AI 로봇 개론 + ESP32·MicroPython 기초

2일차

포니봇 플랫폼 이해 + 모션 제어

3일차

센서 통합·환경 인지 + 센서 자율주행

4일차

로봇 연동 AI 웹앱 + AI 기반 로봇 제어

5일차

미니 프로젝트 & 발표

※ '스마트폰의 AI가 보고 듣고 판단해 포니봇을 조종한다' — 실제로 동작하는 웹앱을 직접 만들어 봅니다.

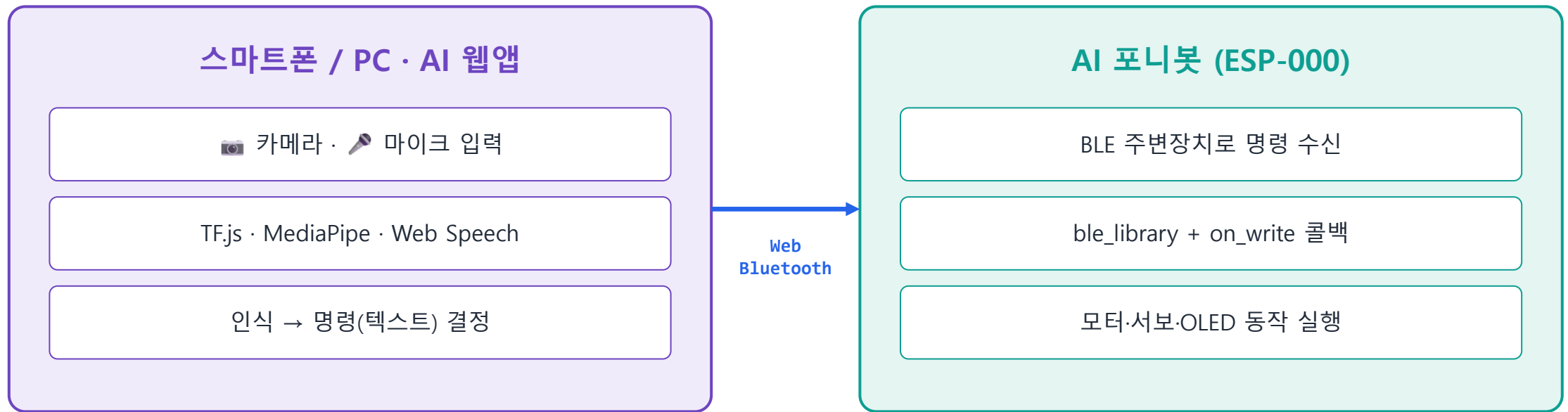
[07] 3H

로봇 연동 AI 웹앱 서비스

통신(Web Bluetooth/Serial)과 Edge AI(TF.js·MediaPipe), 그리고 객체·포즈·손·얼굴·음성 인식 모델을 폭넓게 살펴봅니다. (이론)

전체 구조 — AI 웹앱이 포니봇을 조종한다

스마트폰/PC의 'AI 웹앱'이 카메라·마이크로 인식하고, 그 결과를 무선(Web Bluetooth)으로 포니봇에 명령합니다.



→ 무거운 AI는 '브라우저'가, 빠른 동작은 '포니봇'이. 새 AI를 붙일 땐 '인식→명령'만 바꾸면 됩니다.

Edge AI — 왜 브라우저에서 AI를 돌릴까

서버(클라우드)로 보내지 않고 기기/브라우저에서 직접 AI를 실행하는 방식입니다.

빠름

네트워크 지연 없이
실시간 처리

사생활 보호

영상·음성이 기기
밖으로 안 나감

간편

앱 설치 없이
웹페이지로 실행

무료

별도 서버·요금
불필요

대표 도구

- TensorFlow.js — 브라우저(자바스크립트)에서 딥러닝 모델 실행
 - MediaPipe — 손·얼굴·포즈 실시간 추적 (구글)
 - Web Speech API — 브라우저 내장 음성 인식
- 이 도구들로 카메라·마이크 입력을 그 자리에서 AI로 처리합니다.

AI가 할 수 있는 일 — 보기 · 듣기 · 자세 읽기

이번 차시의 AI는 입력에 따라 크게 셋으로 나뉩니다. 만들 주제에 맞춰 골라 씁니다.

보기 (카메라)

MobileNet · COCO-SSD
FaceMesh

→ 물체·사람·표정 인식

듣기 (마이크)

Web Speech · Whisper
소리 분류

→ 음성 명령·소리 인식

자세 (카메라)

MediaPipe Hands
MediaPipe Pose

→ 손·몸 동작 인식

어떤 입력이든 결과는 '명령'으로 바뀌어 같은 방식으로 포니봇에 전달됩니다.

Web Bluetooth / Web Serial 통신

웹앱(브라우저)이 별도 앱 없이 '웹 표준 API'로 로봇과 직접 통신합니다.

Web Bluetooth



- 무선(BLE)으로 연결
- 스마트폰·노트북에서 자유 이동
- 포니봇이 BLE 기기 'ESP-000'
- 오늘 실습의 기본 방식

Web Serial



- USB 케이블로 유선 연결
- 연결이 안정적·빠름
- 선이 있어 이동은 제한적
- 고정형 테스트·디버깅에 적합

※ 둘 다 '브라우저에서 직접' 통신 — 설치 없이 웹페이지가 로봇과 대화합니다. (Chrome/Edge 지원)

BLE 통신 구조 자세히 — 포니봇은 '주변장치'

BLE는 '중앙(Central)'이 '주변장치(Peripheral)'에 연결해 데이터를 주고받습니다.



서비스 / 특성 (Service / Characteristic)

포니봇은 표준 'Nordic UART 서비스(NUS)'를 사용합니다.

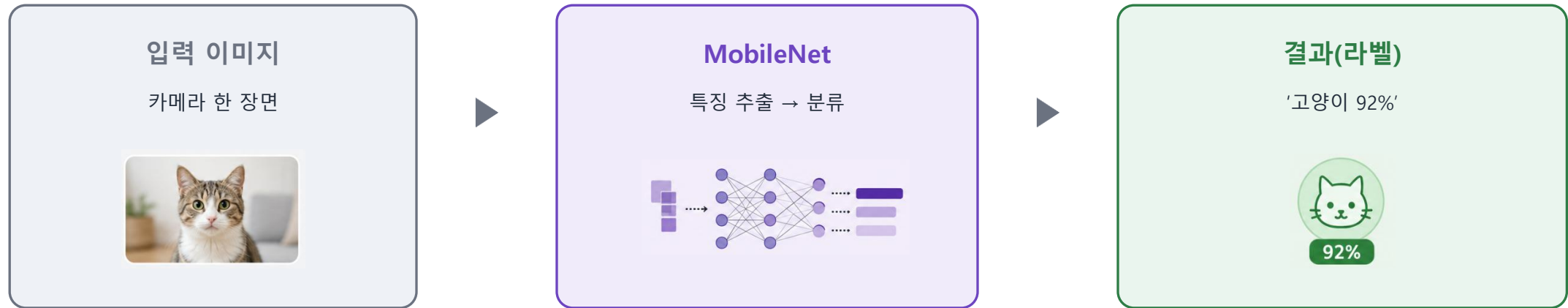
· 서비스 UUID: 6E400001-... · RX(쓰기): 6E400002-... · TX(알림): 6E400003-...

웹앱은 RX 특성에 명령 문자(예: 'forward')를 '쓰고', 포니봇은 on_write 콜백으로 그 값을 받습니다.

→ 이 규칙(UUID·기기이름)은 ble_library.py가 처리하므로, 우리는 '명령 문자열'만 신경 쓰면 됩니다.

MobileNet — 객체 분류 (Image Classification)

이미지를 보고 '무엇인지' 한 가지로 분류하는 가벼운 모델입니다.



특징

가벼워서 스마트폰·브라우저에서도 실시간. 미리 학습된 모델을 바로 사용(TF.js).



로봇 활용

본 물체의 종류를 인식해 '이건 따라가/저건 피해' 같은 명령으로 변환.

COCO-SSD — 객체 검출 (위치·크기)

MobileNet이 '무엇인지'만 본다면, COCO-SSD는 '무엇이 화면 어디에, 얼마나 크게' 있는지까지 찾습니다.

- 여러 물체를 동시에 찾고, 각 물체의 박스 좌표(x,y,w,h)와 종류·신뢰도를 줍니다.
- 박스의 '가로 위치' → 좌/우 판단, '크기' → 가까움/멈춤 판단.
- '사람 추종 로봇'의 핵심 모델 (TF.js로 브라우저 실행).



핵심 포인트

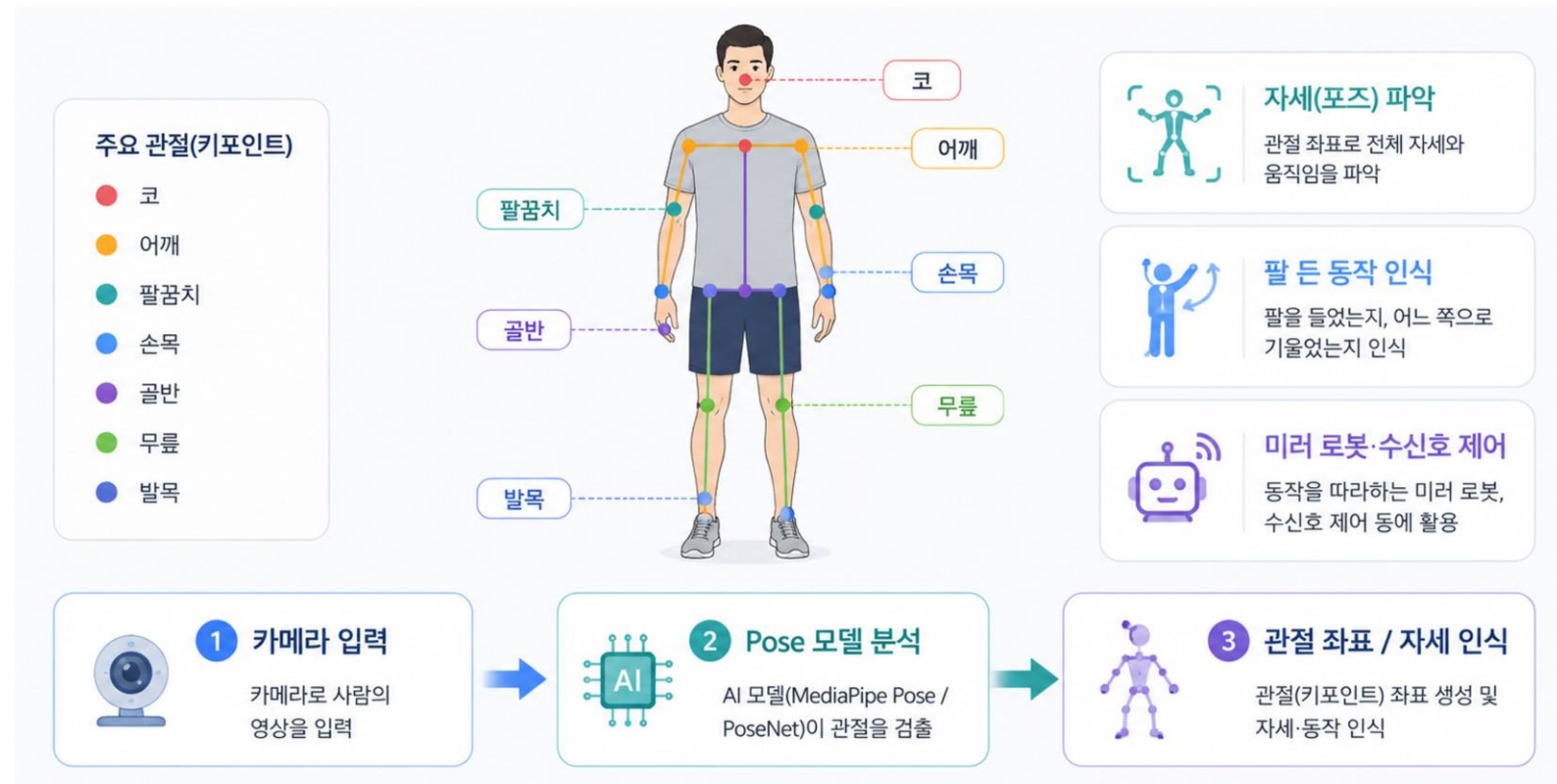
- ✓ 큰 박스 = 더 가까이 있음
- ✓ 왼쪽/오른쪽 위치 = 이동 방향 결정



MediaPipe Pose / PoseNet — 전신 관절

사람의 코·어깨·팔꿈치·손목·무릎 등 '관절(키포인트)' 좌표를 실시간으로 찾습니다.

- 관절 좌표로 '자세(포즈)'와 '동작'을 파악
- 팔을 들었는지, 어느 쪽으로 기울었는지 인식
- 동작을 따라하는 '미러 로봇', 수신호 제어 등에 활용



MediaPipe Hands — 손 관절 21개

한 손의 관절 21개(손목·마디·손끝) 좌표를 실시간으로 찾습니다. '손 제스처 제어'의 핵심.



- 손끝과 마디의 좌표를 비교해 '펴진 손가락 개수'나 '손 모양'을 판별
- 예: 손끝(y)이 마디(y)보다 위에 있으면 그 손가락은 '펴짐'
- 개수·모양을 명령으로 매핑 → 손짓만으로 로봇 조종 (오늘 실습에서 사용)

MediaPipe FaceMesh — 얼굴/표정

얼굴의 수백 개 점(랜드마크)으로 표정·고개 방향·눈/입 움직임을 인식합니다.



무엇을 알 수 있나

웃음/찡그림 등 표정, 고개를 돌린 방향, 눈 깜빡임·입 벌림 등.



로봇 활용

표정에 반응(다가오기/물러나기), 고개 방향으로 주행, OLED 표정 미러링.



예시 아이디어

- 웃으면 다가오고 무표정이면 정지하는 '교감 로봇'
- 고개를 왼쪽으로 돌리면 좌회전 — 손이 자유로운 핸즈프리 조종
- 졸음(눈 감김) 감지 알림 등 안전 응용

음성 명령 인식 개요

음성을 텍스트로 바꾼 뒤(STT), 키워드를 명령으로 매핑합니다. 음성 → 텍스트 → 명령 → 전송.



두 가지 키워드 방식

- Keyword Spotting — 정해진 단어 몇 개만 인식(가볍고 빠름, 오프라인)
- 전체 음성 인식(STT) — 자유 발화를 텍스트로(정확, 표현 다양). 우리는 Web Speech/Whisper 사용.
→ 어느 쪽이든 '인식된 텍스트에서 키워드를 찾아 명령으로 바꾸는' 단계는 동일합니다.

음성 인식 옵션 비교 — 무엇을 쓸까

방식	정확도	속도	오프라인	비고
Web Speech API	높음(한국어)	실시간	✗ (인터넷)	Chrome·무설치
Whisper(브라우저)	중~높음	보통	✓	모델 다운로드(base 권장)
클라우드 STT	최고	보통	✗	API 키·비용(CLOVA 등)

추천

명령 제어용이면 Web Speech API(빠름·정확·간편)가 1순위. 오프라인이 꼭 필요하다면 Whisper(base/small).

Teachable Machine — 나만의 AI 학습

코딩 없이 '내가 모은 데이터'로 AI를 직접 학습시키고, 브라우저(TF.js)로 바로 쓸 수 있습니다.



왜 좋은가 (수업 추천)

이미지·포즈·소리 모델을 직접 만들 수 있어 '데이터→학습→추론'의 AI 원리를 체험하기 최고.

예) 내가 만든 표지판 카드 인식 → 주행, 박수 소리 분류 → 명령. (teachablemachine.withgoogle.com)

AI 모델 한눈에 비교

모델	입력	출력	로봇 활용	실행
MobileNet	이미지	분류 라벨	객체 인식	TF.js
COCO-SSD	이미지	객체+위치(박스)	사람 추종	TF.js
MediaPipe Hands	영상	손 관절 21	제스처 제어	MediaPipe
MediaPipe Pose	영상	전신 관절	모션 연동	MediaPipe
FaceMesh	영상	얼굴 랜드마크	표정 반응	MediaPipe
Web Speech	음성	텍스트	음성 명령	브라우저
Teachable Machine	사용자 데이터	커스텀 분류	맞춤 인식	TF.js

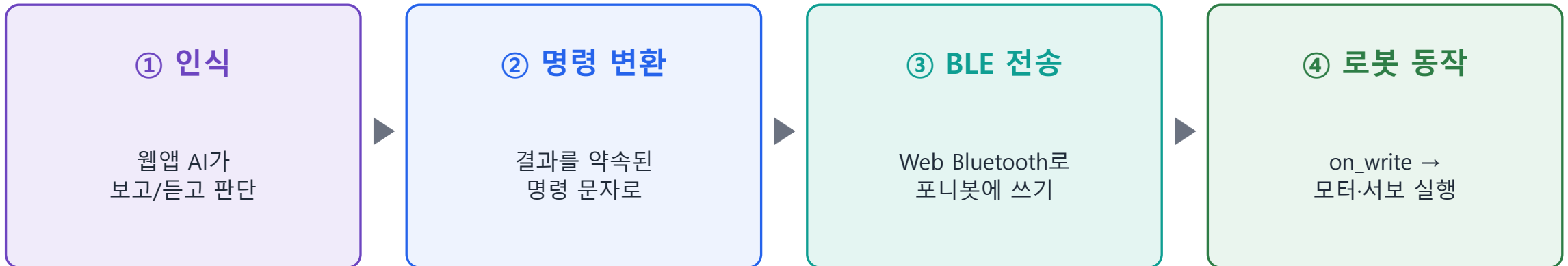
[08] 4H

AI 기반 로봇 제어 — 실습

BLE 연결부터 음성·손 제스처·객체 인식까지, 실제로 동작하는 웹앱을 만들고 포니봇을 제어합니다. (이론 & 실습)

AI → 로봇 제어 연동 흐름

'인식 → 명령으로 변환 → BLE 전송 → 로봇 동작' 4단계. 새 AI를 붙여도 이 골격은 같습니다.



오늘의 약속된 명령

forward(전진) · backward(후진) · left(좌회전) · right(우회전) · stop(정지)
웹앱은 이 문자열을 보내고, 포니봇은 같은 문자열을 받아 동작합니다.

실습 준비물 & 환경

포니봇 + Thonny

ai_ponybot.py, ble_library.py 두 파일이 보드에 업로드돼 있어야 함

브라우저

Chrome 또는 Edge (안드로이드 Chrome 포함). iOS Safari는 Web Bluetooth 미지원

HTTPS 주소

마이크·카메라는 HTTPS에서만 동작 → GitHub Pages 등에 호스팅

인터넷

AI 모델(CDN)·Web Speech 사용 시 필요

※ 먼저 '수동 버튼 → BLE 연결'이 되는지 확인한 뒤 AI(음성·카메라)를 켜면 문제를 좁히기 쉽습니다.

포니봇 BLE 코드 — 명령 받기

ble_test.py

```

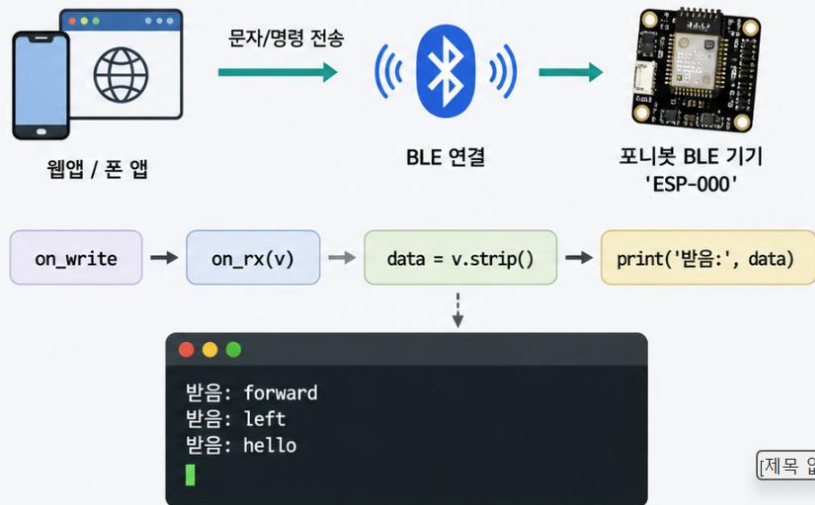
from time import sleep
import ble_library      # 보드에 업로드 필요
import bluetooth

ble = bluetooth.BLE()
p = ble_library.BLESimplePeripheral(ble, 'ESP-000')

def on_rx(v):           # 데이터 받으면 호출(콜백)
    data = v.strip()    # 받은 텍스트
    print('받음:', data)

p.on_write(on_rx)
while True:
    sleep(1)

```



동작

포니봇이 'ESP-000' BLE 기기가 됩니다. 웹앱/폰 앱에서 글자를 보내면 셀에 출력됩니다.

도전 / 팁

ble_library.py가 없으면 import 오류가 납니다 — ai_ponybot.py와 같은 곳에 업로드하세요.

포니봇 명령 핸들러 — 표준 동작 매핑

robot.py

```
from ai_ponybot import i2c, PonyMotor, PonyOLED
import ble_library, bluetooth

motor = PonyMotor(i2c); oled = PonyOLED(i2c)
ble = bluetooth.BLE()
p = ble_library.BLESimplePeripheral(ble, 'ESP-000')

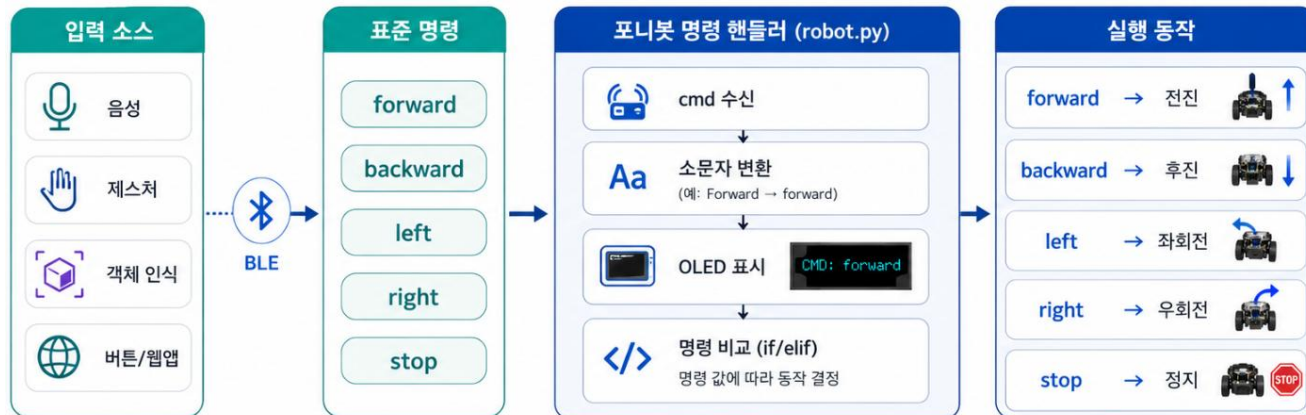
def on_rx(v):
    cmd = v.strip().lower()
    oled.clear(); oled.write_line(0, 'CMD: ' + cmd); oled.show()
    if cmd == 'forward': motor.drive('forward', 50)
    elif cmd == 'backward': motor.drive('backward', 50)
    elif cmd == 'left': motor.drive('left', 45)
    elif cmd == 'right': motor.drive('right', 45)
    elif cmd == 'stop': motor.drive('stop')
p.on_write(on_rx)
while True: pass
```

동작

오늘 만드는 모든 웹앱(음성·제스처·객체)이 이 한 개의 로봇 코드와 함께 동작합니다.

도전 / 팁

명령에 'spin', 'dance' 등을 추가해 동작을 늘려 보세요. 서보 명령도 추가 가능.



동작 명령 확장 — 서보·표정 추가

robot_ext.py

```
from ai_ponybot import i2c, PonyMotor, PonyServo, PonyOLED
import ble_library, bluetooth
```

```
motor = PonyMotor(i2c); servo = PonyServo(motor.pwm)
oled = PonyOLED(i2c)
ble = bluetooth.BLE()
p = ble_library.BLESimplePeripheral(ble, 'ESP-000')
```

```
def on_rx(v):
    c = v.strip().lower()
    if c == 'grab':
        servo.set_angle(1, 120) # 그리퍼 닫기
    elif c == 'release':
        servo.set_angle(1, 0) # 열기
    elif c == 'happy':
        oled.clear()
        oled.write_line(0, '^_^')
        oled.show()
    elif c == 'spin':
        motor.drive('left', 60)
    else:
        motor.drive(c, 50)
p.on_write(on_rx)
while True: pass
```



동작

주행 외에 'grab/release(그리퍼)', 'happy(표정)', 'spin(회전)' 같은 명령도 받게 확장.

도전 / 팁

AI 인식 결과를 이런 새 명령에 연결하면 표현이 풍부해집니다. 나만의 명령을 추가해보세요.

웹앱 측 — Web Bluetooth 연결 & 전송 패턴

모든 AI 웹앱이 공유하는 핵심 코드입니다. (연결 → 명령 쓰기)

ble.js

```
const NUS = '6e400001-b5a3-f393-e0a9-e50e24dcca9e';
const RX = '6e400002-b5a3-f393-e0a9-e50e24dcca9e';
let rxChar = null;

async function connect() {
  const dev = await navigator.bluetooth.requestDevice({
    filters: [{ name: 'ESP-000' }], optionalServices: [NUS] });
  const server = await dev.gatt.connect();
  const svc = await server.getPrimaryService(NUS);
  rxChar = await svc.getCharacteristic(RX);
}

async function send(cmd) { // 명령 전송
  await rxChar.writeValueWithoutResponse(
    new TextEncoder().encode(cmd));
}
```



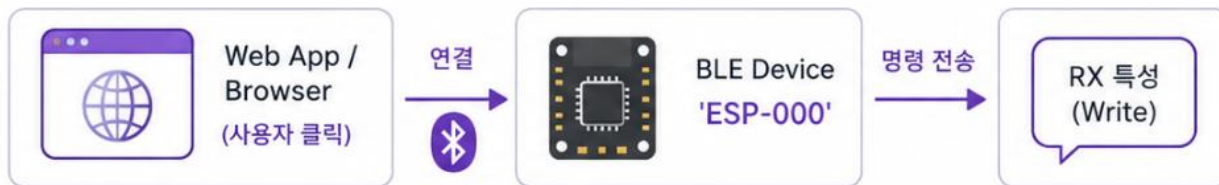
포인트

- name 'ESP-000'으로 찾기
- RX 특성에 명령 '쓰기'
- send('forward') 형태로 사용



주의

requestDevice는 버튼 클릭 같은 사용자 동작에서만 호출 가능. HTTPS 필요.



[실습 A] 버튼 원격 제어 웹앱

buttons.html

```
<!-- 버튼을 누르면 BLE로 명령 전송 (가장 단순) -->
<button onclick="connect()">연결</button>

<button onclick="send('forward')">전진</button>
<button onclick="send('left')">좌회전</button>
<button onclick="send('stop')">정지</button>
<button onclick="send('right')">우회전</button>
<button onclick="send('backward')">후진</button>

<!-- connect(), send()는 앞 슬라이드 ble.js 사용 -->
```

동작

버튼만으로 포니봇을 원격 조종합니다. AI를 붙이기 전, 연결·동작 확인용으로 먼저 만듭니다.

도전 / 팁

이게 되면 BLE 경로가 정상! 이제 버튼 대신 'AI 인식 결과'가 send()를 호출하게 바꾸면 됩니다.

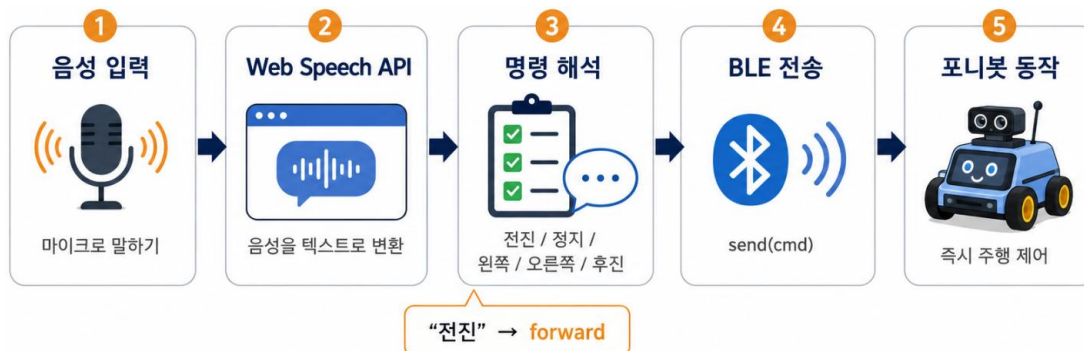


[실습 B] 음성 제어 — Web Speech API

voice.js

```
const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
const rec = new SR();
rec.lang = 'ko-KR';
rec.continuous = true;      // 계속 듣기
rec.interimResults = true;  // 빠른 반응

rec.onresult = (e) => {
  let t = '';
  for (let i = e.resultIndex; i < e.results.length; i++)
    t += e.results[i][0].transcript;
  const cmd = toCommand(t.trim()); // 키워드→명령
  if (cmd) send(cmd);
};
rec.onend = () => rec.start(); // 끊기면 자동 재시작
rec.start();
```



동작

“전진/정지/왼쪽/오른쪽/후진”을 말하면 즉시 전송. 모델 다운로드 없이 실시간 동작.

도전 / 팁

toCommand()에서 ‘출발/스톱’ 등 표현 추가.
Chrome-인터넷 필요(iOS 미지원).

[실습 B] 키워드 매핑 & 사용법

toCommand()

```
function toCommand(text) {
  const t = text.toLowerCase();
  if (t.includes('전진') || t.includes('앞으로')) return 'forward';
  if (t.includes('후진') || t.includes('뒤로')) return 'backward';
  if (t.includes('왼') || t.includes('좌회전')) return 'left';
  if (t.includes('오른') || t.includes('우회전')) return 'right';
  if (t.includes('정지') || t.includes('멈')) return 'stop';
  return null;
}
```

사용 순서

- ① 포니봇 연결
- ② 듣기 시작
- ③ 말하면 바로 전달

안정화 팁

같은 명령 1.2초 내 중복은 무시. 단어를 짧고 또렷하게 말하면 정확.



[실습 B-옵션] 오프라인 음성 — Whisper

whisper.js

```
import { pipeline } from
  'https://cdn.jsdelivr.net/npm/@xenova/transformers@2.17.2';

// 정확도↑ : whisper-base / whisper-small
const asr = await pipeline('automatic-speech-recognition',
  'Xenova/whisper-base');

// 녹음한 16kHz 모노 오디오를 인식
const out = await asr(audio, { language:'korean', task:'transcribe' });
const cmd = toCommand(out.text);
if (cmd) send(cmd);
```



환경: 인터넷 연결 불필요 (오프라인)



인터넷 없이 실행



모델 다운로드 후 사용



개인정보 보호

동작

인터넷 음성서버 없이 브라우저에서 인식(오프라인 가능). 첫 실행 시 모델을 내려받습니다.

도전 / 팁

tiny→base/small로 바꾸면 한국어 정확 ↑ (대신 느려짐). 인터넷이 불안정한 환경에 적합.

[실습 C] 손 제스처 제어 — MediaPipe Hands

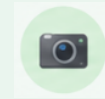
gesture.js

```
const hands = new Hands({ locateFile: f =>
  `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${f}` });
hands.setOptions({ maxNumHands:1, modelComplexity:0 });
hands.onResults(onResults);

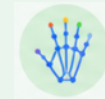
function onResults(res){
  const lm = res.multiHandLandmarks?.[0];
  let n = lm ? countFingers(lm) : -1;
  send(cmdFor(n));      // 안정화 후 전송
}
```



동작



카메라로 손을 인식

손랜드마크 추출 후
손가락 개수 계산개수에 따라 로봇에
명령 전송

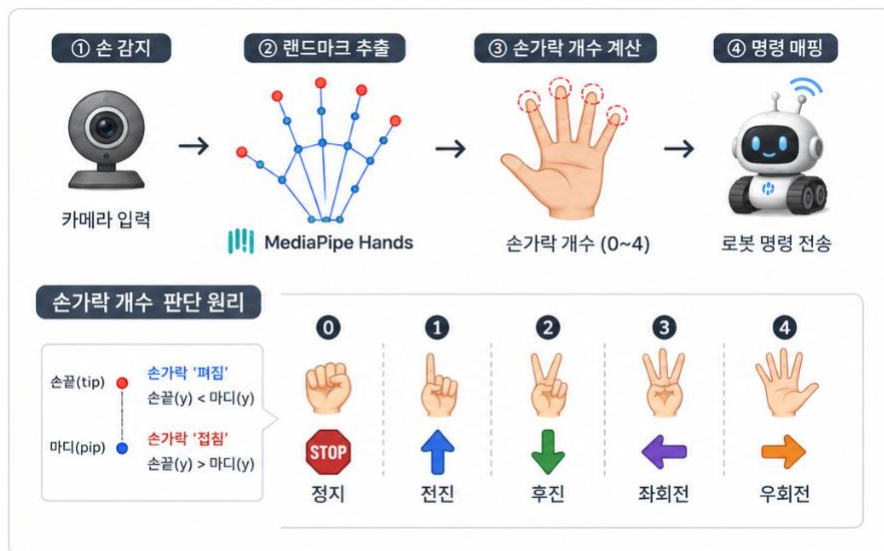
도전 / 팁

- Pose로 바꾸면 전신 동작 제어.
- 손 위치(좌/우)로 조종하는 방식도 가능.

[실습 C] 손가락 개수 인식 원리

countFingers()

```
function countFingers(lm){
  // 손끝 y < 마디 y → 그 손가락 '펴짐'
  const pairs=[[8,6],[12,10],[16,14],[20,18]];
  let c=0;
  for(const [tip,pip] of pairs)
    if(lm[tip].y < lm[pip].y) c++;
  return c; // 0~4
}
```



※ 손바닥을 카메라로, 손가락은 위로 향하면 인식이 잘 됩니다.

0개 / 손 없음 → 정지



1개 → 전진



2개 → 후진



3개 → 좌회전



4개 → 우회전



[실습 D] 객체 인식 자율주행 — 사람 추종

follow.js

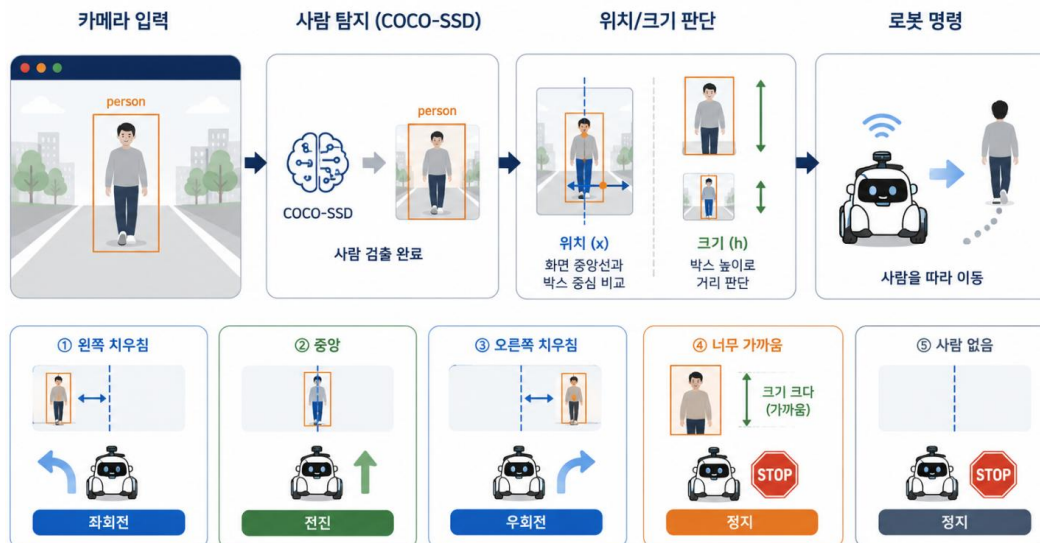
```
const model = await cocoSsd.load(); // TF.js COCO-SSD
async function loop(){
  const preds = await model.detect(video);
  const person = preds.find(p => p.class === 'person');
  if(!person){ send('stop'); }
  else {
    const [x,,w,h] = person.bbox;
    const cx = x + w/2, frac = cx / video.videoWidth;
    if(h > video.videoHeight*0.7) send('stop'); // 가까움
    else if(frac < 0.4) send('left');
    else if(frac > 0.6) send('right');
    else send('forward');
  }
  requestAnimationFrame(loop);
}
```

동작

카메라로 사람을 찾아 화면 가운데에 오도록 좌/우/전진하고, 가까우면 멈춥니다 (사람 추종).

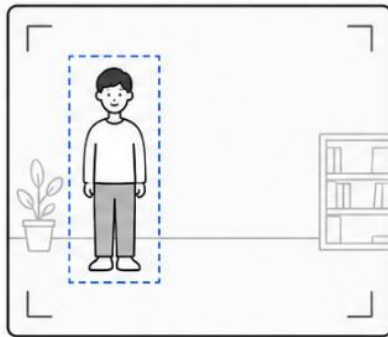
도전 / 팁

'person' 대신 다른 클래스로 물체 추종. 크기로 거리 추정해 속도 가변도 가능.



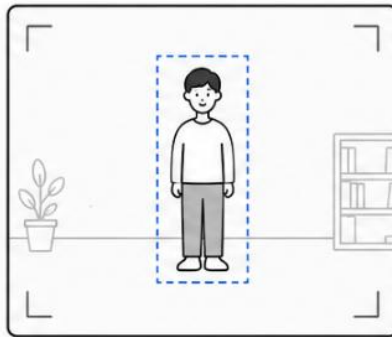
[실습 D] 사람 추종 — 동작 원리

화면 속 사람 박스의 '가로 위치'와 '크기'로 좌/우/전진/정지를 결정합니다.



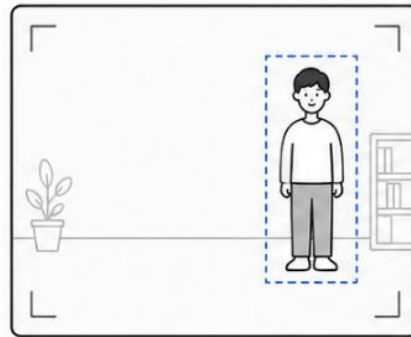
좌회전

`send('left')`



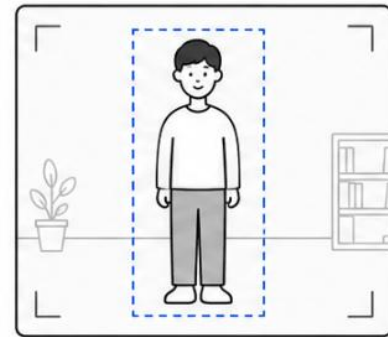
전진

`send('forward')`



우회전

`send('right')`



정지

`send('stop')`

팁

박스 크기로 거리를 추정해 '가까우면 정지/천천히'. 인식이 끊기면 안전하게 정지(stop).

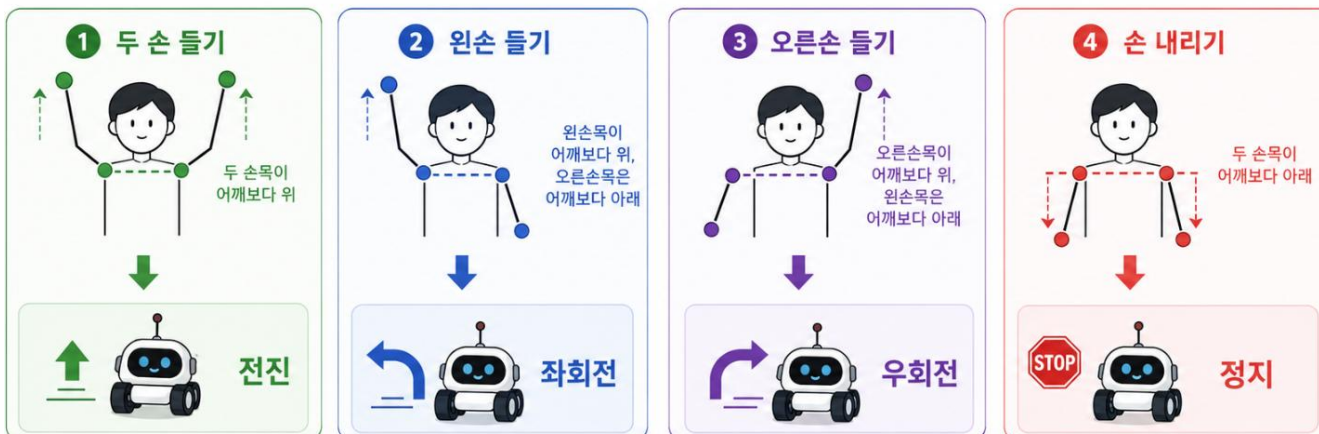
[실습 E] 포즈/제스처 모션 연동

pose.js

```
// MediaPipe Pose 결과(관절 좌표)로 제스처 판단
function onPose(res){
  const k = res.poseLandmarks; if(!k) return;
  const Lw = k[15], Rw = k[16]; // 양 손목
  const Ls = k[11], Rs = k[12]; // 양 어깨

  if(Lw.y < Ls.y && Rw.y < Rs.y) send('forward'); // 두 손 위로
  else if(Lw.y < Ls.y) send('left'); // 왼손만 위
  else if(Rw.y < Rs.y) send('right'); // 오른손만 위
  else send('stop');
}
```

어깨와 손목의 상대 위치로 동작 결정



동작

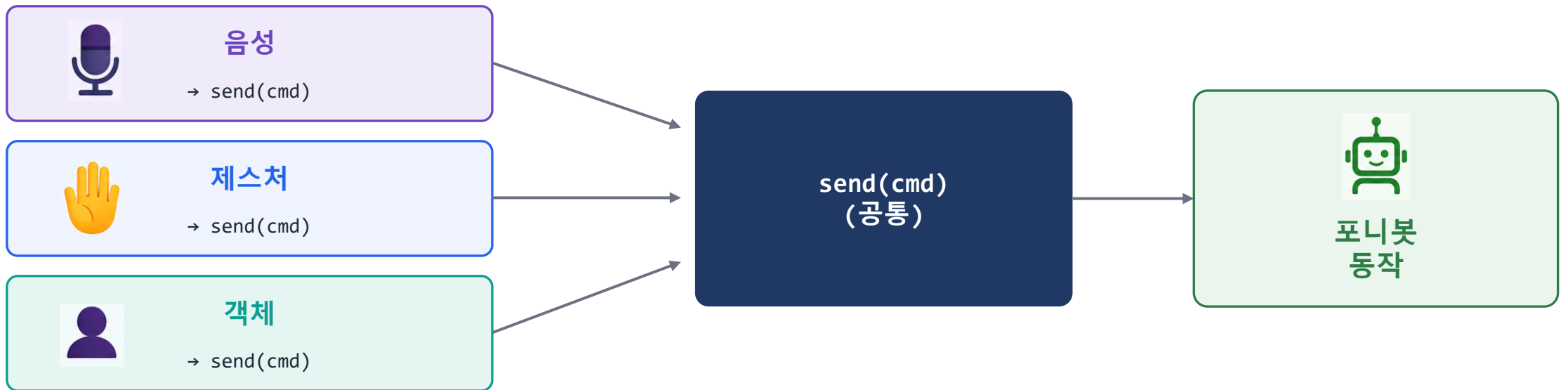
손을 들어 올린 자세로 로봇을 조종합니다. 두 손=전진, 한 손=회전, 내리면 정지.

도전 / 팁

어깨/엉덩이 기울기로 방향 판단, 서보(팔)와 연동해 '미러 로봇' 만들기.

[심화] 음성 + 제스처 통합 컨트롤

여러 입력을 한 페이지에서 함께 쓰면 더 똑똑한 컨트롤러가 됩니다. 모두 같은 `send()`를 호출합니다.

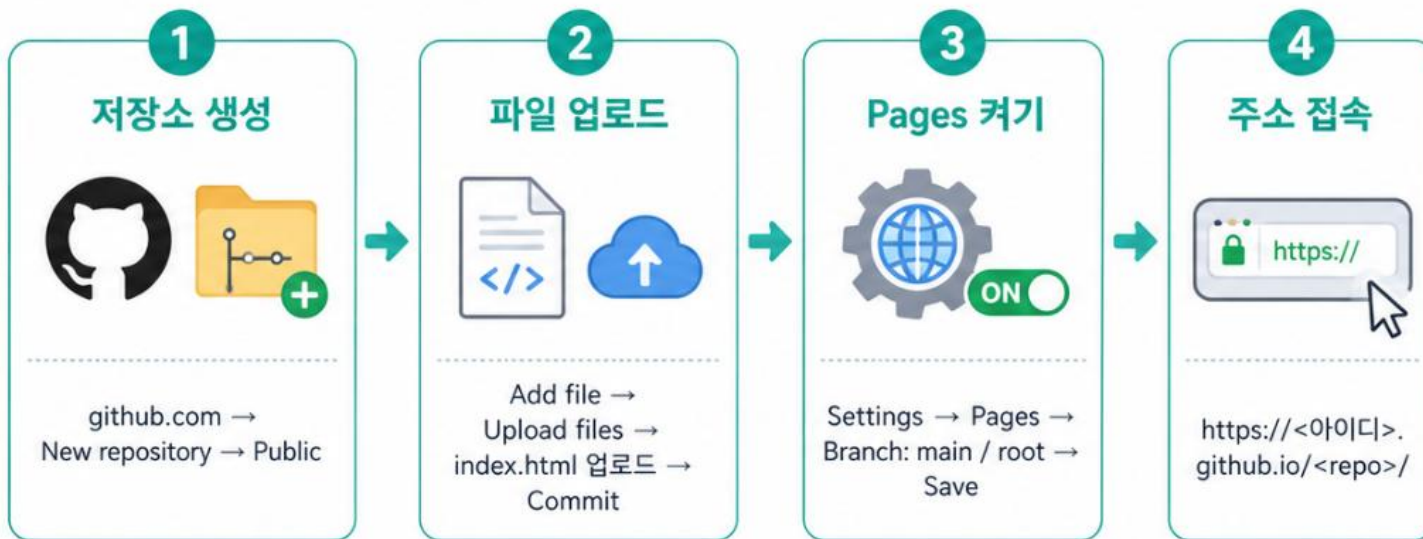


설계 방법

- 탭/버튼으로 모드 전환(음성/제스처/객체) 하거나, 동시에 켜고 우선순위 정하기
- 충돌 방지: 마지막에 들어온 명령 우선, 또는 '정지'는 항상 최우선
- 로봇 코드(robot.py)와 `send()`는 그대로 — 입력만 늘리면 됩니다.

배포 — GitHub Pages로 HTTPS 호스팅

웹앱(HTML)을 GitHub Pages에 올리면 자동으로 HTTPS로 안전하게 호스팅됩니다!



자동 HTTPS

안전하게 웹앱을 배포 완료!

https://<아이디>.github.io/<repo>/

마이크·카메라 사용 가능

Chrome/Edge 브라우저에서 주소를 열면 PC 없이도 바로 접속할 수 있어요!

모바일 실행 & 디버깅 체크리스트



권한 / 환경



- ✓ HTTPS 주소로 열기(http://IP는 마이크·카메라 차단)
- ✓ Chrome/Edge·안드로이드 Chrome 사용(iOS 미지원)
- ✓ 마이크·카메라 권한 '허용' (주소창 자물쇠 → 사이트 권한)
- ✓ 휴대폰 설정 → 앱 → Chrome → 권한 확인
- ✓ 블루투스 켜기, 포니봇 전원·코드 실행 중



안 될 때 점검



- ✓ 수동 버튼으로 BLE 연결·동작 먼저 확인
- ✓ ESP-000이 목록에 안 보임 → 포니봇 코드 실행/전원 확인
- ✓ ble_library import 오류 → 파일 업로드 확인
- ✓ 마이크 권한 거부 → HTTPS·권한 재확인
- ✓ 동작이 끊김 → 거리·배터리·간섭 확인



AI IDEAS

다양한 AI 예제 & 프로젝트

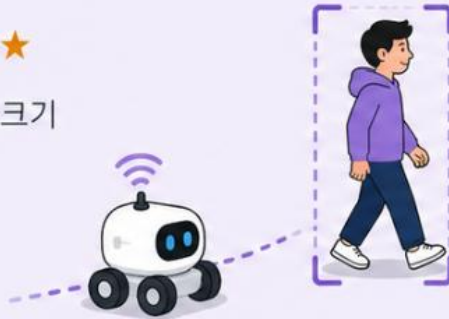
같은 '인식→명령→BLE' 구조에 여러 AI를 끼워 만드는 흥미로운 주제들. (★=난이도)

AI 예제 — 카메라(비전) ①

카메라 + TF.js/MediaPipe로 만드는 비전 프로젝트.

사람 추종 로봇 ★★

COCO-SSD로 사람 위치·크기
인식 → 따라다니기.
(실습 D 확장)



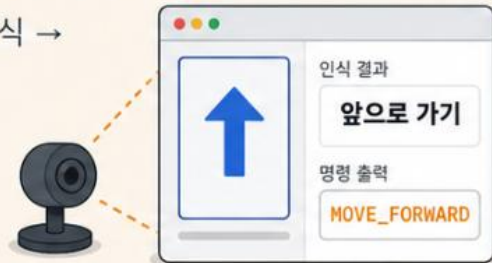
스마트 분류 물류 ★★★

MobileNet 물체 인식 +
라인·컬러 → 종류별
구역 배달.



Teachable Machine 인식기 ★

직접 학습한 카드·표지판 인식 →
명령. 커스텀 AI 입문 최고.



QR 미션 주행 ★★

jsQR로 QR 명령 읽어
자동 주행. 코스에
QR 배치.



AI 예제 — 카메라(비전) ②

얼굴·표정·시선을 쓰는 비전 프로젝트.

1 표정 반응 로봇



FaceMesh로 웃음/무표정 인식 → 다가오기/정지. OLED 표정 미러.

2 고개로 조종



고개 돌린 방향으로 주행 — 손이 자유로운 핸드프리.

3 졸음·주의 알림



눈 감김 감지 → 경고(LED·소리). 안전 응용.

4 색·표지판 추적



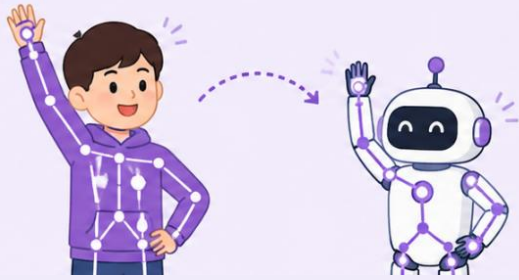
특정 색/표지판을 찾아 그쪽으로 이동.

AI 예제 — 포즈 · 제스처

MediaPipe Hands/Pose로 몸·손 동작을 명령으로.

미러 댄스 로봇 ★★

Pose로 팔 동작 따라하기(서보+주행). '따라쟁이' 로봇.



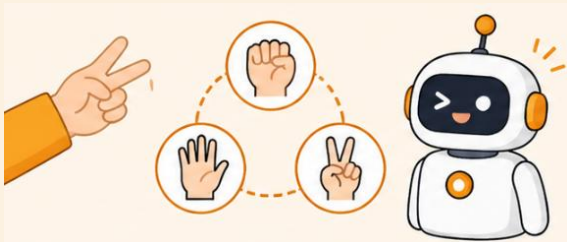
수신호 교통경찰 ★★

정지/진행/방향 손동작으로 제어. (제스처 앱 확장)



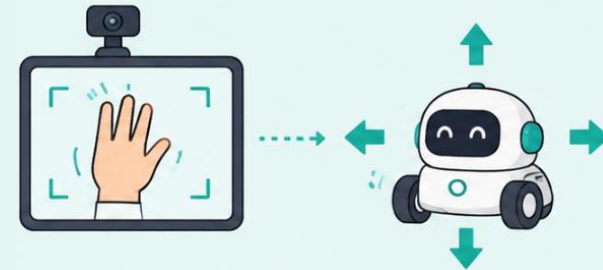
가위바위보 대결 ★

손 모양(가위/바위/보) 인식 → 승부 반응, OLED 표정.



손 위치 조종 ★

손이 화면 좌/우/위/아래 → 방향 제어. 직관적.



AI 예제 — 음성 · 언어

음성 인식·언어 모델로 '말로 부리는' 로봇.

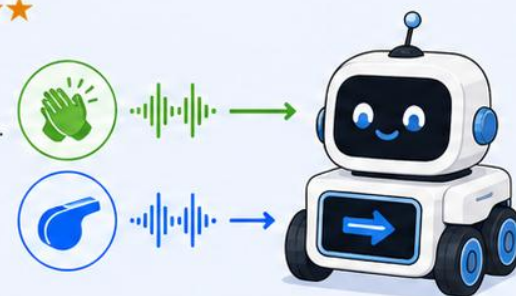
음성 비서 로봇 ★★

자유 발화 → 의도 파악 →
주행 + OLED 응답.



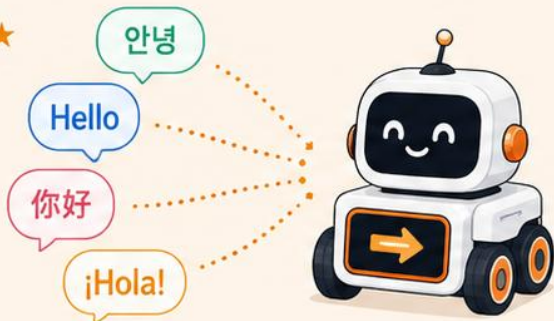
소리 패턴 제어 ★★

Teachable Machine
소리 모델(박수·휘파람) → 명령.



다국어 명령 ★★

Whisper 다국어로
여러 언어 명령을
같은 동작으로.



퀴즈·스토리 로봇 ★★★

음성 질문 →
(LLM API) 답변 →
OLED 표시·반응.



AI 예제 — 융합 · 게임 · 창작

센서 자율주행 + AI, 그리고 게임·창작 프로젝트.

감독형 자율주행 ★★★

센서로 자율주행 + 음성/제스처로 사람이 개입.



감정 동반 로봇 ★★★

표정+음성 톤 → 감정 추정 → LED·표정·움직임 반응.



AI 보물찾기/미로 ★★★

초음파 회피 자율 + 카메라로 목표(색/QR) 탐색.



드로잉 로봇 ★★

음성·제스처 지시대로 바닥에 도형 그리기(서보 펜).



AI 예제 — 교육 · 전시 · 생활

조금 더 확장한 응용 아이디어들.

① 전시 안내 로봇

이쪽으로
가시면 전시 2관
입니다!

인R 코드 스캔

전시 안내

QR/표지판 인식

코스를 돌며 안내·설명 (OLED·소리)

② 분리수거 도우미

플라스틱!
파란색 통에
넣어주세요.

인식 중...

인식 결과
V (Victory)

플라스틱 캔·금속 종이 일반 쓰레기

물체 종류 인식 (MobileNet)

종류별 구역으로 안내

③ 수화·손동작 학습기

정답이에요!
잘했어요!

퀴즈 문제

손모양 인식(Teachble Machine)

맞히기 게임으로 학습

④ 반려 로봇

오늘 기분이
좋아!

표정·음성 분석

기분이 좋다니
나도 행복해!

표정·음성 반응 (다가오기·표정·색)

정서 교감으로 친밀감 형성

프로젝트 선택 가이드



처음 도전하기 좋음 (★)

- Teachable Machine 인식기
- 가위바위보 로봇
- 손 위치 조종 / 사람 추종



발표 임팩트 큼 (★★★★)

- 스마트 분류 물류
- 감정 동반 로봇
- 감독형 자율주행 / AI 미로



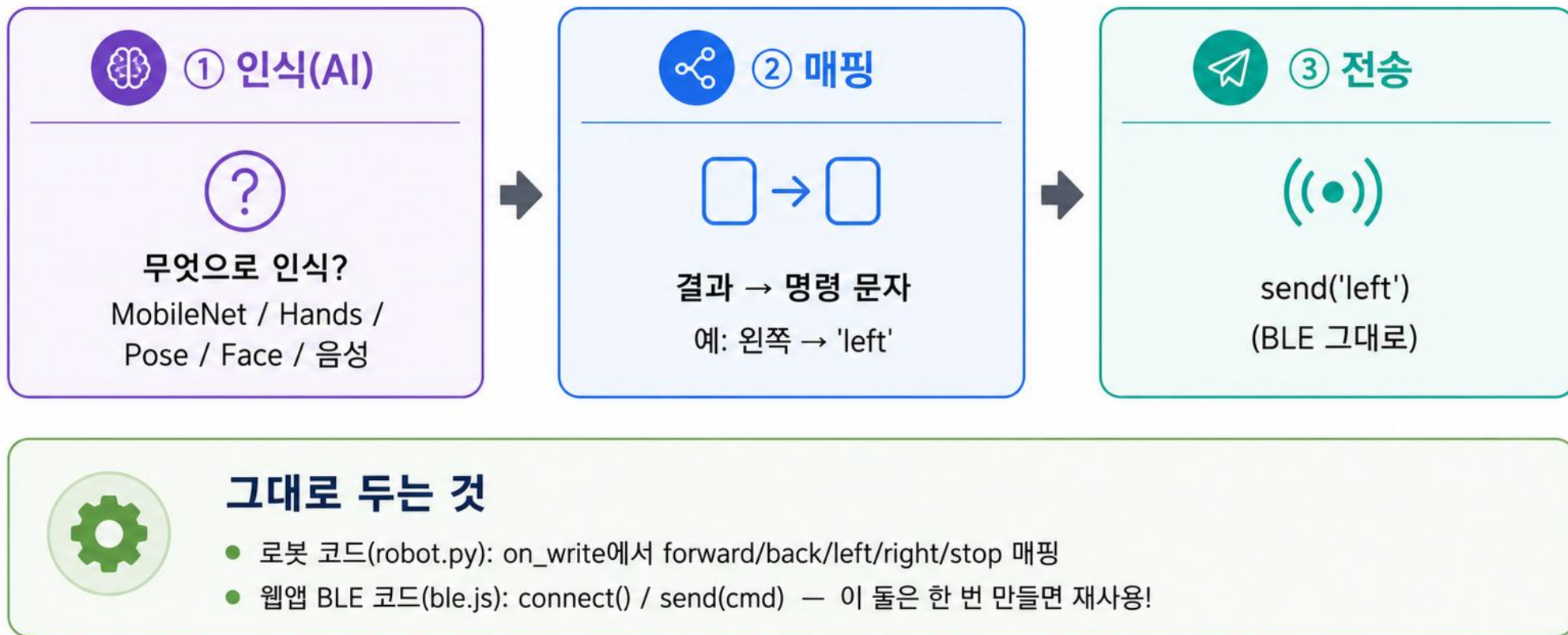
고를 때 체크

- ① 5시간 안에 '핵심 동작 1개'를 완성할 수 있나? (작게 시작)
- ② 가진 부품·센서·카메라로 가능한가?
- ③ 어떤 AI를 쓰고(인식), 어떤 명령으로 바뀌(매핑), 로봇이 무엇을 하나(동작)?
→ 이 세 가지만 정하면 바로 만들 수 있습니다.



새 AI 붙이는 공통 레시피

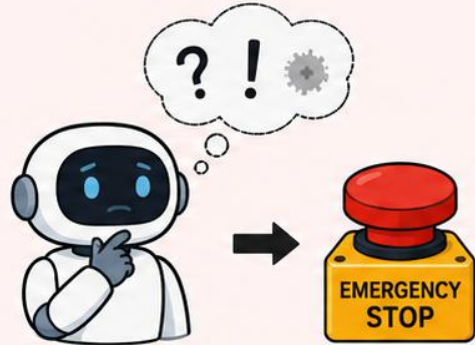
어떤 AI든 아래 3칸만 바꾸면 포니봇과 연결됩니다 — 나머지(BLE·로봇 코드)는 그대로!



AI 로봇의 안전 · 윤리 · 한계

인식 오류 대비

AI는 틀릴 수 있음 →
애매하면 '정지'가 기본.
비상 정지 수단 두기.



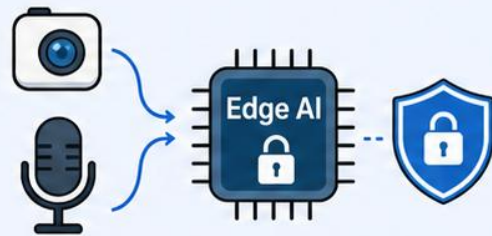
안전

사람·물체와 충돌 주의.
받침대 테스트 후
바닥 주행.
속도 적당히.



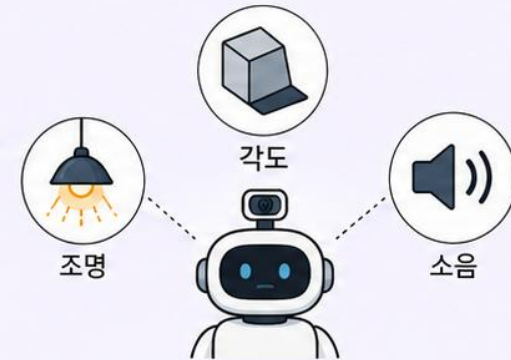
개인정보

카메라·음성은 기기 안에서
처리(Edge AI).
영상 저장·공유 주의.



한계 이해

조명·각도·소음에 영향.
학습 데이터에
없는 상황엔 약함.



참고 자료 & 도구 링크



TensorFlow.js

🌐 www.tensorflow.org/js

📄 브라우저 딥러닝 (MobileNet-COCO-SSD)



MediaPipe

🌐 developers.google.com/mediapipe

📄 손·얼굴·포즈 실시간 인식



Teachable Machine

🌐 teachablemachine.withgoogle.com

📄 코딩 없이 나만의 AI 학습



Web Bluetooth / Web Speech

🌐 developer.mozilla.org (MDN)

📄 브라우저 통신·음성 인식 문서

※ 우리가 만든 웹앱(음성·제스처)과 `robot.py` / `ble.js`는 그대로 재사용해 새 주제로 확장하세요.

4일차 정리 & 다음 차시

오늘 한 것

- AI 웹앱 ↔ 포니봇 구조와 Web Bluetooth(NUS-ESP-000) 통신 이해
- TF.js·MediaPipe·Web Speech 등 Edge AI와 여러 인식 모델
- BLE 연결부터 음성·손 제스처·객체 인식 자율주행까지 실제 웹앱 제작
- GitHub Pages 배포, 모바일 실행, 다양한 AI 예제와 '공통 레시피'

다음 차시 (5일차) 주행·센서·AI 제어를 모아 **나만의 AI 로봇 미니 프로젝트**를 기획·제작하고 발표합니다.

멀티모달 인지 기반 피지컬 AI 로봇 시스템 구현 과정

5일차 강의자료

미니 프로젝트 · 프로젝트 발표

1~4일차에 배운 주행·센서·AI 제어를 모아 '나만의 AI 로봇'을 만들고 발표합니다 — 과정의 완성!

실습 5H

발표 2H

팀 프로젝트

5일 과정 한눈에 보기 · 드디어 마지막 5일차!

1일차

AI 로봇 개론 + ESP32·MicroPython 기초

2일차

포니봇 플랫폼 이해 + 모션 제어

3일차

센서 통합·환경 인지 + 센서 자율주행

4일차

로봇 연동 AI 웹앱 + AI 기반 로봇 제어

5일차

미니 프로젝트 & 발표 (총정리)

※ 오늘은 주행(2일차)·센서 자율주행(3일차)·AI 웹앱 제어(4일차)를 '하나의 작품'으로 통합합니다.

[09] 5H 실습

미니 프로젝트

주제 선정 → 설계 → 팀별 구현 → 통합 테스트·디버깅 → 결과물 문서화. 배운 모든 기술을 하나로 모읍니다.

오늘의 진행 흐름

미니 프로젝트는 6단계로 진행합니다. (5시간 실습 + 2시간 발표)

① 주제 선정



무엇을 만들지
정하고 설계

② 설계



입력·처리·출력
구성도 작성

③ 구현



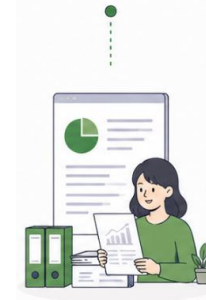
팀별 역할 분담
단계적 개발

④ 테스트



통합 시험 &
디버깅

⑤ 문서화



결과물 정리
(보고서)

⑥ 발표



시연 & 피드백

핵심 원칙

‘작게 시작해서 점점 키우기’ — 가장 기본 동작 하나를 먼저 완성하고, 되는 것을 확인한 뒤 기능을 하나씩 추가합니다.

5시간 타임박스 — 시간 배분

막판에 몰리지 않도록 시간을 미리 나눠 두세요. (대략의 가이드)

09:00-09:30	주제 확정 · 설계
09:30-10:00	시스템 구성 · 역할 분담
10:00-14:00	핵심 기능 구현 (점진적)
14:00-15:30	통합 테스트 · 디버깅
15:30-17:00	문서화 · 시연

※ 구현(10:00-14:00)에 가장 많은 시간을. 단, 14:00 이후엔 새 기능 추가를 멈추고 '되는 상태'를 다듬으세요.

프로젝트 주제 아이디어 (기본)

스마트 물류 로봇



라인 추종 + 컬러 구역 분류 + 장애물 회피

미로 탈출 로봇



초음파로 벽을 피하며 길 찾기

순찰·경비 로봇



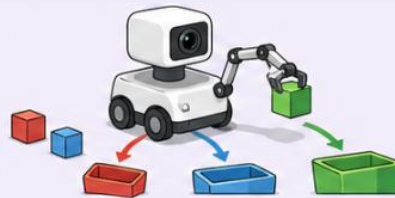
코스 주행 + 장애물 감지 + 경광등(LED)

라인 레이싱



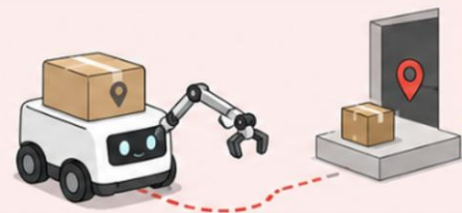
라인 트레이싱 속도 경쟁(P제어)

분류 도우미



컬러·물체 인식으로 구역 분류

배달 로봇



목표까지 주행 + 그리퍼로 집기·놓기

※ 1~3일차의 주행·센서 기술 중심 주제들입니다.

프로젝트 주제 아이디어 (AI 융합)

AI 안내·사람 추종



음성 명령 로봇



손 제스처 로봇



표정 반응 로봇



Teachable 인식기



감독형 자율주행



※ 4일차의 AI 웹앱(음성·제스처·객체)을 활용한 주제들 — 웹앱 + BLE로 포니봇 제어.

좋은 주제 고르기 & 선정 워크시트

아래 4가지를 점검하며 '5시간 안에 완성 가능한' 주제를 고릅니다.

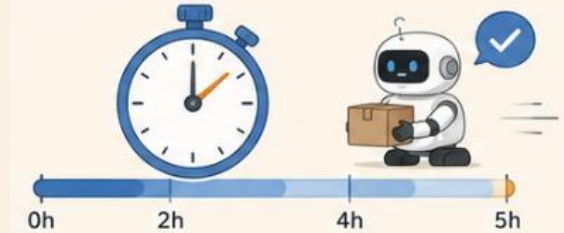
✓ 실현 가능성

지금 가진 부품·센서·카메라로 가능한가?



✓ 시간

5시간 안에 핵심 동작을 완성할 수 있나?



✓ 재미·의미

팀이 흥미를 느끼고 쓸모가 있나?



✓ 학습

배운 기술(주행·센서·AI)을 잘 활용하나?



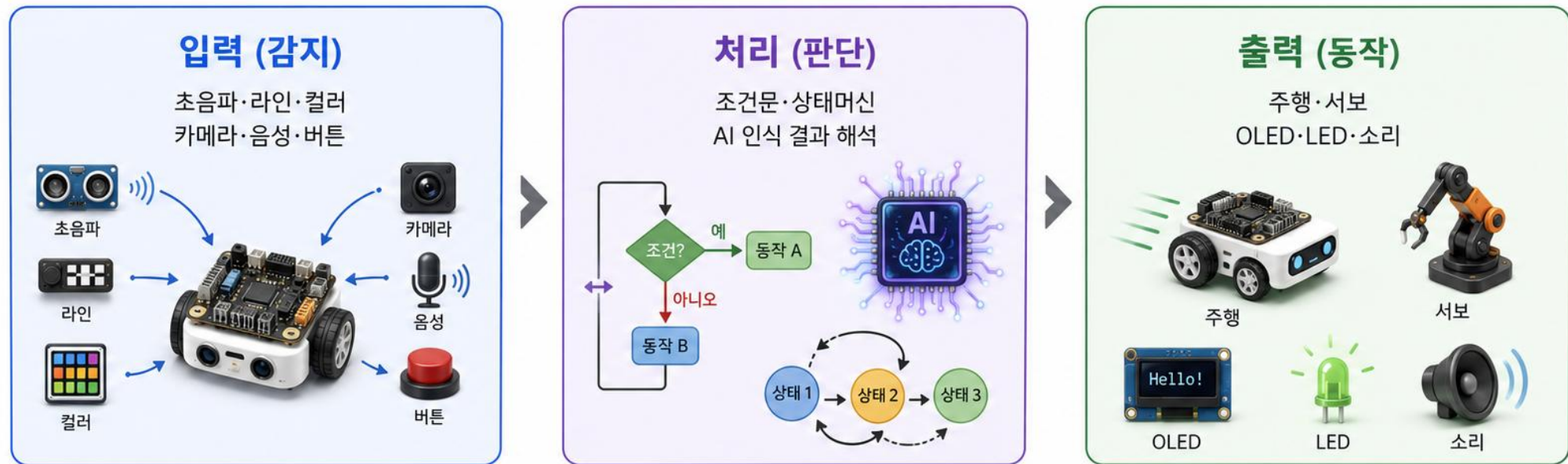
주제 선정 워크시트 (팀에서 한 문장씩 채우기)

- 우리 로봇은 (누구)를 위해 (무엇)을 한다.
- 핵심 기능 1가지: • 사용 기술: 주행 / 센서() / AI()



프로젝트 설계 — 입력 · 처리 · 출력

로봇이 하는 일을 '무엇을 입력받아(감지) → 어떻게 판단해(처리) → 무엇을 하는가(출력)'로 정의합니다.



왜 이렇게 나누나

이 세 칸만 정하면 코드의 뼈대(감지→판단→동작)가 나오고, 역할 분담·테스트도 쉬워집니다.



감지



판단

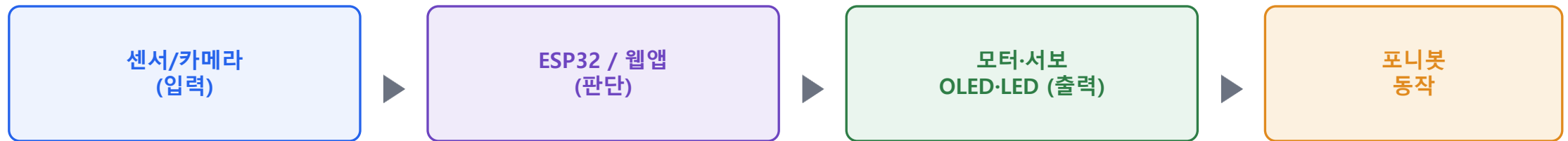


동작



시스템 구성도 그리기 (템플릿)

부품과 정보 흐름을 그림으로 정리하면 팀이 같은 그림을 보고 협업할 수 있습니다.



구성도에 넣을 것

- 사용하는 입력(센서·카메라·마이크)과 연결 핀/방식
 - 판단 위치(포니봇 코드인지, 웹앱 AI인지)와 통신(BLE)
 - 출력(어떤 동작을 하는가)
 - AI를 쓰면: '웹앱(인식) → BLE 명령 → 포니봇(동작)' 흐름을 함께 표시
- 종이에 손으로 그려도 충분합니다 — 화살표로 '정보가 흐르는 방향'만 분명히!

빌딩 블록 ① — 주행 · 센서 (1~3일차)

지금까지 배운 기능을 레고처럼 조합하세요. 필요한 것만 골라 씁니다.

주행 (2일차)

drive() / mecanum() / move()
전후·횡·대각·회전·속도

초음파 (3일차)

거리 측정 → 장애물 회피·거리 유지

라인 (3일차)

검은 선 추종·정지선 감지(P제어)

컬러 (3일차)

색 인식 → 구역 분류·신호등

표시·구동

OLED·LED 매트릭스·서보(그리퍼)

입력

버튼 A/B · 마이크(소리)

빌딩 블록 ② — AI 웹앱 (4일차)

스마트폰/PC의 AI가 인식 → BLE 명령 → 포니봇 동작. 새 AI도 '인식→명령'만 바꾸면 됩니다.

음성 (Web Speech)

"전진/정지" 말로 조종
실시간·무설치

손 제스처 (Hands)

손가락 개수·손짓으로 제어

객체 (COCO-SSD)

사람·물체 위치 → 추종

포즈 (Pose)

전신 동작 → 모션 연동

표정 (FaceMesh)

표정·고개 방향 → 반응

Teachable Machine

내가 학습시킨 커스텀 인식

주제 설계 예시 ① — 스마트 물류 로봇

라인을 따라가며 구역을 분류하고 장애물을 피하는 자율 물류 로봇. 난이도 ★★★

입력 (감지)

라인 센서(경로), 컬러 센서(구역), 초음파(장애물)

처리 (판단)

선을 따라가다 빨강 구역이면 '하역(정지)', 앞이 막히면 정지 — 상태로 구분

출력 (동작)

주행(라인 추종), OLED에 현재 구역·상태 표시

사용 기술 2일차 주행 + 3일차 라인·컬러·초음파 (센서 퓨전)

주제 설계 예시 ② — AI 안내 / 사람 추종

카메라로 사람을 찾아 따라다니는 안내 로봇. 난이도 ★★

입력 (감지)

스마트폰 카메라 영상 (웹앱)

처리 (판단)

COCO-SSD로 사람 위치·크기 인식 → 좌/우/전진/정지 명령 결정(웹앱)

출력 (동작)

BLE로 명령 전송 → 포니봇 주행, OLED로 상태 표시

사용 기술 4일차 객체 인식(COCO-SSD) + Web Bluetooth + 2일차 주행

주제 설계 예시 ③ — 제스처 / 음성 제어 로봇

손짓이나 말로 조종하는 핸드프리 로봇. 난이도 ★★

입력 (감지)

스마트폰 카메라(손) 또는 마이크(음성)

처리 (판단)

MediaPipe Hands(손가락 개수) 또는 Web Speech(키워드) → 명령 변환(웹앱)

출력 (동작)

BLE로 명령 전송 → 포니봇 주행/서보 동작

사용 기술 4일차 손 제스처 / 음성(Web Speech) + Web Bluetooth

팀별 구현 — 역할 분담 & 개발 순서

역할 분담

하드웨어 조립·배선·센서/카메라 장착

로봇 SW 포니봇 코드(주행·센서·BLE)

AI 웹앱 인식·BLE 연동(필요 시)

문서·발표 기록·보고서·발표 준비

개발 순서 (점진적)

- ① 가장 기본 동작 1개부터 완성 (예: 직진, 연결)
- ② 센서·AI를 하나씩 추가하며 매번 테스트
- ③ 잘 되는 버전을 저장(백업)해 두기
- ④ 마지막에 OLED·발표용 표시 다듬기

핵심: '되는 것'을 늘려가기. 한 번에 다 만들려 하지 않기!

AI 웹앱 ↔ 포니봇 빠른 연결 (4일차 복습)

AI 주제라면 이 두 조각만 있으면 됩니다 — 포니봇은 명령을 받고, 웹앱은 명령을 보냅니다.

robot.py (포니봇)

```
from ai_ponybot import i2c, PonyMotor
import ble_library, bluetooth
motor = PonyMotor(i2c)
p = ble_library.BLESimplePeripheral(
    bluetooth.BLE(), 'ESP-000')
def on_rx(v):
    c = v.strip().lower()
    if c=='forward': motor.drive('forward',50)
    elif c=='left': motor.drive('left',45)
    elif c=='right': motor.drive('right',45)
    else: motor.drive('stop')
p.on_write(on_rx)
while True: pass
```

ble.js (웹앱)

```
// 연결 후 특성 얻기
rxChar = await svc.getCharacteristic(RX);

// AI 인식 결과 → 전송
function send(cmd){
    rxChar.writeValueWithoutResponse(
        new TextEncoder().encode(cmd));
}

// 예: 손가락 1개 인식 →
send('forward');
```

통합 스타터 코드 (감지 → 판단 → 동작)

센서 기반 주제는 이 골격에서 시작합니다. 필요한 센서·동작만 바꿔 확장하세요.

starter.py

```
from ai_ponybot import i2c, PonyMotor, PonyOLED, PonySonar
from machine import Pin
from time import sleep

motor = PonyMotor(i2c); oled = PonyOLED(i2c)
sonar = PonySonar()
trig = Pin(18, Pin.OUT); echo = Pin(19, Pin.IN)

def sense():                # ① 감지
    return sonar.measure(trig, echo)

def decide(d):              # ② 판단
    if d != -1 and d < 15: return 'stop'
    return 'go'

def act(cmd):               # ③ 동작
    if cmd=='go': motor.drive('forward', 50)
    else:          motor.drive('stop')

while True:
    d = sense(); cmd = decide(d); act(cmd)
    oled.clear(); oled.write_line(0, cmd); oled.show()
    sleep(0.05)
```

확장 방법

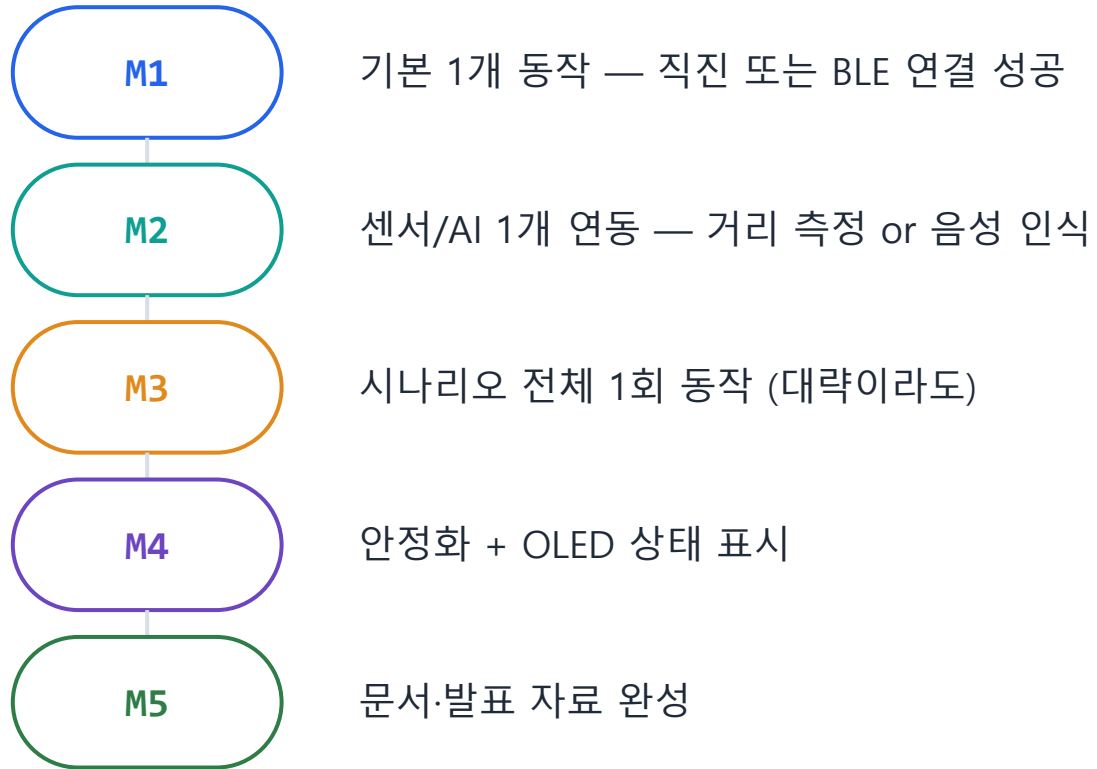
- sense()에 라인·컬러 추가
- decide()에 규칙·상태 추가
- act()에 서보·OLED 추가

AI 연동 시

decide() 대신 웹앱이 보낸 명령으로
act() 호출 → AI 제어로 확장(슬라이드 17).

개발 마일스톤 — 체크포인트

'되는 지점'을 단계로 정해두면 막혔을 때 어디로 되돌아갈지 분명합니다.



※ 각 마일스톤마다 코드를 저장(백업). 새 시도가 망가지면 마지막 성공 버전으로 복귀!

통합 테스트 & 디버깅

부분이 다 되어도 '합치면' 안 될 수 있습니다. 단계적으로 시험하고 문제를 좁힙니다.

테스트 전략

- 부품별로 따로 테스트 (모터만 / 센서만)
- 그다음 둘씩 합쳐서 테스트
- 전체를 합쳐 시나리오대로 시험
- 바뀌는 받침대 위에서 먼저 확인
- 한 번에 한 가지만 바꾸기

기본 디버깅 도구

- `print()`로 센서값·상태 찍어보기
- `i2c.scan()`으로 연결 장치 주소 확인
- OLED에 현재 상태 띄워 눈으로 확인
- 수동 버튼/명령으로 한 동작씩 검증
- 값(속도·임계값·sleep) 하나씩 조절

자주 막히는 문제 Top (빠른 해결)

● 반응이 없음

전원·USB·핀 번호 확인. 코드가 실행 중인지 확인.

● 센서값 이상/0

i2c.scan()으로 주소 확인, 배선·방향 점검.

● BLE 연결 안 됨

포니봇 코드 실행·전원 확인, ble_library 업로드 확인, ESP-000 검색.

● 마이크·카메라 거부

HTTPS 주소로 열기(GitHub Pages), 권한 '허용'.

● 동작이 반대로

방향코드·부호·좌우 모터 매핑 점검.

● 너무 빠름/느림

속도·sleep·임계값 조절.

결과물 문서화 (보고서)

나중에 다시 보고 발전시킬 수 있도록, 아래 항목을 한 장으로 정리합니다.

① 프로젝트 개요

주제·목표·누구를 위한 로봇인가

② 시스템 구성

사용한 부품·센서·AI·구성도

③ 핵심 코드

주요 동작 코드와 간단한 설명

④ 결과

무엇이 동작했는지(사진·영상)

⑤ 한계

잘 안 된 점·어려웠던 점

⑥ 개선 아이디어

다음에 더 해보고 싶은 것

[10] 1.5H 발표

프로젝트 발표

팀별 프로젝트를 발표하고, 종합 피드백으로 개선점을 찾으며, 다음 학습 로드맵을 안내합니다. (발표 & 피드백)

발표 자료 구성 템플릿

발표 슬라이드는 5장 내외로 간결하게. 아래 구성을 추천합니다.

1. 제목·팀

프로젝트 이름·팀원·한 줄 소개

2. 문제·주제

왜 만들었나, 무엇을 해결하나

3. 시스템 구성

구성도 + 사용한 기술(주행·센서·AI)

4. 시연/결과

동작 사진·영상, 핵심 기능

5. 배운 점·개선

어려움·해결, 다음 계획

※ 글은 적게, 그림·시연 위주로. 한 슬라이드 한 메시지!

발표 준비 — 무엇을 말할까

발표는 '문제 → 해결 → 시연 → 배운 점' 순서로 (팀당 약 5분).



시연 체크리스트

배터리 충전 · 코드 최종본 업로드 · 시연 공간(선·장애물·조명) 준비 · 인터넷/HTTPS 주소 확인(AI 웹앱) · 안 될 때 대비 영상 준비 · 말하는 사람/조작하는 사람 분담

시연 시나리오 짜기

'무엇을 → 어떤 순서로 → 어떻게 보여줄지' 미리 대본을 만들면 실수가 줄어듭니다.

시나리오 예 (사람 추종)

① 연결 시연 → ② 사람이 좌/우로 이동 → ③ 로봇이 따라옴 → ④ 가까이 가면 멈춤 보여주기

시나리오 예 (음성)

① '전진' → 직진 → ② '정지' → 멈춤 → ③ '왼쪽/오른쪽' → 회전 → OLED 표시 보여주기

준비 팁

- 핵심 동작 2~3개만 확실하게 (욕심내지 않기)
- 실패 대비: 미리 찍어둔 '성공 영상'을 준비
- 시연 중 한 명은 설명, 한 명은 로봇 조작 — 역할 분담
- 환경 변수(조명·소음·바닥) 미리 확인

평가 & 피드백 기준 (루브릭)

평가 항목	배점	설명
기획·창의성	20	주제의 독창성과 문제 정의
기술 완성도	30	동작 안정성과 기능 구현 수준
기술 이해	20	센서·AI·제어 원리의 이해
협업	15	역할 분담과 팀워크
발표·시연	15	전달력과 시연 완성도

※ '완벽함'보다 '무엇을 시도하고 무엇을 배웠는가'를 중요하게 봅니다.

동료 피드백 — 함께 성장하기

다른 팀 발표를 보며 아래 3가지를 적어 주세요. 칭찬은 구체적으로, 제안은 따뜻하게!

좋았던 점

인상적이었던 기능·아이디어
'특히 __가 멋졌어요'

개선 아이디어

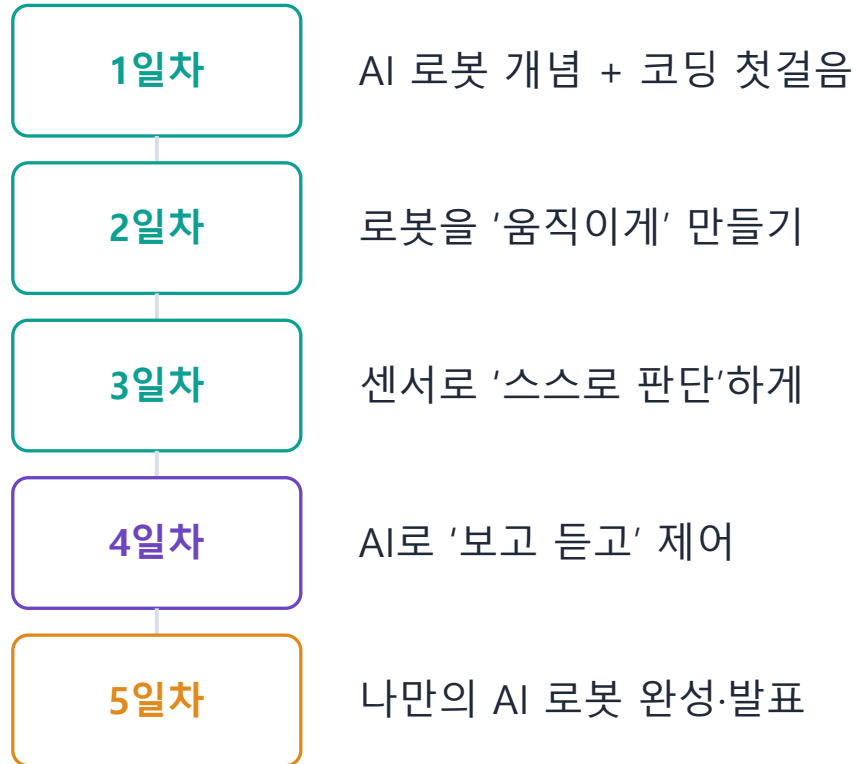
이렇게 하면 더 좋을 것 같아요
'__를 추가하면?'

궁금한 점

물어보고 싶은 질문
'__는 어떻게 했나요?'

피드백은 '평가'가 아니라 '서로 돕는 선물'입니다.

5일 과정 회고 — 우리가 해낸 것



센서·코딩·AI·로봇 제어를 모두 직접 다뤄본 '피지컬 AI 메이커'가 되었습니다. 정말 수고했습니다!

향후 학습 로드맵 (Lv.2 심화 연계)

이번 과정(Lv.1)에서 기초를 다졌습니다. 더 깊이 가고 싶다면 이런 길이 있습니다.

심화 자율주행

PID 정밀 제어, 경로 계획, SLAM

AI 모델 직접 학습

Teachable Machine → 나만의 인식 모델 제작

고급 컴퓨터 비전

객체 추적·세분화, 실시간 영상 처리

로봇 프레임워크

ROS, 다중 로봇·통신, 시뮬레이션

무엇보다, 직접 '만들어 보는 것'이 가장 빠른 길입니다. 작은 프로젝트를 계속 이어가 보세요!

5일 과정 수료

수고하셨습니다!

여러분은 이제 센서로 감지하고, 스스로 판단하며, AI로 제어하는
'멀티모달 인지 기반 피지컬 AI 로봇 시스템'을 직접 만들 수 있는 메이커입니다.

감지 (센서)

판단 (코딩)

AI 제어

창작 (프로젝트)